Universidad Simón Bolívar

Programa de Ingeniería de Sistemas

Ingeniería de Software I

Segundo examen práctico: Spec-Driven Development con IA

Caso ReparaCampus

Equipo: **[PENDIENTE: EQUIPO]**

Integrantes

Rafael Eduardo May Recuero · **[PENDIENTE: CODIGO]**

Jean Marco Oyola De Martino · **[PENDIENTE: CODIGO]**

José Leonardo Hernández Pedrosa · 22511273118

Cristian David Diaz España · **[PENDIENTE: CODIGO]**

Jorge Luis González Arroyo · **[PENDIENTE: CODIGO]**

Luis Carlo Daza Ospino · **[PENDIENTE: CODIGO]**

Grupo: **[PENDIENTE: GRUPO]**

Docente: **[PENDIENTE: DOCENTE]**

Fecha: 2026-10-13

Repositorio: https://github.com/LUCHOMAN0529/reparacampus

Tag release-examen · SHA b59faa1128708c5cc405d0b315c0226c610c5489

2. Índice

2. Índice 2
3. Problema y alcance 4
3.1 Necesidad 4
3.2 Actores y permisos 4
3.3 Requisitos y especificaciones 4
3.4 Alcance 5
3.5 Reglas que no pueden cambiarse 5
3.6 Preguntas al cliente 5
3.7 Supuestos pendientes de confirmar 6
3.8 Registro de revisión 7
4. Cinco SPECS 8
4.1 SPEC S01 · Registrar una incidencia 8
4.2 SPEC S02 · Priorizar y asignar 15
4.3 SPEC S03 · Atender una incidencia 22
4.4 SPEC S04 · Validar la solución: confirmar, rechazar y reabrir 29
4.5 SPEC S05 · Consultar, historizar y tablero 37
5. Diseño 45
5.1 Componentes 45
5.2 Modelo de datos 46
5.3 Estados y transiciones 48
5.4 Contratos compartidos 49
5.5 Flujo de validación de una operación 50
5.6 Decisión de arquitectura 51
5.7 Registro de revisión 51
5.8 ADR-001 · Reglas de negocio en una capa de dominio y servicios, con SQL directo sobre SQLite 51
6. IA e implementación 54
6.1 Herramientas y versiones 54
6.2 Cómo se usó la IA 54
6.3 Evidencia de las cinco funcionalidades 54
6.4 Bitácora de IA 55
6.5 Revisión asistida por IA de las SPECS v0.1 59
6.6 Segunda revisión asistida por IA de las SPECS (v0.2) 61
7. Validación 65
7.1 Ejecución final sobre el tag de entrega 65
7.2 Matriz de trazabilidad 67
7.3 Diez escenarios automatizados de referencia 68

7.4 Matriz SPEC frente a código 69
7.5 Comprobación manual 73
7.6 Hallazgos y regresión 74
7.7 ¿Las pruebas detectan errores? 74
7.8 Límites de las pruebas nuevas 76
8. Riesgos 77
8.1 R1 (IA) · La IA inventa una regla de negocio 77
8.2 R2 (IA) · Las pruebas aceptan el mismo error que el código 77
8.3 R3 (producto) · Un usuario accede a reportes ajenos 78
8.4 R4 (producto) · Historial engañoso tras una operación fallida 78
8.5 Riesgos residuales declarados 79
8.6 Situaciones abiertas del trabajo del equipo 79
9. Entrega reproducible 81
9.1 Repositorio y versión evaluada 81
9.2 Instalación desde una copia limpia 81
9.3 Arranque y seed 81
9.4 Cuentas ficticias 81
9.5 Comandos de pruebas 81
9.6 Limitaciones conocidas 82
10. Contribuciones y cierre 83
10.1 Contribuciones por integrante 83
10.2 Integración 83
10.3 Conclusión 83
10.4 Comentarios de los integrantes sobre las SPECS 84
10.5 Decisiones: confirmadas y pendientes 85
11. Referencias 89
11.1 Correspondencia con este repositorio 89

3. Problema y alcance

Tabla 1. *Problema y alcance*

Campo	Valor
Versión	0.5 (integrada en `main`; revisión de otro integrante pendiente)
Autor	Luis Carlo Daza Ospino, con asistencia de IA (Claude)
Revisor	Propuesto: Rafael Eduardo May Recuero. El equipo no ha confirmado la asignación; revisión pendiente
Fecha	2026-10-08

3.1 Necesidad

> *"Nos reportan un daño por un mensaje, lo atendemos por otro y a veces lo damos por cerrado sin saber si quedó resuelto".*

La coordinación de mantenimiento de la Universidad Simón Bolívar no tiene un registro único de las incidencias de infraestructura. Los reportes llegan por canales distintos, no queda constancia de quién los atiende y se cierran sin que la persona afectada confirme la solución.

El prototipo ReparaCampus debe permitir **registrar** reportes, **priorizarlos**, **asignar** su atención y **comprobar** la solución con quien reportó, dejando un historial que no pueda alterarse.

3.2 Actores y permisos

Tabla 2. *Actores y permisos*

Actor	Puede	No puede
Solicitante	Registrar incidencias; consultar las propias; confirmar, rechazar o reabrir la solución de las propias.	Ver o operar incidencias ajenas; asignar; atender.
Coordinador	Consultar todas las incidencias; asignar un técnico activo a una incidencia REGISTRADA; ver el tablero.	Registrar; atender; confirmar, rechazar o reabrir en nombre del solicitante; reasignar.
Técnico	Consultar sus asignaciones; iniciar la atención; registrar la solución.	Ver u operar incidencias no asignadas a él; cerrar; asignar.

El servidor obtiene identidad y rol de la sesión y comprueba rol y pertenencia en cada operación. Ocultar un botón no es un control.

3.3 Requisitos y especificaciones

Tabla 3. *Requisitos y especificaciones*

Requisito	Resumen	Historia	SPEC
RF01	Registrar una incidencia	H01	S01
RF02	Calcular prioridad y asignar técnico	H02	S02
RF03	Iniciar atención y registrar solución	H03	S03
RF04	Confirmar, rechazar y reabrir	H04	S04
RF05	Consultar, historizar y tablero	H05	S05

3.4 Alcance

Incluye: aplicación web con lógica en servidor y persistencia en SQLite; inicio y cierre de sesión con cinco cuentas ficticias (dos solicitantes, un coordinador, dos técnicos activos); las cinco funcionalidades anteriores; historial inmutable.

Excluye (límites del prototipo): adjuntos, notificaciones, hosting, IA dentro del producto, edición de reportes, cierre automático, reasignación de una incidencia ya ASIGNADA, restricción por especialidad del técnico, gestión de usuarios y autorregistro.

Catálogos fijos:

- Ubicaciones: `LAB-01`, `AULA-201`, `BIB-01`.
- Categorías: `ELECTRICIDAD`, `HIDRAULICA`, `MOBILIARIO`, `TIC`.
- Impacto: `BAJO`, `ALTO`.
- Prioridad (calculada): `CRITICA`, `ALTA`, `NORMAL`.
- Estados: `REGISTRADA`, `ASIGNADA`, `EN_ATENCION`, `PENDIENTE_VALIDACION`, `CERRADA`.

3.5 Reglas que no pueden cambiarse

1. Solo existen las transiciones descritas en el caso (ver diagrama de estados).
2. Una operación rechazada conserva estado e historial.
3. No hay cierre automático, edición de reportes ni reasignación.
4. El coordinador no confirma ni reabre en nombre del solicitante.
5. La reapertura se acepta si el tiempo desde el último cierre es menor o igual a 48 horas, calculado por el servidor con fechas UTC.
6. Los límites de texto se aplican después de retirar los espacios externos.
7. Estado e historial se guardan juntos, sin efectos parciales.
8. Los textos no se ejecutan como HTML ni como scripts.

3.6 Preguntas al cliente

Estas preguntas están **sin responder**. El 2026-10-08 el equipo informó que no puede consultar a la docente antes del cierre de la entrega: la única sesión con ella coincide con el día del parcial. Por eso el prototipo se construye con el supuesto indicado y cada pregunta se declara como pendiente. Una respuesta de la IA no es una decisión del cliente.

Tabla 4. *Preguntas al cliente*

ID	Pregunta	Por qué importa	Supuesto de trabajo	Respuesta del cliente
Q1	Cuando un solicitante o un técnico intenta abrir u operar una incidencia que no le pertenece, ¿el sistema debe decir que no tiene permiso o debe responder como si la incidencia no existiera?	Define si se revela la existencia de reportes ajenos y qué código de respuesta se prueba.	Se responde 404 (no encontrada), sin revelar que existe.	Sin respuesta; se mantiene el supuesto

ID	Pregunta	Por qué importa	Supuesto de trabajo	Respuesta del cliente
Q2	¿El porcentaje de cierre del tablero cuenta las incidencias que están CERRADAS en este momento o las que alguna vez se cerraron? ¿Cómo se redondea el decimal?	Cambia el resultado tras una reapertura y en valores como 1/3.	Cuenta el estado actual, de modo que reabrir reduce el porcentaje. Redondeo aritmético (mitad hacia arriba): 1/3 = 33,3 %; 2/3 = 66,7 %.	Sin respuesta; se mantiene el supuesto
Q3	¿Con qué precisión se mide el plazo de 48 horas de la reapertura y qué significa “un instante después”?	Determina el caso límite que se prueba.	El servidor compara marcas UTC con precisión de microsegundos: exactamente 48 h se acepta; 48 h más un microsegundo se rechaza.	Sin respuesta; se mantiene el supuesto

3.7 Supuestos pendientes de confirmar

Tabla 5. *Supuestos pendientes de confirmar*

ID	Supuesto	SPEC afectada
SUP-01	El código de la incidencia tiene el formato `INC-000001`, consecutivo y generado por el servidor.	S01
SUP-02	El riesgo para personas se envía como valor explícito `true` o `false`. Un valor ausente se rechaza: no se asume `false`.	S01
SUP-03	Los valores de catálogo se comparan de forma exacta, con distinción de mayúsculas: `alto` se rechaza.	S01
SUP-04	La longitud de un texto se cuenta en caracteres después de retirar espacios externos; los espacios y saltos de línea interiores cuentan.	S01, S03, S04
SUP-05	Un técnico solo ve las incidencias que tiene asignadas, no las que están sin asignar.	S03, S05
SUP-06	Un técnico es “activo” si su cuenta está marcada como activa. Una cuenta inactiva no puede iniciar sesión ni recibir asignaciones.	S02
SUP-07	El plazo de reapertura se cuenta desde el cierre más reciente; cada reapertura y nuevo cierre reinicia el plazo.	S04
SUP-08	El coordinador ve el detalle y el historial completo de todas las incidencias, pero solo puede asignar.	S05
SUP-09	Los filtros de estado y prioridad se combinan con “y”. Un valor de filtro fuera del catálogo se rechaza con error en lugar de devolver una lista vacía.	S05
SUP-10	En el tablero, “CRITICA no cerradas” lista las incidencias con prioridad CRITICA cuyo estado actual no es CERRADA.	S05
SUP-11	Las contraseñas de las cinco cuentas ficticias se publican en el README, porque son datos de prueba.	Autenticación
SUP-12	La «solución vigente» de una incidencia es la última registrada; es la que se confirma o se rechaza.	S04
SUP-13	El historial se muestra en el orden en que se registraron los eventos; ante dos eventos con la misma marca de tiempo, el desempate es su identificador consecutivo.	S05

ID	Supuesto	SPEC afectada
SUP-14	Registros enviados a la vez se atienden uno tras otro; todos se crean, con códigos consecutivos distintos. **Alcance de la evidencia:** se comprobó con ocho registros simultáneos, desde ocho hilos de un mismo proceso, contra un archivo SQLite (P-S01-11). No es una prueba de carga ni una garantía general de concurrencia: con más carga, un registro podría agotar la espera del bloqueo de SQLite (5 segundos por defecto) y fallar con error, sin dejar datos parciales; ese caso no está probado.	S01

3.8 Registro de revisión

Tabla 6. *Registro de revisión*

Fecha	Versión	Revisor	Decisión	Observaciones
Pendiente	0.5	Rafael Eduardo May Recuero (propuesto)	Pendiente	

4. Cinco SPECS

Cada especificación incluye su historia, criterios de aceptación, plan de diseño, tareas, versión y registro de revisión. Los archivos fuente están en la carpeta specs/ del repositorio.

4.1 SPEC S01 · Registrar una incidencia

Tabla 7. *SPEC S01 · Registrar una incidencia*

Campo	Valor
Versión	0.5 (integrada en `main`; revisión de otro integrante pendiente)
Autor	Luis Carlo Daza Ospino, con asistencia de IA (Claude)
Revisor	Propuesto: Rafael Eduardo May Recuero. El equipo no ha confirmado la asignación; revisión pendiente
Fecha	2026-10-08
Requisito asociado	RF01

Cambios de la versión 0.2 (2026-10-08): se agregan los límites 500 y 501 a la prueba P-S01-02. Origen: revisión asistida por IA, que no reemplaza la revisión del integrante asignado.

Cambios de la versión 0.3 (2026-10-08): se precisa cómo se interpreta el riesgo enviado por el formulario; nuevo AC-S01-10 (registros simultáneos); la tabla de pruebas refleja las pruebas reales e identifica las que faltan. Origen: segunda revisión asistida por IA, que tampoco reemplaza la revisión del integrante asignado.

Cambios de la versión 0.4 (2026-10-08): las pruebas P-S01-11 y P-S01-12 ya están automatizadas y se registran con su nombre real; se anota el alcance comprobado de AC-S01-10. No cambia ningún requisito ni criterio.

Cambios de la versión 0.5 (2026-10-08): sin cambios de requisitos ni de criterios. La SPEC queda integrada en `main` junto con el código y las pruebas; las referencias a pruebas ya no dependen de una rama; el revisor figura como propuesto, porque el equipo no ha confirmado la asignación.

4.1.1 Historia (H01)

Como **solicitante**, quiero **registrar una incidencia de infraestructura con su ubicación, categoría, descripción, impacto y riesgo para personas**, para **que mantenimiento la conozca por un único canal y quede constancia de mi reporte**.

4.1.2 Alcance y exclusiones

Incluye: formulario de registro, validación en servidor, generación de código, autor, fecha, estado y prioridad, y el evento de creación en el historial.

Excluye: adjuntos, edición o eliminación del reporte después de creado, registro en nombre de otra persona y notificaciones. La regla de cálculo de la prioridad se especifica en S02.

4.1.3 Entradas

Tabla 8. *Entradas*

Campo	Tipo	Obligatorio	Valores o límites
`ubicacion`	texto	Sí	Exactamente `LAB-01`, `AULA-201` o `BIB-01`.

Campo	Tipo	Obligatorio	Valores o límites
`categoria`	texto	Sí	Exactamente `ELECTRICIDAD`, `HIDRAULICA`, `MOBILIARIO` o `TIC`.
`descripcion`	texto	Sí	De 20 a 500 caracteres después de retirar espacios externos.
`impacto`	texto	Sí	Exactamente `BAJO` o `ALTO`.
`riesgo_personas`	booleano	Sí	El formulario envía el texto `true` (Sí) o `false` (No). Cualquier otro valor se rechaza: `True`, `1`, `on`, `si`, vacío o campo ausente. No hay valor por defecto.

El cliente no envía código, autor, fecha, estado ni prioridad. Si los envía, el servidor los ignora.

4.1.4 Precondiciones y permisos

- Sesión iniciada. Sin sesión, el servidor redirige al inicio de sesión y no crea nada.
- Rol `SOLICITANTE`. Coordinador y técnico reciben 403.
- El autor es siempre el usuario de la sesión.

4.1.5 Reglas y proceso

1. Comprobar sesión y rol.
2. Retirar espacios externos de la descripción.
3. Validar los cinco campos. Si alguno falla, responder 400 con la lista de errores por campo y no guardar nada.
4. Calcular la prioridad con la regla de S02.
5. En una sola transacción: insertar la incidencia con estado `REGISTRADA`, autor y fecha UTC del servidor; generar el código `INC-` más seis dígitos consecutivos; insertar el evento `CREAR`.
6. Confirmar la transacción y redirigir al detalle de la incidencia.

4.1.6 Salidas y cambios persistidos

- Una fila en `incidencias` con `codigo`, `solicitante_id`, `ubicacion`, `categoria`, `descripcion`, `impacto`, `riesgo_personas`, `prioridad`, `estado = REGISTRADA`, `creada_en`.
- Una fila en `eventos` con `accion = CREAR`, `actor_id` = autor, `estado_anterior` vacío, `estado_nuevo = REGISTRADA` y la misma fecha.
- Respuesta: redirección 302 al detalle, donde se ve el código generado.

4.1.7 Errores y efectos que deben evitarse

Tabla 9. *Errores y efectos que deben evitarse*

Situación	Respuesta	Efecto
Sin sesión	302 al inicio de sesión	Ninguno
Rol distinto de solicitante	403	Ninguno
Campo ausente, fuera de catálogo o fuera de límites	400 con mensaje por campo	Ninguno

No debe ocurrir: una incidencia sin evento de creación o un evento sin incidencia; texto del usuario interpretado como HTML; autor, fecha, estado o prioridad tomados de la petición.

4.1.8 Criterios de aceptación

AC-S01-01 (funcionamiento normal). Dado el solicitante `solicitante1` con sesión iniciada y ninguna incidencia registrada, cuando envía ubicación `LAB-01`, categoría `ELECTRICIDAD`, descripción "El tomacorriente del puesto 4 no tiene energía", impacto `BAJO` y riesgo `false`, entonces se crea una incidencia con código `INC-000001`, autor `solicitante1`, fecha UTC del servidor, estado `REGISTRADA` y prioridad `NORMAL`, y su historial tiene exactamente un evento `CREAR` de `solicitante1` hacia `REGISTRADA`.

AC-S01-02 (permisos). Dado un usuario con rol coordinador o técnico y sesión iniciada, cuando envía un registro con datos válidos, entonces recibe 403 y no se crea incidencia ni evento. Dado un visitante sin sesión, cuando envía el registro, entonces es redirigido al inicio de sesión y no se crea nada.

AC-S01-03 (límites de la descripción). Dado un solicitante con sesión iniciada, cuando envía una descripción de 19 caracteres, entonces recibe 400 y no se crea nada; cuando envía una de 20 caracteres, entonces se crea; cuando envía una de 500, entonces se crea; cuando envía una de 501, entonces recibe 400. Los espacios externos no cuentan: 19 caracteres rodeados de espacios se rechazan.

AC-S01-04 (impacto inválido). Dado un solicitante con sesión iniciada, cuando envía impacto `MEDIO`, `alto` o vacío, entonces recibe 400 con un error en el campo impacto y no se crea incidencia ni evento.

AC-S01-05 (riesgo inválido). Dado un solicitante con sesión iniciada, cuando envía riesgo `quiza` o no envía el campo, entonces recibe 400 con un error en el campo riesgo y no se crea incidencia ni evento.

AC-S01-06 (catálogos). Dado un solicitante con sesión iniciada, cuando envía una ubicación o una categoría que no pertenece al catálogo, o no la envía, entonces recibe 400 y no se crea nada.

AC-S01-07 (texto no ejecutable). Dado un solicitante con sesión iniciada, cuando registra la descripción `<script>alert(1)</script> la silla está rota`, entonces la descripción se guarda literal y, al mostrar el detalle, la respuesta contiene `<script>` y no contiene la etiqueta `<script>alert(1)</script>`.

AC-S01-08 (campos generados por el servidor). Dado un solicitante con sesión iniciada, cuando envía datos válidos junto con `codigo`, `estado=CERRADA`, `prioridad=CRITICA`, `solicitante_id` de otro usuario o una fecha, entonces esos valores se ignoran: el autor es el de la sesión, el estado es `REGISTRADA`, la prioridad es la calculada y la fecha es la del servidor.

AC-S01-09 (sin efectos parciales). Dado un fallo al guardar el evento de creación, cuando el servidor procesa el registro, entonces la transacción se revierte y no queda la incidencia.

AC-S01-10 (registros simultáneos). Dadas varias solicitudes de registro válidas enviadas al mismo tiempo, cuando el servidor las procesa, entonces todas se crean, cada una con un código distinto y consecutivo y con su propio evento `CREAR`; nunca hay dos incidencias con el mismo código ni una incidencia sin evento. Alcance comprobado: ocho solicitudes simultáneas en un mismo proceso contra SQLite (ver SUP-14); no se ha probado con más carga.

4.1.9 Diseño y tareas vinculadas a cada AC

Diseño en plan.md; tareas en tasks.md.

Tabla 10. *Diseño y tareas vinculadas a cada AC*

AC	Tareas
AC-S01-01	T-S01-01, T-S01-03, T-S01-04, T-S01-05
AC-S01-02	T-S01-02, T-S01-04
AC-S01-03 a 06	T-S01-03
AC-S01-07	T-S01-05
AC-S01-08, 09	T-S01-04
Todos	T-S01-06
AC-S01-10	T-S01-07

4.1.10 Pruebas y resultados esperados

El resultado esperado se define aquí, antes de ejecutar, y no se calcula con la función de producción. Salvo que se indique otra cosa, cada prueba parte de una base SQLite nueva y aislada que solo contiene las cinco cuentas del seed, con el reloj del servidor fijado en `2026-10-01T08:00:00Z`. Todas las pruebas de esta tabla existen en `tests/` y se ejecutaron dentro de la batería completa. Los resultados están en `docs/validacion/validacion.md` y la ejecución sobre el tag de entrega, en `docs/validacion/ejecucion-final.md`.

Tabla 11. *Pruebas y resultados esperados*

ID	AC	Precondición y entrada	Esperado	Prueba automatizada
P-S01-01	AC-S01-01	`solicitante1` registra `LAB-01`, `ELECTRICIDAD`, la descripción del criterio, `BAJO`, riesgo `false`	302 al detalle; 1 incidencia `INC-000001`, `REGISTRADA`, `NORMAL`, autor `solicitante1`, fecha `2026-10-01T08:00:00Z`; 1 evento `CREAR`	`test_p_s01_01_registro_valido`
P-S01-02	AC-S01-03	Descripción de 19, 20, 500 y 501 caracteres, en ese orden	19: 400 y 0 incidencias. 20: 302. 500: 302. 501: 400. Al final, 2 incidencias y 2 eventos	`test_p_s01_02_limites_de_la_descripcion`
P-S01-03	AC-S01-04, 05, 06	Diez registros con un único campo inválido: impacto `MEDIO`, `alto` o vacío; riesgo `quiza` o ausente; ubicación `LAB-99` o ausente; categoría `JARDINERIA` o ausente; descripción ausente	400 en cada caso; 0 incidencias; 0 eventos	`test_p_s01_03_valores_invalidos`
P-S01-04	AC-S01-02	`coordinador1` y `tecnico1` envían un registro válido	403; 0 incidencias; 0 eventos	`test_p_s01_04_otros_roles_no_registran`

ID	AC	Precondición y entrada	Esperado	Prueba automatizada
P-S01-05	AC-S01-07	Descripción `<script>alert(1)</script> la silla está rota`	Guardada literal; el detalle contiene `<script>` y no la etiqueta	`test_p_s01_05_el_texto_no_se_ejecuta_como_html`
P-S01-06	AC-S01-08	Registro válido de `solicitante1` que además envía `codigo=INC-777777`, `estado=CERRADA`, `prioridad=CRITICA`, `solicitante_id` de `solicitante2` y `creada_en=2020-01-01…`	Se crea `INC-000001`, `REGISTRADA`, `NORMAL`, autor `solicitante1`, fecha del servidor; no existe `INC-777777`	`test_p_s01_06_el_servidor_ignora_campos_generados`
P-S01-07	AC-S01-02	Visitante sin sesión envía un registro válido	302 al inicio de sesión; 0 incidencias	`test_sin_sesion_no_registra`
P-S01-08	AC-S01-03	Descripción de 19 caracteres rodeada de espacios; de 20 rodeada de espacios y salto de línea	400; 302 y se guarda sin los espacios externos	`test_los_espacios_externos_no_cuentan`
P-S01-09	AC-S01-09	Registro válido mientras se fuerza un fallo al guardar el evento	La operación falla; 0 incidencias; 0 eventos	`test_sin_efectos_parciales_si_falla_el_evento`
P-S01-10	AC-S01-01	Dos registros seguidos, de `solicitante1` y de `solicitante2`	`INC-000001` y `INC-000002`, cada uno con su autor	`test_los_codigos_son_consecutivos`
P-S01-11	AC-S01-10	Ocho registros válidos enviados a la vez desde ocho hilos de un mismo proceso, cada uno con su sesión; cinco repeticiones	8 respuestas 302; códigos `INC-000001` a `INC-000008` sin repetir; cada incidencia con su autor, su descripción y un evento `CREAR`	`test_p_s01_11_registros_simultaneos` en `tests/test_concurrencia.py` (commit `6b99fec`)
P-S01-12	AC-S01-05	Riesgo `True`, `1`, `on`, `TRUE`, `si` y `true` (con espacio inicial)	400 en cada caso; 0 incidencias; 0 eventos	`test_p_s01_12_el_riesgo_solo_acepta_true_o_false` (commit `6b99fec`)

4.1.11 Decisión de revisión / versión aprobada

Tabla 12. *Decisión de revisión / versión aprobada*

Fecha	Versión revisada	Revisor	Decisión	Observaciones
Pendiente	0.5	Rafael Eduardo May Recuero (propuesto)	Pendiente	

4.1.12 Plan de diseño · S01 Registrar

Versión 0.5 · 2026-10-08 · ajustado a la implementación · depende de arquitectura.

Componentes que intervienen

Tabla 13. *Componentes que intervienen*

Capa	Elemento	Responsabilidad
Interfaz	`templates/incidencia_nueva.html`	Formulario con listas de catálogo y opción explícita Sí/No para el riesgo.
Rutas	`GET` y `POST /incidencias/nueva`	Leer el formulario, llamar al servicio, traducir errores a 400/403.
Autenticación	`requiere_rol("SOLICITANTE")`	Sesión y rol antes de cualquier lectura de datos.
Dominio	`dominio.validar_registro(datos)`	Validación pura de los cinco campos; devuelve datos limpios o errores por campo.
Dominio	`dominio.calcular_prioridad(impacto, riesgo)`	Regla de S02.
Servicio	`servicios.registrar_incidencia(db, usuario, datos, ahora)`	Transacción: incidencia + código + evento `CREAR`.
Persistencia	Tablas `incidencias` y `eventos`	Restricciones `CHECK` de catálogo como segunda barrera.

Decisiones

- La validación vive en el dominio, sin dependencia de Flask, para probarla sin servidor.
- El código se deriva del identificador autoincremental dentro de la misma transacción: `INC-` + identificador con seis dígitos.
- La fecha proviene del reloj del servidor, que la aplicación toma de su configuración (`RELOJ`, por defecto `reloj.ahora_utc`), nunca del formulario.
- El consecutivo se calcula dentro de la transacción, después de `BEGIN IMMEDIATE`, que serializa a los escritores; la columna `codigo` es única en la base.
- El servicio recibe solo los cinco campos permitidos; cualquier otro campo de la petición se descarta en la ruta.
- Jinja escapa automáticamente; no se usa el filtro `safe` con texto del usuario.

Orden de comprobaciones

Sesión → rol → validación de datos → transacción.

4.1.13 Tareas · S01 Registrar

Versión 0.5 · 2026-10-08. Cada tarea se registra como issue de GitHub con su RF, SPEC y AC.

El estado refleja lo que existe en el repositorio. «Implementada» significa que el código y sus pruebas existen y están integrados en `main`, no que la tarea esté terminada: ninguna se da por terminada hasta que otro integrante la revise. La integración la hizo el autor sin esa revisión. El revisor lo define el equipo.

Tabla 14. *Tareas · S01 Registrar*

Tarea	Descripción	AC	Archivos	Prueba	Responsable	Revisor	Estado
T-S01-01	Esquema de `usuarios`, `incidencias` y `eventos`; seed con las cinco cuentas ficticias.	AC-S01-01	`app/schema.sql`, `app/db.py`	P-S01-01	Luis Carlo Daza Ospino, con IA	Por asignar	Implementada en `c5e7ef7`; integrada en `main`; revisión de otro integrante pendiente
T-S01-02	Inicio y cierre de sesión con contraseñas con hash; decorador de rol.	AC-S01-02	`app/auth.py`, `app/templates/login.html`	P-S01-04	Luis Carlo Daza Ospino, con IA	Por asignar	Implementada en `c5e7ef7`; integrada en `main`; revisión de otro integrante pendiente
T-S01-03	Validación de los cinco campos en el dominio.	AC-S01-03 a 06	`app/dominio.py`	P-S01-02, P-S01-03	Luis Carlo Daza Ospino, con IA	Por asignar	Implementada en `c5e7ef7`; integrada en `main`; revisión de otro integrante pendiente
T-S01-04	Servicio de registro transaccional: incidencia, código y evento `CREAR`.	AC-S01-01, 02, 08, 09	`app/servicios.py`	P-S01-01, P-S01-06	Luis Carlo Daza Ospino, con IA	Por asignar	Implementada en `7bc08f2`; integrada en `main`; revisión de otro integrante pendiente
T-S01-05	Ruta y formulario de registro; detalle con texto escapado.	AC-S01-01, 07	`app/rutas.py`, `app/templates/`	P-S01-05	Luis Carlo Daza Ospino, con IA	Por asignar	Implementada en `7bc08f2`; integrada en `main`; revisión de otro integrante pendiente
T-S01-06	Pruebas automatizadas de S01.	Todos	`tests/test_s01_registro.py`	P-S01-01 a 10	Luis Carlo Daza Ospino, con IA	Por asignar	Implementada en `7bc08f2`; integrada en `main`; revisión de otro integrante pendiente
T-S01-07	Pruebas de registros simultáneos y de valores de riesgo `True`, `1` y `on`.	AC-S01-05, 10	`tests/test_concurrencia.py`, `tests/test_s01_registro.py`	P-S01-11, P-S01-12	Luis Carlo Daza Ospino, con IA	Por asignar	Implementada en `6b99fec`; integrada en `main`; revisión de otro integrante pendiente

4.2 SPEC S02 · Priorizar y asignar

Tabla 15. *SPEC S02 · Priorizar y asignar*

Campo	Valor
Versión	0.5 (integrada en `main`; revisión de otro integrante pendiente)
Autor	Luis Carlo Daza Ospino, con asistencia de IA (Claude)
Revisor	Propuesto: Jean Marco Oyola De Martino. El equipo no ha confirmado la asignación; revisión pendiente
Fecha	2026-10-08
Requisito asociado	RF02

Cambios de la versión 0.2 (2026-10-08): sin cambios de contenido; las sugerencias recibidas ya estaban cubiertas. Origen: revisión asistida por IA, que no reemplaza la revisión del integrante asignado.

Cambios de la versión 0.3 (2026-10-08): se aclara que el rol se comprueba antes que la existencia de la incidencia; nuevo AC-S02-10 (sin efectos parciales); la tabla de pruebas refleja las pruebas reales e identifica la que falta. Origen: segunda revisión asistida por IA, que tampoco reemplaza la revisión del integrante asignado.

Cambios de la versión 0.4 (2026-10-08): la prueba P-S02-10 ya está automatizada y se registra con su nombre real. No cambia ningún requisito ni criterio.

Cambios de la versión 0.5 (2026-10-08): sin cambios de requisitos ni de criterios. La SPEC queda integrada en `main` junto con el código y las pruebas; las referencias a pruebas ya no dependen de una rama; el revisor figura como propuesto, porque el equipo no ha confirmado la asignación.

4.2.1 Historia (H02)

Como **coordinador de mantenimiento**, quiero **que el sistema calcule la prioridad de cada incidencia y me permita asignarla a un técnico activo**, para **atender primero lo que pone en riesgo a las personas y que cada reporte tenga un responsable**.

4.2.2 Alcance y exclusiones

Incluye: cálculo automático de la prioridad al registrar; asignación de una incidencia `REGISTRADA` a un técnico activo; paso a `ASIGNADA`; evento en el historial.

Excluye: reasignación o desasignación, cambio manual de la prioridad, restricción por especialidad del técnico, balanceo de carga y notificaciones.

4.2.3 Entradas

Cálculo de prioridad (no es entrada del usuario; usa los datos validados en S01):

Tabla 16. *Entradas (2)*

Dato	Tipo	Valores
`impacto`	texto	`BAJO`, `ALTO`
`riesgo_personas`	booleano	`true`, `false`

Asignación:

Tabla 17. *Entradas (3)*

Campo	Tipo	Obligatorio	Valores o límites
`codigo` (en la ruta)	texto	Sí	Código de una incidencia existente.
`tecnico_id`	entero	Sí	Identificador de un usuario con rol `TECNICO` y cuenta activa.

4.2.4 Precondiciones y permisos

- Sesión iniciada con rol `COORDINADOR`. Solicitante y técnico reciben 403.
- La incidencia existe (si no, 404) y está en `REGISTRADA` (si no, 409).
- El técnico existe, tiene rol `TECNICO` y está activo (si no, 400).

4.2.5 Reglas y proceso

Prioridad. La calcula el servidor; nadie la elige ni la modifica.

Tabla 18. *Reglas y proceso*

Riesgo para personas	Impacto	Prioridad
`true`	`BAJO` o `ALTO`	`CRITICA`
`false`	`ALTO`	`ALTA`
`false`	`BAJO`	`NORMAL`

Asignación.

1. Comprobar sesión y rol.
2. Buscar la incidencia por código.
3. Comprobar que su estado es `REGISTRADA`.
4. Validar el técnico.
5. En una sola transacción: registrar la asignación (incidencia, técnico, coordinador, fecha), fijar el técnico en la incidencia, cambiar el estado a `ASIGNADA` e insertar el evento `ASIGNAR`.

4.2.6 Salidas y cambios persistidos

- `incidencias.prioridad` fijada al registrar.
- Una fila en `asignaciones`; como máximo una por incidencia.
- `incidencias.tecnico_id` y `estado = ASIGNADA`.
- Un evento `ASIGNAR` con actor coordinador, `REGISTRADA → ASIGNADA`, fecha y técnico asignado.
- Respuesta: redirección 302 al detalle.

4.2.7 Errores y efectos que deben evitarse

Tabla 19. *Errores y efectos que deben evitarse (2)*

Situación	Respuesta	Efecto
Rol distinto de coordinador	403, exista o no la incidencia: el rol se comprueba primero	Ninguno
Incidencia inexistente	404	Ninguno

Situación	Respuesta	Efecto
Incidencia en estado distinto de `REGISTRADA`	409	Ninguno; conserva el técnico original
Técnico ausente, inexistente, inactivo o usuario que no es técnico	400	Ninguno

No debe ocurrir: una incidencia `ASIGNADA` sin técnico; dos asignaciones para la misma incidencia; un cambio de técnico después de asignar; una prioridad distinta de la que da la tabla.

4.2.8 Criterios de aceptación

AC-S02-01 (prioridad crítica). Dado un registro válido con riesgo para personas `true`, cuando se crea la incidencia con impacto `BAJO` o con impacto `ALTO`, entonces su prioridad es `CRITICA` en ambos casos.

AC-S02-02 (prioridad alta). Dado un registro válido con riesgo `false` e impacto `ALTO`, cuando se crea la incidencia, entonces su prioridad es `ALTA`.

AC-S02-03 (prioridad normal). Dado un registro válido con riesgo `false` e impacto `BAJO`, cuando se crea la incidencia, entonces su prioridad es `NORMAL`.

AC-S02-04 (asignación normal). Dada la incidencia `INC-000001` en `REGISTRADA` y el técnico activo `tecnico1`, cuando el coordinador la asigna a `tecnico1`, entonces queda `ASIGNADA` con técnico `tecnico1`, existe una asignación con coordinador, técnico y fecha, y el historial suma un evento `ASIGNAR` del coordinador de `REGISTRADA` a `ASIGNADA`.

AC-S02-05 (permisos). Dada una incidencia `REGISTRADA`, cuando un solicitante (incluso su dueño) o un técnico intenta asignarla, entonces recibe 403 y la incidencia sigue `REGISTRADA`, sin técnico y sin eventos nuevos.

AC-S02-06 (asignación repetida y estado incompatible). Dada una incidencia ya `ASIGNADA` a `tecnico1`, cuando el coordinador intenta asignarla a `tecnico2` o de nuevo a `tecnico1`, entonces recibe 409, el técnico sigue siendo `tecnico1`, hay una sola asignación y no hay eventos nuevos. Lo mismo ocurre en `EN_ATENCION`, `PENDIENTE_VALIDACION` y `CERRADA`.

AC-S02-07 (técnico inválido). Dada una incidencia `REGISTRADA`, cuando el coordinador envía un `tecnico_id` ausente, inexistente, de una cuenta inactiva o de un usuario que no es técnico, entonces recibe 400 y la incidencia sigue `REGISTRADA` sin técnico ni eventos nuevos.

AC-S02-08 (incidencia inexistente). Dado el coordinador con sesión iniciada, cuando intenta asignar el código `INC-999999`, entonces recibe 404 y no cambia nada.

AC-S02-09 (prioridad no manipulable). Dado cualquier usuario, cuando envía un valor de prioridad en el registro o en la asignación, entonces se ignora y la prioridad sigue siendo la calculada.

AC-S02-10 (sin efectos parciales). Dada una incidencia `REGISTRADA`, cuando falla el guardado del evento durante la asignación, entonces la transacción se revierte: la incidencia sigue `REGISTRADA` y sin técnico, no hay asignación y el historial no cambia.

4.2.9 Diseño y tareas vinculadas a cada AC

Diseño en plan.md; tareas en tasks.md.

Tabla 20. *Diseño y tareas vinculadas a cada AC (2)*

AC	Tareas
AC-S02-01 a 03, 09	T-S02-01
AC-S02-04	T-S02-02, T-S02-03, T-S02-04
AC-S02-05 a 08	T-S02-03
Todos	T-S02-05
AC-S02-10	T-S02-06

4.2.10 Pruebas y resultados esperados

El resultado esperado se define aquí, antes de ejecutar, y no se calcula con la función de producción. Salvo que se indique otra cosa, cada prueba parte de una base SQLite nueva y aislada que solo contiene las cinco cuentas del seed, con el reloj del servidor fijado en `2026-10-01T08:00:00Z`. Todas las pruebas de esta tabla existen en `tests/` y se ejecutaron dentro de la batería completa. Los resultados están en `docs/validacion/validacion.md` y la ejecución sobre el tag de entrega, en `docs/validacion/ejecucion-final.md`.

Tabla 21. *Pruebas y resultados esperados (2)*

ID	AC	Precondición y entrada	Esperado	Prueba automatizada
P-S02-01	AC-S02-01, 02, 03	Registro con (riesgo, impacto): (`true`, `BAJO`); (`true`, `ALTO`); (`false`, `ALTO`); (`false`, `BAJO`)	`CRITICA`; `CRITICA`; `ALTA`; `NORMAL`	`test_p_s02_01_prioridad_calculada`
P-S02-02	AC-S02-04	`INC-000001` en `REGISTRADA` con 1 evento; a las 08:30 `coordinador1` la asigna a `tecnico1`	302; `ASIGNADA`; técnico `tecnico1`; 1 asignación con coordinador y fecha `08:30`; eventos `CREAR`, `ASIGNAR`	`test_p_s02_02_asignacion_correcta`
P-S02-03	AC-S02-06	`INC-000001` ya `ASIGNADA` a `tecnico1`; el coordinador la asigna a `tecnico2` y, en otro caso, de nuevo a `tecnico1`	409; técnico `tecnico1`; 1 asignación; 2 eventos	`test_p_s02_03_asignacion_repetida`
P-S02-04	AC-S02-05	`solicitante1` (dueño), `solicitante2` y `tecnico1` intentan asignar una `REGISTRADA`	403; `REGISTRADA` sin técnico; 1 evento	`test_p_s02_04_otros_roles_no_asignan`
P-S02-05	AC-S02-07	`tecnico_id` de una cuenta inactiva, de un solicitante, inexistente, ausente y no numérico	400; `REGISTRADA` sin técnico; 0 asignaciones; 1 evento	`test_p_s02_05_tecnico_invalido`

ID	AC	Precondición y entrada	Esperado	Prueba automatizada
P-S02-06	AC-S02-06	`INC-000001` en `EN_ATENCION`, `PENDIENTE_VALIDACION` y `CERRADA`; el coordinador la asigna a `tecnico2`	409; mismo estado; técnico `tecnico1`; 1 asignación; mismo historial	`test_no_se_asigna_fuera_de_registrada` (commit `de49d68`)
P-S02-07	AC-S02-08	El coordinador asigna `INC-999999`	404; 0 asignaciones	`test_incidencia_inexistente`
P-S02-08	AC-S02-09	Asignación que además envía `prioridad=CRITICA` sobre una incidencia `NORMAL`. En el registro, ver P-S01-06	La prioridad sigue `NORMAL`	`test_la_prioridad_no_se_cambia_al_asignar`
P-S02-09	AC-S02-06	Con `INC-000001` asignada, `INSERT` de una segunda asignación y `UPDATE` del técnico directamente en la base	La base rechaza ambos; el técnico sigue siendo `tecnico1`	`test_la_base_impide_una_segunda_asignacion`
P-S02-10	AC-S02-10	Asignación válida mientras se fuerza un fallo al guardar el evento	La operación falla; las tablas `incidencias`, `asignaciones`, `soluciones`, `cierres` y `eventos` quedan idénticas fila a fila (`REGISTRADA` sin técnico, 0 asignaciones, 1 evento); sin el fallo, la misma operación se completa	`test_un_fallo_al_guardar_el_evento_revierte_toda_la_operacion`, caso `P-S02-10-asignar`, en `tests/test_atomicidad.py` (commit `6b99fec`)

4.2.11 Decisión de revisión / versión aprobada

Tabla 22. *Decisión de revisión / versión aprobada (2)*

Fecha	Versión revisada	Revisor	Decisión	Observaciones
Pendiente	0.5	Jean Marco Oyola De Martino (propuesto)	Pendiente	

4.2.12 Plan de diseño · S02 Priorizar y asignar

Versión 0.5 · 2026-10-08 · ajustado a la implementación · depende de arquitectura y de S01.

Componentes que intervienen

Tabla 23. *Componentes que intervienen (2)*

Capa	Elemento	Responsabilidad
Dominio	`dominio.calcular_prioridad(impacto, riesgo)`	Función pura con la tabla de prioridad. Se invoca desde el registro de S01.
Dominio	`dominio.TRANSICIONES`	Tabla única de transiciones permitidas: acción → (estado origen, estado destino, rol).
Rutas	`POST /incidencias/<codigo>/asignar`	Leer `tecnico_id`, llamar al servicio.
Servicio	`servicios.asignar(db, usuario, codigo, tecnico_id, ahora)`	Comprobaciones y transacción.
Interfaz	Bloque de asignación en `incidencia_detalle.html`	Lista de técnicos activos; visible solo al coordinador y en `REGISTRADA`.
Persistencia	Tabla `asignaciones` con `incidencia_id` único	La base impide una segunda asignación aunque falle la comprobación del servicio.

Decisiones

- La prioridad se guarda en la incidencia al crearla y no se recalcula, porque los datos de origen no se editan.
- "No reasignar" se garantiza en tres niveles: no existe la acción en la tabla de transiciones, el servicio exige `REGISTRADA`, y la base tiene restricción de unicidad.
- La actualización del estado usa la condición `WHERE estado = 'REGISTRADA'` y se verifica que afectó una fila; si no, se revierte y se responde 409. Esto cubre dos asignaciones simultáneas.

Orden de comprobaciones

Sesión → rol → existencia de la incidencia → estado → validez del técnico → transacción.

4.2.13 Tareas · S02 Priorizar y asignar

Versión 0.5 · 2026-10-08. Cada tarea se registra como issue de GitHub con su RF, SPEC y AC.

El estado refleja lo que existe en el repositorio. «Implementada» significa que el código y sus pruebas existen y están integrados en `main`, no que la tarea esté terminada: ninguna se da por terminada hasta que otro integrante la revise. La integración la hizo el autor sin esa revisión. El revisor lo define el equipo.

Tabla 24. *Tareas · S02 Priorizar y asignar*

Tarea	Descripción	AC	Archivos	Prueba	Responsable	Revisor	Estado
T-S02-01	Función pura de prioridad e integración en el registro.	AC-S02-01, 02, 03, 09	`app/dominio.py`, `app/servicios.py`	P-S02-01	Luis Carlo Daza Ospino, con IA	Por asignar	Implementada en `c5e7ef7`, `7bc08f2`; integrada en `main`; revisión de otro integrante pendiente
T-S02-02	Tabla `asignaciones` con unicidad por incidencia.	AC-S02-04, 06	`app/schema.sql`	P-S02-02, P-S02-03	Luis Carlo Daza Ospino, con IA	Por asignar	Implementada en `c5e7ef7`; integrada en `main`; revisión de otro integrante pendiente
T-S02-03	Servicio de asignación con comprobaciones de rol, estado y técnico.	AC-S02-04 a 08	`app/servicios.py`, `app/dominio.py`	P-S02-02 a 05	Luis Carlo Daza Ospino, con IA	Por asignar	Implementada en `8479352`; integrada en `main`; revisión de otro integrante pendiente
T-S02-04	Ruta de asignación y bloque en el detalle para el coordinador.	AC-S02-04	`app/rutas.py`, `app/templates/incidencia_detalle.html`	P-S02-02	Luis Carlo Daza Ospino, con IA	Por asignar	Implementada en `8479352`; integrada en `main`; revisión de otro integrante pendiente
T-S02-05	Pruebas automatizadas de S02.	Todos	`tests/test_s02_prioridad_asignacion.py`	P-S02-01 a 09	Luis Carlo Daza Ospino, con IA	Por asignar	Implementada en `8479352`, `de49d68`; integrada en `main`; revisión de otro integrante pendiente
T-S02-06	Prueba de que la asignación no deja efectos parciales.	AC-S02-10	`tests/test_atomicidad.py`	P-S02-10	Luis Carlo Daza Ospino, con IA	Por asignar	Implementada en `6b99fec`; integrada en `main`; revisión de otro integrante pendiente

4.3 SPEC S03 · Atender una incidencia

Tabla 25. *SPEC S03 · Atender una incidencia*

Campo	Valor
Versión	0.6 (integrada en `main`; comentario de un integrante recibido)
Autor	Luis Carlo Daza Ospino, con asistencia de IA (Claude)
Revisor	José Leonardo Hernández Pedrosa: comentario recibido el 2026-10-13, sin decisión explícita de aprobación
Fecha	2026-10-08
Requisito asociado	RF03

Cambios de la versión 0.2 (2026-10-08): sin cambios de contenido; se descartó una sugerencia errónea (ver registro). Origen: revisión asistida por IA, que no reemplaza la revisión del integrante asignado.

Cambios de la versión 0.3 (2026-10-08): nuevo AC-S03-09 (sin efectos parciales); la tabla de pruebas indica el estado de partida de cada conteo de eventos, refleja las pruebas reales e identifica la que falta. Origen: segunda revisión asistida por IA, que tampoco reemplaza la revisión del integrante asignado.

Cambios de la versión 0.4 (2026-10-08): la prueba P-S03-12 ya está automatizada y se registra con su nombre real. No cambia ningún requisito ni criterio.

Cambios de la versión 0.5 (2026-10-08): sin cambios de requisitos ni de criterios. La SPEC queda integrada en `main` junto con el código y las pruebas; las referencias a pruebas ya no dependen de una rama; el revisor figura como propuesto, porque el equipo no ha confirmado la asignación.

Cambios de la versión 0.6 (2026-10-13): a partir del comentario de José Leonardo Hernández Pedrosa, se aclara qué ocurre si una incidencia no puede resolverse y quién la cierra. No cambia ningún requisito, criterio ni el código. Ver comentarios de los integrantes.

4.3.1 Historia (H03)

Como **técnico**, quiero **iniciar la atención de las incidencias que tengo asignadas y registrar la solución que apliqué**, para **que el solicitante pueda comprobarla y quede constancia de mi trabajo**.

4.3.2 Alcance y exclusiones

Incluye: inicio de atención (`ASIGNADA → EN_ATENCION`); registro de solución (`EN_ATENCION → PENDIENTE_VALIDACION`), también después de un rechazo o una reapertura; eventos en el historial.

Excluye: cierre por parte del técnico, edición o borrado de una solución registrada, atención por un técnico distinto del asignado, adjuntos, registro de tiempos o materiales.

Si la incidencia no puede resolverse: el caso no define un estado para eso y no se pueden agregar transiciones. La incidencia sigue en `EN_ATENCION` hasta que el técnico registre una solución; no hay cierre automático ni por parte del técnico.

Cuándo se cierra: solo cuando el solicitante dueño confirma una solución en `PENDIENTE_VALIDACION`, según S04.

4.3.3 Entradas

Tabla 26. *Entradas (4)*

Operación	Campo	Tipo	Obligatorio	Valores o límites
Iniciar	`codigo` (en la ruta)	texto	Sí	Código de una incidencia asignada al técnico.
Registrar solución	`codigo` (en la ruta)	texto	Sí	Igual.
Registrar solución	`solucion`	texto	Sí	De 20 a 800 caracteres después de retirar espacios externos.

4.3.4 Precondiciones y permisos

- Sesión iniciada con rol `TECNICO`. Solicitante y coordinador reciben 403.
- La incidencia existe y está asignada al técnico de la sesión. Si no existe o está asignada a otro técnico, la respuesta es 404 (supuesto Q1).
- Iniciar exige estado `ASIGNADA`; registrar solución exige `EN_ATENCION`. En otro estado, 409.

4.3.5 Reglas y proceso

Iniciar atención.

1. Comprobar sesión, rol y que la incidencia está asignada al técnico.
2. Comprobar estado `ASIGNADA`.
3. En una transacción: cambiar a `EN_ATENCION` e insertar el evento `INICIAR_ATENCION`.

Registrar solución.

1. Comprobar sesión, rol y asignación.
2. Comprobar estado `EN_ATENCION`.
3. Retirar espacios externos y validar la longitud.
4. En una transacción: insertar la solución (incidencia, técnico, texto, fecha), cambiar a `PENDIENTE_VALIDACION` e insertar el evento `REGISTRAR_SOLUCION` enlazado a esa solución.

Cada solución es un registro nuevo. Las soluciones anteriores nunca se modifican ni se eliminan.

4.3.6 Salidas y cambios persistidos

- Iniciar: `estado = EN_ATENCION`; evento `INICIAR_ATENCION` (`ASIGNADA → EN_ATENCION`).
- Registrar solución: fila nueva en `soluciones`; `estado = PENDIENTE_VALIDACION`; evento `REGISTRAR_SOLUCION` (`EN_ATENCION → PENDIENTE_VALIDACION`) con referencia a la solución.
- Respuesta: redirección 302 al detalle.

4.3.7 Errores y efectos que deben evitarse

Tabla 27. *Errores y efectos que deben evitarse (3)*

Situación	Respuesta	Efecto
Rol distinto de técnico	403	Ninguno
Incidencia inexistente o asignada a otro técnico	404	Ninguno
Iniciar en estado distinto de `ASIGNADA`	409	Ninguno
Registrar solución en estado distinto de `EN_ATENCION`	409	Ninguno

Situación	Respuesta	Efecto
Solución ausente o fuera de 20–800 caracteres	400	Ninguno

No debe ocurrir: una solución guardada sin cambio de estado o un cambio de estado sin solución; que el técnico pase la incidencia a `CERRADA`; que se pierda o sobrescriba una solución anterior.

4.3.8 Criterios de aceptación

AC-S03-01 (iniciar atención). Dada la incidencia `INC-000001` en `ASIGNADA` con técnico `tecnico1`, cuando `tecnico1` inicia la atención, entonces queda `EN_ATENCION` y el historial suma un evento `INICIAR_ATENCION` de `tecnico1` de `ASIGNADA` a `EN_ATENCION` con su fecha.

AC-S03-02 (registrar solución). Dada la incidencia `INC-000001` en `EN_ATENCION` con técnico `tecnico1`, cuando `tecnico1` registra la solución "Se reemplazó el tomacorriente y se probó con carga", entonces queda `PENDIENTE_VALIDACION`, existe una solución con ese texto, autor `tecnico1` y fecha, y el historial suma un evento `REGISTRAR_SOLUCION` enlazado a ella.

AC-S03-03 (técnico no asignado y otros roles). Dada una incidencia asignada a `tecnico1`, cuando `tecnico2` intenta iniciarla o registrarle una solución, entonces recibe 404 y no cambia el estado ni el historial. Cuando lo intenta un solicitante o el coordinador, entonces recibe 403 sin cambios.

AC-S03-04 (estados incompatibles). Dada una incidencia en `ASIGNADA`, cuando su técnico registra una solución sin haber iniciado la atención, entonces recibe 409, no se guarda la solución y el estado sigue `ASIGNADA`. Dada una incidencia en `EN_ATENCION`, `PENDIENTE_VALIDACION` o `CERRADA`, cuando su técnico intenta iniciarla, entonces recibe 409 sin cambios. Dada una incidencia en `PENDIENTE_VALIDACION` o `CERRADA`, cuando su técnico registra otra solución, entonces recibe 409 sin cambios.

AC-S03-05 (límites de la solución). Dada una incidencia en `EN_ATENCION`, cuando su técnico envía una solución de 19 caracteres, de 801 caracteres, vacía o de 19 caracteres rodeados de espacios, entonces recibe 400 y la incidencia sigue `EN_ATENCION` sin solución ni eventos nuevos. Cuando envía una de exactamente 20 o exactamente 800, entonces se acepta.

AC-S03-06 (conservación de soluciones). Dada una incidencia que volvió a `EN_ATENCION` por un rechazo o una reapertura y que ya tiene una solución, cuando su técnico registra una solución nueva, entonces hay dos soluciones, la primera conserva su texto, autor y fecha, y el estado es `PENDIENTE_VALIDACION`.

AC-S03-07 (el técnico no cierra). Dado un técnico con una incidencia asignada en cualquier estado, cuando intenta confirmarla, entonces recibe 403 y la incidencia no pasa a `CERRADA`.

AC-S03-08 (texto no ejecutable). Dada una solución que contiene `<img src=x onerror=alert(1)>`, cuando se muestra en el detalle, entonces aparece escapada y no como etiqueta.

AC-S03-09 (sin efectos parciales). Dada una incidencia `ASIGNADA` o `EN_ATENCION`, cuando falla el guardado del evento al iniciar la atención o al registrar la solución, entonces la transacción se revierte: el estado no cambia, no queda ninguna solución nueva y el historial no cambia.

4.3.9 Diseño y tareas vinculadas a cada AC

Diseño en plan.md; tareas en tasks.md.

Tabla 28. *Diseño y tareas vinculadas a cada AC (3)*

AC	Tareas
AC-S03-01	T-S03-02, T-S03-04
AC-S03-02, 06	T-S03-01, T-S03-03, T-S03-04
AC-S03-03, 04, 07	T-S03-02, T-S03-03
AC-S03-05	T-S03-03
AC-S03-08	T-S03-04
Todos	T-S03-05
AC-S03-09	T-S03-06

4.3.10 Pruebas y resultados esperados

El resultado esperado se define aquí, antes de ejecutar, y no se calcula con la función de producción. Salvo que se indique otra cosa, cada prueba parte de una base SQLite nueva y aislada que solo contiene las cinco cuentas del seed, con el reloj del servidor fijado en `2026-10-01T08:00:00Z`. Todas las pruebas de esta tabla existen en `tests/` y se ejecutaron dentro de la batería completa. Los resultados están en `docs/validacion/validacion.md` y la ejecución sobre el tag de entrega, en `docs/validacion/ejecucion-final.md`.

Tabla 29. *Pruebas y resultados esperados (3)*

ID	AC	Precondición y entrada	Esperado	Prueba automatizada
P-S03-01	AC-S03-01	`INC-000001` `ASIGNADA` a `tecnico1`, con 2 eventos (`CREAR`, `ASIGNAR`); a las 09:00 `tecnico1` inicia	302; `EN_ATENCION`; 3 eventos; el tercero es `INICIAR_ATENCION` de `tecnico1` con fecha `09:00`	`test_p_s03_01_iniciar_atencion`
P-S03-02	AC-S03-02	`INC-000001` `EN_ATENCION`, con 3 eventos; `tecnico1` registra la solución del AC-S03-02	302; `PENDIENTE_VALIDACION`; 1 solución de `tecnico1`; 4 eventos; el cuarto enlaza la solución	`test_p_s03_02_registrar_solucion`
P-S03-03	AC-S03-03	`tecnico2` inicia la incidencia `ASIGNADA` a `tecnico1`, y le registra una solución cuando está `EN_ATENCION`	404; mismo estado y mismo historial; 0 soluciones	`test_p_s03_03_tecnico_no_asignado`
P-S03-04	AC-S03-04	`INC-000001` `ASIGNADA`, con 2 eventos; `tecnico1` registra una solución sin iniciar	409; `ASIGNADA`; 0 soluciones; 2 eventos	`test_p_s03_04_solucion_antes_de_iniciar`
P-S03-05	AC-S03-05	`INC-000001` `EN_ATENCION`; solución de 19, 20, 800 y 801 caracteres, vacía, y de 19 rodeada de espacios	19: 400. 20: 302. 800: 302. 801: 400. Vacía: 400. Con espacios: 400. En los rechazos: `EN_ATENCION`, 0 soluciones, 3 eventos	`test_p_s03_05_limites_de_la_solucion`
P-S03-06	AC-S03-03	`solicitante1` y `coordinador1` intentan iniciar y registrar solución sobre una `ASIGNADA`	403; `ASIGNADA`; 2 eventos	`test_otros_roles_no_atienden`

ID	AC	Precondición y entrada	Esperado	Prueba automatizada
P-S03-07	AC-S03-04	`tecnico1` inicia una incidencia en `EN_ATENCION`, `PENDIENTE_VALIDACION` y `CERRADA`	409; mismo estado y mismo historial	`test_iniciar_en_estado_incompatible` (`CERRADA` desde `de49d68`)
P-S03-08	AC-S03-04	`tecnico1` registra otra solución en `PENDIENTE_VALIDACION` y en `CERRADA`	409; sigue habiendo 1 solución	`test_segunda_solucion_sin_rechazo_se_rechaza` (`CERRADA` desde `de49d68`)
P-S03-09	AC-S03-08	Solución `<img src=x onerror=alert(1)>` se cambió la pieza	El detalle la muestra escapada	`test_la_solucion_no_se_ejecuta_como_html`
P-S03-10	AC-S03-06	Nueva solución tras un rechazo y tras una reapertura	Ver P-S04-02 y P-S04-03: 2 soluciones, la primera intacta	`test_p_s04_02_rechazo_y_nueva_solucion`, `test_p_s04_03_reapertura_y_nuevo_cierre`
P-S03-11	AC-S03-07	`tecnico1` y `tecnico2` intentan confirmar una `PENDIENTE_VALIDACION`	Ver P-S04-05: 403; 0 cierres	`test_p_s04_05_solo_el_duenio_valida`
P-S03-12	AC-S03-09	Iniciar y registrar solución mientras se fuerza un fallo al guardar el evento	La operación falla; las cinco tablas quedan idénticas fila a fila (mismo estado, 0 soluciones nuevas, mismo historial); sin el fallo, la misma operación se completa	`test_un_fallo_al_guardar_el_evento_revierte_toda_la_operacion`, casos `P-S03-12-iniciar` y `P-S03-12-solucion`, en `tests/test_atomicidad.py` (commit `6b99fec`)

4.3.11 Decisión de revisión / versión aprobada

Tabla 30. *Decisión de revisión / versión aprobada (3)*

Fecha	Versión revisada	Revisor	Decisión	Observaciones
2026-10-13	0.5	José Leonardo Hernández Pedrosa	Comentario recibido, sin decisión de aprobar o rechazar	Dos aclaraciones incorporadas en la 0.6; una observación ya cubierta; una mejora opcional no implementada. Ver `docs/revision/comentarios-integrantes.md`

4.3.12 Plan de diseño · S03 Atender

Versión 0.5 · 2026-10-08 · ajustado a la implementación · depende de arquitectura, S01 y S02.

Componentes que intervienen

Tabla 31. *Componentes que intervienen (3)*

Capa	Elemento	Responsabilidad
Rutas	`POST /incidencias/<codigo>/iniciar`	Llamar al servicio de inicio.
Rutas	`POST /incidencias/<codigo>/solucion`	Leer `solucion`, llamar al servicio.
Dominio	`dominio.validar_texto(valor, minimo, maximo, campo)`	Recorte y límites; se reutiliza en S04 para los motivos.
Servicio	`servicios.iniciar_atencion(...)` y `servicios.registrar_solucion(...)`	Comprobaciones y transacción.

Capa	Elemento	Responsabilidad
Consultas	`consultas.obtener(db, usuario, codigo)`	Devuelve la incidencia solo si el usuario puede verla; si no, 404. Se comparte con S04 y S05.
Interfaz	Bloques “Iniciar atención” y “Registrar solución” en el detalle	Visibles solo al técnico asignado y en el estado correspondiente.
Persistencia	Tabla `soluciones`, solo inserción	Disparadores que impiden modificar o borrar.

Decisiones

- La pertenencia se comprueba comparando `incidencias.tecnico_id` con el usuario de la sesión, nunca con un dato del formulario.
- La solución y el cambio de estado van en la misma transacción; si el evento falla, no queda la solución.
- El evento guarda `solucion_id` para que el historial muestre el texto sin duplicarlo.

Orden de comprobaciones

Sesión → rol → visibilidad (asignación) → estado → validación del texto → transacción.

4.3.13 Tareas · S03 Atender

Versión 0.5 · 2026-10-08. Cada tarea se registra como issue de GitHub con su RF, SPEC y AC.

El estado refleja lo que existe en el repositorio. «Implementada» significa que el código y sus pruebas existen y están integrados en `main`, no que la tarea esté terminada: ninguna se da por terminada hasta que otro integrante la revise. La integración la hizo el autor sin esa revisión. El revisor lo define el equipo.

Tabla 32. *Tareas · S03 Atender*

Tarea	Descripción	AC	Archivos	Prueba	Responsable	Revisor	Estado
T-S03-01	Tabla `soluciones` de solo inserción.	AC-S03-02, 06	`app/schema.sql`	P-S03-02	Luis Carlo Daza Ospino, con IA	Por asignar	Implementada en `c5e7ef7`; integrada en `main`; revisión de otro integrante pendiente
T-S03-02	Servicio de inicio de atención con comprobación de asignación y estado.	AC-S03-01, 03, 04	`app/servicios.py`	P-S03-01, P-S03-03	Luis Carlo Daza Ospino, con IA	Por asignar	Implementada en `20ebd53`; integrada en `main`; revisión de otro integrante pendiente
T-S03-03	Servicio de registro de solución con validación de 20 a 800 caracteres.	AC-S03-02 a 06	`app/servicios.py`, `app/dominio.py`	P-S03-02, P-S03-04, P-S03-05	Luis Carlo Daza Ospino, con IA	Por asignar	Implementada en `20ebd53`; integrada en `main`; revisión de otro integrante pendiente
T-S03-04	Rutas y bloques de la interfaz del técnico.	AC-S03-01, 02, 08	`app/rutas.py`, `app/templates/incidencia_detalle.html`	P-S03-01, P-S03-02	Luis Carlo Daza Ospino, con IA	Por asignar	Implementada en `20ebd53`; integrada en `main`; revisión de otro integrante pendiente
T-S03-05	Pruebas automatizadas de S03.	Todos	`tests/test_s03_atencion.py`	P-S03-01 a 11	Luis Carlo Daza Ospino, con IA	Por asignar	Implementada en `20ebd53`, `de49d68`; integrada en `main`; revisión de otro integrante pendiente
T-S03-06	Prueba de que iniciar y registrar solución no dejan efectos parciales.	AC-S03-09	`tests/test_atomicidad.py`	P-S03-12	Luis Carlo Daza Ospino, con IA	Por asignar	Implementada en `6b99fec`; integrada en `main`; revisión de otro integrante pendiente

4.4 SPEC S04 · Validar la solución: confirmar, rechazar y reabrir

Tabla 33. *SPEC S04 · Validar la solución: confirmar, rechazar y reabrir*

Campo	Valor
Versión	0.5 (integrada en `main`; revisión de otro integrante pendiente)
Autor	Luis Carlo Daza Ospino, con asistencia de IA (Claude)
Revisor	Propuesto: Cristian David Diaz España. El equipo no ha confirmado la asignación; revisión pendiente
Fecha	2026-10-08
Requisito asociado	RF04

Cambios de la versión 0.2 (2026-10-08): se agregan los límites 300 y 301 a la prueba P-S04-04. Origen: revisión asistida por IA, que no reemplaza la revisión del integrante asignado.

Cambios de la versión 0.3 (2026-10-08): se define «solución vigente»; se precisa que un motivo formado solo por espacios se rechaza; nuevo AC-S04-11 (sin efectos parciales); la tabla de pruebas refleja las pruebas reales e identifica las que faltan. Origen: segunda revisión asistida por IA, que tampoco reemplaza la revisión del integrante asignado.

Cambios de la versión 0.4 (2026-10-08): las pruebas P-S04-15 y P-S04-16 ya están automatizadas y se registran con su nombre real. No cambia ningún requisito ni criterio.

Cambios de la versión 0.5 (2026-10-08): sin cambios de requisitos ni de criterios. La SPEC queda integrada en `main` junto con el código y las pruebas; las referencias a pruebas ya no dependen de una rama; el revisor figura como propuesto, porque el equipo no ha confirmado la asignación.

4.4.1 Historia (H04)

Como **solicitante dueño de una incidencia**, quiero **confirmar o rechazar la solución propuesta y reabrir el caso si el daño reaparece poco después del cierre**, para **que una incidencia solo quede cerrada cuando yo compruebe que se resolvió**.

4.4.2 Alcance y exclusiones

Incluye: confirmar (`PENDIENTE_VALIDACION → CERRADA`); rechazar con motivo (`PENDIENTE_VALIDACION → EN_ATENCION`); reabrir con motivo dentro de 48 horas desde el último cierre (`CERRADA → EN_ATENCION`); conservación de técnico, soluciones y cierres.

Excluye: cierre automático por tiempo; confirmación, rechazo o reapertura por el coordinador o el técnico; cambio de técnico al rechazar o reabrir; reapertura fuera de plazo; edición de motivos.

4.4.3 Entradas

Tabla 34. *Entradas (5)*

Operación	Campo	Tipo	Obligatorio	Valores o límites
Confirmar	`codigo` (en la ruta)	texto	Sí	Código de una incidencia propia.
Rechazar	`motivo`	texto	Sí	De 10 a 300 caracteres después de retirar espacios externos.
Reabrir	`motivo`	texto	Sí	De 10 a 300 caracteres después de retirar espacios externos.

La hora efectiva de cada operación la fija el servidor en UTC. Cualquier fecha enviada por el cliente se ignora.

4.4.4 Precondiciones y permisos

- Sesión iniciada con rol `SOLICITANTE`. Coordinador y técnico reciben 403.
- La incidencia existe y su autor es el usuario de la sesión. Si no existe o es de otro solicitante, la respuesta es 404 (supuesto Q1).
- Confirmar y rechazar exigen `PENDIENTE_VALIDACION`. Reabrir exige `CERRADA`. En otro estado, 409.
- Reabrir exige que el tiempo transcurrido desde el último cierre sea menor o igual a 48 horas. Fuera de plazo, 409.

4.4.5 Reglas y proceso

Solución vigente. Es la última solución registrada para la incidencia, es decir, la que la llevó al estado `PENDIENTE_VALIDACION` actual. Si hubo rechazos o reaperturas, las soluciones anteriores se conservan pero ya no son la vigente.

Confirmar. En una transacción: registrar un cierre (incidencia, solución vigente, quién confirma, fecha), cambiar a `CERRADA` e insertar el evento `CONFIRMAR_SOLUCION` enlazado al cierre.

Rechazar. Validar el motivo. En una transacción: cambiar a `EN_ATENCION` e insertar el evento `RECHAZAR_SOLUCION` con el motivo y la referencia a la solución rechazada. El técnico no cambia. La solución rechazada se conserva.

Reabrir.

1. Comprobar sesión, rol, pertenencia y estado `CERRADA`.
2. Tomar la fecha del cierre más reciente de la incidencia.
3. Calcular `ahora_utc − fecha_del_último_cierre` con el reloj del servidor. Si es mayor que 48 horas, rechazar con 409.
4. Validar el motivo.
5. En una transacción: cambiar a `EN_ATENCION` e insertar el evento `REABRIR` con el motivo y la referencia al cierre que se reabre. El técnico no cambia. Todos los cierres y soluciones anteriores se conservan.

El límite es inclusivo: exactamente 48 horas se acepta; 48 horas más un microsegundo se rechaza (supuesto Q3).

4.4.6 Salidas y cambios persistidos

Tabla 35. *Salidas y cambios persistidos*

Operación	Estado	Registros nuevos
Confirmar	`CERRADA`	Una fila en `cierres`; evento `CONFIRMAR_SOLUCION`.
Rechazar	`EN_ATENCION`	Evento `RECHAZAR_SOLUCION` con motivo.
Reabrir	`EN_ATENCION`	Evento `REABRIR` con motivo.

`incidencias.tecnico_id` no cambia en ninguna de las tres. Respuesta: redirección 302 al detalle.

4.4.7 Errores y efectos que deben evitarse

Tabla 36. *Errores y efectos que deben evitarse (4)*

Situación	Respuesta	Efecto
Rol distinto de solicitante	403	Ninguno
Incidencia inexistente o de otro solicitante	404	Ninguno
Confirmar o rechazar fuera de `PENDIENTE_VALIDACION`	409	Ninguno
Reabrir fuera de `CERRADA`	409	Ninguno
Reabrir después de 48 horas	409	Ninguno
Motivo ausente o fuera de 10–300 caracteres	400	Ninguno

No debe ocurrir: un cierre sin evento o un evento sin cierre; pérdida de una solución o de un cierre anterior; cambio de técnico; uso de una hora elegida por el navegador; que el coordinador o el técnico cierren o reabran.

4.4.8 Criterios de aceptación

AC-S04-01 (confirmar). Dada la incidencia propia `INC-000001` en `PENDIENTE_VALIDACION` con técnico `tecnico1` y una solución registrada, cuando `solicitante1` la confirma, entonces queda `CERRADA`, existe un cierre con esa solución, `solicitante1` y la fecha del servidor, y el historial suma un evento `CONFIRMAR_SOLUCION` de `PENDIENTE_VALIDACION` a `CERRADA`.

AC-S04-02 (rechazar). Dado un reporte propio `PENDIENTE_VALIDACION` con técnico `tecnico1` y solución registrada, cuando el solicitante lo rechaza con “El daño sigue presente”, entonces queda `EN_ATENCION`, conserva `tecnico1` y la solución anterior, y registra quién rechazó, cuándo y por qué.

AC-S04-03 (motivo de rechazo inválido). Dado un reporte propio `PENDIENTE_VALIDACION`, cuando el solicitante lo rechaza con un motivo de 9 caracteres, de 301, vacío, formado solo por espacios o de 9 rodeados de espacios, entonces recibe 400 y el reporte sigue `PENDIENTE_VALIDACION` sin eventos nuevos. Con exactamente 10 o exactamente 300 se acepta.

AC-S04-04 (pertenencia y rol). Dada la incidencia de `solicitante1` en `PENDIENTE_VALIDACION`, cuando `solicitante2` intenta confirmarla o rechazarla, entonces recibe 404 sin cambios. Cuando lo intenta el coordinador o el técnico asignado, entonces recibe 403 sin cambios. Lo mismo aplica a la reapertura de una incidencia `CERRADA`.

AC-S04-05 (estados incompatibles). Dada una incidencia propia en `REGISTRADA`, `ASIGNADA`, `EN_ATENCION` o `CERRADA`, cuando su dueño intenta confirmarla o rechazarla, entonces recibe 409 sin cambios. Dada una incidencia propia en un estado distinto de `CERRADA`, cuando su dueño intenta reabrirla, entonces recibe 409 sin cambios.

AC-S04-06 (reabrir en el límite). Dada una incidencia propia `CERRADA` cuyo último cierre ocurrió en `2026-10-01T10:00:00Z` con técnico `tecnico1`, cuando su dueño la reabre con el motivo “La fuga volvió a aparecer” y el reloj del servidor marca `2026-10-03T10:00:00Z` (exactamente 48 horas), entonces queda `EN_ATENCION` con `tecnico1`, se conservan el cierre y la solución anteriores, y el historial suma un evento `REABRIR` con el motivo.

AC-S04-07 (reabrir fuera de plazo). Dada la misma incidencia `CERRADA`, cuando su dueño intenta reabrirla y el reloj del servidor marca `2026-10-03T10:00:00.000001Z` (48 horas y un microsegundo), entonces recibe 409 y la incidencia sigue `CERRADA` con el mismo historial.

AC-S04-08 (motivo de reapertura inválido). Dada una incidencia propia `CERRADA` dentro del plazo, cuando su dueño la reabre con un motivo de 9 caracteres, de 301, vacío o formado solo por espacios, entonces recibe 400 y sigue `CERRADA` sin eventos nuevos.

AC-S04-09 (cierres sucesivos). Dada una incidencia reabierta a la que el técnico registró una solución nueva, cuando su dueño la confirma, entonces queda `CERRADA` con dos cierres y dos soluciones conservados, y el plazo de una nueva reapertura se cuenta desde el segundo cierre.

AC-S04-10 (hora del servidor). Dada una incidencia propia `CERRADA` fuera de plazo, cuando su dueño envía la reapertura con un campo de fecha dentro del plazo, entonces el campo se ignora y recibe 409.

AC-S04-11 (sin efectos parciales). Dada una incidencia propia `PENDIENTE_VALIDACION` o `CERRADA` dentro del plazo, cuando falla el guardado del evento al confirmar, rechazar o reabrir, entonces la transacción se revierte: el estado no cambia, no queda ningún cierre nuevo y el historial no cambia.

4.4.9 Diseño y tareas vinculadas a cada AC

Diseño en plan.md; tareas en tasks.md.

Tabla 37. *Diseño y tareas vinculadas a cada AC (4)*

AC	Tareas
AC-S04-01, 09	T-S04-01, T-S04-02, T-S04-05
AC-S04-02, 03	T-S04-03, T-S04-05
AC-S04-04, 05	T-S04-02, T-S04-03, T-S04-04
AC-S04-06, 07, 08, 10	T-S04-04, T-S04-05
Todos	T-S04-06
AC-S04-11 y motivo solo con espacios	T-S04-07

4.4.10 Pruebas y resultados esperados

El resultado esperado se define aquí, antes de ejecutar, y no se calcula con la función de producción. Salvo que se indique otra cosa, cada prueba parte de una base SQLite nueva y aislada que solo contiene las cinco cuentas del seed, con el reloj del servidor fijado en `2026-10-01T08:00:00Z`. Todas las pruebas de esta tabla existen en `tests/` y se ejecutaron dentro de la batería completa. Los resultados están en `docs/validacion/validacion.md` y la ejecución sobre el tag de entrega, en `docs/validacion/ejecucion-final.md`.

Tabla 38. *Pruebas y resultados esperados (4)*

ID	AC	Precondición y entrada	Esperado	Prueba automatizada
P-S04-01	AC-S04-01	Integración: registrar → asignar → iniciar → proponer → confirmar, cada paso con la sesión de su actor	CERRADA; técnico tecnico1; 1 solución; 1 cierre; 5 eventos en orden CREAR, ASIGNAR, INICIAR_ATENCION, REGISTRAR_SOLUCION, CONFIRMAR_SOLUCION, con actores solicitante1, coordinador1, tecnico1, tecnico1, solicitante1	test_p_s04_01_flujo_de_cierre
P-S04-02	AC-S04-02, AC-S03-06	Integración: … → proponer → rechazar «El daño sigue presente» → nueva solución → confirmar	Tras el rechazo: EN_ATENCION, tecnico1, 1 solución. Al final: CERRADA, 2 soluciones en orden, 1 cierre enlazado a la segunda, 7 eventos	test_p_s04_02_rechazo_y_nueva_solucion
P-S04-03	AC-S04-06, 07, 09	Integración: cierre en 2026-10-01T10:00Z → reapertura a las 48 h exactas → nueva solución → nuevo cierre en 2026-10-03T14:00Z → reapertura a 48 h + 1 µs del segundo cierre → reapertura a las 48 h exactas del segundo cierre	302 y EN_ATENCION con tecnico1 y 1 cierre; después CERRADA con 2 soluciones, 2 cierres y 8 eventos; después 409 sin cambios; después 302	test_p_s04_03_reapertura_y_nuevo_cierre
P-S04-04	AC-S04-03	PENDIENTE_VALIDACION con 4 eventos; rechazo con motivo de 9, 301, vacío, 9 rodeado de espacios, 10 y 300 caracteres	9, 301, vacío y con espacios: 400, PENDIENTE_VALIDACION, 4 eventos. 10 y 300: 302, EN_ATENCION	test_p_s04_04_limites_del_motivo_de_rechazo
P-S04-05	AC-S04-04, AC-S03-07	PENDIENTE_VALIDACION de solicitante1; confirman y rechazan solicitante2, coordinador1, tecnico1 y tecnico2	solicitante2: 404. Los demás: 403. PENDIENTE_VALIDACION; 4 eventos; 0 cierres	test_p_s04_05_solo_el_duenio_valida
P-S04-06	AC-S04-01	PENDIENTE_VALIDACION; a las 2026-10-01T10:00Z el dueño confirma	302; CERRADA; 1 cierre con dueño, fecha y la solución vigente; evento enlazado al cierre	test_confirmar
P-S04-07	AC-S04-02	PENDIENTE_VALIDACION; el dueño rechaza con «El daño sigue presente»	302; EN_ATENCION; tecnico1; 1 solución intacta; 0 cierres; evento con actor, fecha, motivo y solución rechazada	test_rechazar_conserva_tecnico_y_solucion
P-S04-08	AC-S04-04	CERRADA de solicitante1; reabren solicitante2, coordinador1 y tecnico1	404; 403; 403. CERRADA; 5 eventos	test_solo_el_duenio_reabre

ID	AC	Precondición y entrada	Esperado	Prueba automatizada
P-S04-09	AC-S04-05	El dueño confirma y rechaza en `REGISTRADA`, `ASIGNADA`, `EN_ATENCION` y `CERRADA`	409 en los ocho casos; mismo estado y mismo historial	`test_validar_en_estado_incompatible`
P-S04-10	AC-S04-05	El dueño reabre en `REGISTRADA`, `ASIGNADA`, `EN_ATENCION` y `PENDIENTE_VALIDACION`	409 en los cuatro casos; mismo estado y mismo historial	`test_reabrir_en_estado_incompatible`
P-S04-11	AC-S04-06	Último cierre en `2026-10-01T10:00:00Z`; reapertura con el reloj en `2026-10-03T10:00:00Z`	302; `EN_ATENCION`; `tecnico1`; 1 cierre y 1 solución conservados; evento `REABRIR` con motivo y fecha	`test_reabrir_exactamente_a_las_48_horas`
P-S04-12	AC-S04-07	Mismo cierre; reapertura con el reloj en `2026-10-03T10:00:00.000001Z`	409; `CERRADA`; los mismos 5 eventos	`test_reabrir_un_instante_despues_de_48_horas`
P-S04-13	AC-S04-08	`CERRADA` dentro del plazo; reapertura con motivo de 9, de 301 y vacío	400; `CERRADA`; 5 eventos	`test_motivo_de_reapertura_invalido`
P-S04-14	AC-S04-10	`CERRADA` hace 72 h; la reapertura envía campos `fecha` y `ahora` dentro del plazo	409; `CERRADA`	`test_la_hora_la_decide_el_servidor`
P-S04-15	AC-S04-03, 08	Rechazo (`PENDIENTE_VALIDACION`, 4 eventos) y reapertura (`CERRADA` dentro del plazo, 5 eventos) con un motivo de 15 espacios, de 300 espacios, y de tabulaciones y saltos de línea	400; mismo estado y mismo técnico; eventos idénticos fila a fila; sin soluciones ni cierres nuevos	`test_p_s04_15_rechazo_con_motivo_solo_de_espacios`, `test_p_s04_15_reapertura_con_motivo_solo_de_espacios` (commit `6b99fec`)
P-S04-16	AC-S04-11	Confirmar, rechazar y reabrir mientras se fuerza un fallo al guardar el evento	La operación falla; las cinco tablas quedan idénticas fila a fila (mismo estado, 0 cierres nuevos, mismo historial); sin el fallo, la misma operación se completa	`test_un_fallo_al_guardar_el_evento_revierte_toda_la_operacion`, casos `P-S04-16-confirmar`, `P-S04-16-rechazar` y `P-S04-16-reabrir`, en `tests/test_atomicidad.py` (commit `6b99fec`)

Las pruebas del plazo usan un reloj controlado inyectado en la aplicación. No se espera tiempo real ni se cambia el reloj del sistema.

4.4.11 Decisión de revisión / versión aprobada

Tabla 39. *Decisión de revisión / versión aprobada (4)*

Fecha	Versión revisada	Revisor	Decisión	Observaciones
Pendiente	0.5	Cristian David Diaz España (propuesto)	Pendiente	

4.4.12 Plan de diseño · S04 Validar

Versión 0.5 · 2026-10-08 · ajustado a la implementación · depende de arquitectura, S01, S02 y S03. Secuencia del rechazo en secuencia-rechazo.puml.

Componentes que intervienen

Tabla 40. *Componentes que intervienen (4)*

Capa	Elemento	Responsabilidad
Rutas	`POST /incidencias/<codigo>/confirmar`, `/rechazar`, `/reabrir`	Leer `motivo` cuando aplica y llamar al servicio.
Dominio	`dominio.validar_texto(motivo, 10, 300, "motivo")`	Recorte y límites.
Dominio	`dominio.dentro_de_plazo_reapertura(cerrada_en, ahora)`	Función pura: `ahora − cerrada_en <= 48 h`.
Servicio	`servicios.confirmar`, `servicios.rechazar`, `servicios.reabrir`	Comprobaciones y transacción.
Reloj	`reloj.ahora_utc()`	Única fuente de la hora; reemplazable en pruebas mediante la configuración de la aplicación.
Persistencia	Tabla `cierres`, solo inserción	Un registro por cada confirmación; nunca se borra al reabrir.

Decisiones

- El "último cierre" es la fila de `cierres` más reciente de la incidencia (la de mayor identificador). La "solución vigente" es la fila de `soluciones` más reciente (`servicios._solucion_vigente`). No se guarda una fecha de cierre en la incidencia, para que no exista un dato que haya que limpiar al reabrir.
- La reapertura no borra ni marca el cierre: el evento `REABRIR` referencia el cierre que se reabre.
- El plazo se evalúa con objetos de fecha con zona UTC, no comparando cadenas.
- Fuera de plazo se responde 409 porque es una condición del estado del recurso, no un dato mal formado.
- El rechazo no crea fila en una tabla propia: el motivo vive en el evento, que es inmutable.

Orden de comprobaciones

Sesión → rol → visibilidad (pertenencia) → estado → plazo (solo reabrir) → validación del motivo → transacción.

4.4.13 Tareas · S04 Validar

Versión 0.5 · 2026-10-08. Cada tarea se registra como issue de GitHub con su RF, SPEC y AC.

El estado refleja lo que existe en el repositorio. «Implementada» significa que el código y sus pruebas existen y están integrados en `main`, no que la tarea esté terminada: ninguna se da por terminada hasta que otro integrante la revise. La integración la hizo el autor sin esa revisión. El revisor lo define el equipo.

Tabla 41. *Tareas · S04 Validar*

Tarea	Descripción	AC	Archivos	Prueba	Responsable	Revisor	Estado
T-S04-01	Tabla `cierres` de solo inserción.	AC-S04-01, 09	`app/schema.sql`	P-S04-01	Luis Carlo Daza Ospino, con IA	Por asignar	Implementada en `c5e7ef7`; integrada en `main`; revisión de otro integrante pendiente
T-S04-02	Servicio de confirmación: cierre, estado y evento.	AC-S04-01, 04, 05, 09	`app/servicios.py`	P-S04-01, P-S04-05	Luis Carlo Daza Ospino, con IA	Por asignar	Implementada en `70468c0`; integrada en `main`; revisión de otro integrante pendiente
T-S04-03	Servicio de rechazo con motivo de 10 a 300 caracteres.	AC-S04-02 a 05	`app/servicios.py`, `app/dominio.py`	P-S04-02, P-S04-04	Luis Carlo Daza Ospino, con IA	Por asignar	Implementada en `70468c0`; integrada en `main`; revisión de otro integrante pendiente
T-S04-04	Reloj inyectable y servicio de reapertura con plazo de 48 horas.	AC-S04-04 a 08, 10	`app/reloj.py`, `app/servicios.py`, `app/dominio.py`	P-S04-03	Luis Carlo Daza Ospino, con IA	Por asignar	Implementada en `c5e7ef7`, `70468c0`; integrada en `main`; revisión de otro integrante pendiente
T-S04-05	Rutas y bloques de la interfaz del solicitante.	AC-S04-01, 02, 06	`app/rutas.py`, `app/templates/incidencia_detalle.html`	P-S04-01 a 03	Luis Carlo Daza Ospino, con IA	Por asignar	Implementada en `70468c0`; integrada en `main`; revisión de otro integrante pendiente
T-S04-06	Pruebas automatizadas de S04, incluidas las tres de integración.	Todos	`tests/test_s04_validacion.py`, `tests/test_integracion.py`	P-S04-01 a 14	Luis Carlo Daza Ospino, con IA	Por asignar	Implementada en `70468c0`; integrada en `main`; revisión de otro integrante pendiente
T-S04-07	Pruebas de motivo solo con espacios y de que confirmar, rechazar y reabrir no dejan efectos parciales.	AC-S04-03, 08, 11	`tests/test_s04_validacion.py`, `tests/test_atomicidad.py`	P-S04-15, P-S04-16	Luis Carlo Daza Ospino, con IA	Por asignar	Implementada en `6b99fec`; integrada en `main`; revisión de otro integrante pendiente

4.5 SPEC S05 · Consultar, historizar y tablero

Tabla 42. *SPEC S05 · Consultar, historizar y tablero*

Campo	Valor
Versión	0.5 (integrada en `main`; revisión de otro integrante pendiente)
Autor	Luis Carlo Daza Ospino, con asistencia de IA (Claude)
Revisor	Propuesto: Jorge Luis González Arroyo. El equipo no ha confirmado la asignación; revisión pendiente
Fecha	2026-10-08
Requisito asociado	RF05

Cambios de la versión 0.2 (2026-10-08): se distingue filtro inválido de filtro válido sin resultados y se aclara quién ve el historial. Origen: revisión asistida por IA, que no reemplaza la revisión del integrante asignado.

Cambios de la versión 0.3 (2026-10-08): se define el orden y el desempate del historial; AC-S05-04 incluye al técnico asignado; la tabla de pruebas refleja las pruebas reales (inmutabilidad en las tres tablas, operaciones rechazadas, cantidades del tablero y redondeo). Origen: segunda revisión asistida por IA, que tampoco reemplaza la revisión del integrante asignado.

Cambios de la versión 0.5 (2026-10-08): sin cambios de requisitos ni de criterios. La SPEC queda integrada en `main` junto con el código y las pruebas; las referencias a pruebas ya no dependen de una rama; el revisor figura como propuesto, porque el equipo no ha confirmado la asignación.

4.5.1 Historia (H05)

Como **coordinador, solicitante o técnico**, quiero **consultar las incidencias que me corresponden, con filtros y con su historial completo, y como coordinador ver un tablero de resumen**, para **saber en qué va cada caso y comprobar qué ocurrió sin depender de mensajes sueltos**.

4.5.2 Alcance y exclusiones

Incluye: listado según permisos con filtros por estado y prioridad; detalle con historial; historial inmutable; tablero del coordinador con cantidades por estado y prioridad, críticas no cerradas y porcentaje de cierre.

Excluye: edición o borrado del historial, búsqueda por texto, paginación, exportación, gráficos, métricas de tiempos y tablero para otros roles.

4.5.3 Entradas

Tabla 43. *Entradas (6)*

Operación	Campo	Tipo	Obligatorio	Valores o límites
Listar	`estado`	texto	No	Vacío, o uno de los cinco estados.
Listar	`prioridad`	texto	No	Vacío, o `CRITICA`, `ALTA`, `NORMAL`.
Detalle	`codigo` (en la ruta)	texto	Sí	Código de una incidencia visible para el usuario.
Tablero	—	—	—	Sin parámetros.

4.5.4 Precondiciones y permisos

Todas las consultas exigen sesión iniciada. La visibilidad la decide el servidor con la identidad de la sesión:

Tabla 44. *Precondiciones y permisos*

Rol	Listado y detalle	Tablero
Solicitante	Solo las incidencias que registró.	403
Técnico	Solo las incidencias que tiene asignadas.	403
Coordinador	Todas.	Permitido

Abrir el detalle de una incidencia inexistente o no visible responde 404 (supuesto Q1).

4.5.5 Reglas y proceso

Listado. Aplicar primero la visibilidad por rol y después los filtros. Con ambos filtros, se combinan con "y". Un filtro vacío no restringe. Un valor fuera del catálogo responde 400. Orden: más recientes primero.

Historial. Cada creación y cada transición exitosa agrega un evento con autor, fecha UTC, estado anterior, estado nuevo y acción. Según la acción, el evento enlaza la solución, el cierre o el motivo. El detalle muestra los eventos en el orden en que se registraron, que coincide con el cronológico. Si dos eventos tienen la misma marca de tiempo, el desempate es el identificador consecutivo del evento, que la base asigna al insertarlo. Se muestran con el texto de las soluciones, los motivos de rechazo y reapertura, y los cierres.

Quien puede ver el detalle de una incidencia ve su historial completo: el solicitante dueño, el técnico asignado y el coordinador.

Inmutabilidad. No existe ninguna operación para editar o borrar eventos, soluciones o cierres. La base de datos rechaza cualquier `UPDATE` o `DELETE` sobre esas tablas. Una operación rechazada no agrega eventos.

Tablero.

- Cantidad por estado, para los cinco estados, incluidos los que están en cero.
- Cantidad por prioridad, para las tres prioridades, incluidas las que están en cero.
- Lista de incidencias con prioridad `CRITICA` cuyo estado actual no es `CERRADA`.
- Porcentaje de cierre = incidencias actualmente `CERRADAS` / total × 100, con un decimal, redondeo aritmético y coma decimal. Con cero incidencias muestra `0,0 %`.

4.5.6 Salidas y cambios persistidos

Las consultas no modifican datos.

- Listado: código, ubicación, categoría, prioridad, estado, fecha y, según el rol, solicitante y técnico.
- Detalle: datos de la incidencia e historial.
- Tablero: los cuatro bloques anteriores.

4.5.7 Errores y efectos que deben evitarse

Tabla 45. *Errores y efectos que deben evitarse (5)*

Situación	Respuesta
Sin sesión	302 al inicio de sesión
Filtro con valor fuera del catálogo	400
Detalle de una incidencia inexistente o no visible	404
Tablero solicitado por solicitante o técnico	403

No debe ocurrir: que un filtro amplíe la visibilidad; que el listado o el detalle revelen incidencias ajenas; que un evento cambie o desaparezca; que el texto del usuario se interprete como HTML; un porcentaje calculado sobre cierres históricos en lugar del estado actual.

4.5.8 Criterios de aceptación

AC-S05-01 (visibilidad del listado). Dadas dos incidencias de `solicitante1` y una de `solicitante2`, de las cuales una está asignada a `tecnico1`, cuando cada usuario abre el listado, entonces `solicitante1` ve sus dos, `solicitante2` ve la suya, `tecnico1` ve solo la asignada, `tecnico2` no ve ninguna y el coordinador ve las tres.

AC-S05-02 (filtros). Dado el coordinador y un conjunto con incidencias en distintos estados y prioridades, cuando filtra por `estado=ASIGNADA`, entonces ve solo las `ASIGNADA`; cuando filtra por `prioridad=CRITICA`, entonces ve solo las `CRITICA`; cuando aplica ambos, entonces ve solo las que cumplen los dos. Dado un solicitante, cuando filtra, entonces el resultado nunca incluye incidencias ajenas.

AC-S05-03 (filtro inválido). Dado un usuario con sesión iniciada, cuando lista con `estado=ABIERTA` o `prioridad=URGENTE`, entonces recibe 400. Cuando lista con un valor válido que ninguna incidencia visible cumple, entonces recibe 200 con la lista vacía.

AC-S05-04 (historial completo). Dada una incidencia que recorrió registro, asignación, inicio, solución, rechazo, nueva solución, confirmación y reapertura, cuando su dueño, el técnico asignado o el coordinador abre el detalle, entonces el historial muestra ocho eventos en orden cronológico, cada uno con autor, fecha, estado anterior, estado nuevo y acción, e incluye el texto de las dos soluciones, el motivo del rechazo, el cierre y el motivo de la reapertura.

AC-S05-05 (historial inmutable). Dado un historial con eventos, cuando se intenta ejecutar `UPDATE` o `DELETE` sobre `eventos`, `soluciones` o `cierres`, entonces la base de datos lo rechaza y los registros no cambian. No existe ninguna ruta de la aplicación para editar o borrar historial.

AC-S05-06 (operación rechazada sin rastro). Dada una incidencia con N eventos, cuando se rechaza una operación por permiso, estado, plazo o datos inválidos, entonces la incidencia conserva su estado y sigue teniendo exactamente N eventos.

AC-S05-07 (tablero vacío). Dado el coordinador y ninguna incidencia registrada, cuando abre el tablero, entonces ve cero en todos los estados y prioridades, ninguna crítica pendiente y un porcentaje de cierre de `0,0 %`.

AC-S05-08 (porcentaje de cierre). Dadas cuatro incidencias de las cuales una está `CERRADA`, cuando el coordinador abre el tablero, entonces el porcentaje de cierre es `25,0 %`. Cuando esa incidencia se reabre, entonces el porcentaje es `0,0 %` y su cierre anterior sigue en el historial.

AC-S05-09 (cantidades y críticas). Dadas cuatro incidencias —dos `CRITICA` (una `CERRADA`, una `ASIGNADA`), una `ALTA` `REGISTRADA` y una `NORMAL` `REGISTRADA`—, cuando el coordinador abre el tablero, entonces ve por estado: `REGISTRADA` 2, `ASIGNADA` 1, `EN_ATENCION` 0, `PENDIENTE_VALIDACION` 0, `CERRADA` 1; por prioridad: `CRITICA` 2, `ALTA` 1, `NORMAL` 1; y en críticas no cerradas solo la `ASIGNADA`.

AC-S05-10 (permiso del tablero). Dado un solicitante o un técnico con sesión iniciada, cuando abre el tablero, entonces recibe 403.

AC-S05-11 (detalle ajeno). Dada una incidencia de `solicitante1`, cuando `solicitante2` abre su detalle, entonces recibe 404. Dada una incidencia asignada a `tecnico1`, cuando `tecnico2` abre su detalle, entonces recibe 404.

AC-S05-12 (persistencia). Dadas incidencias y eventos guardados, cuando la aplicación se detiene y vuelve a iniciar sobre el mismo archivo de base de datos, entonces el listado, el detalle y el historial muestran los mismos datos.

AC-S05-13 (acceso con cuentas). Dado un visitante sin sesión, cuando abre el listado, un detalle o el tablero, entonces es redirigido al inicio de sesión. Dadas credenciales incorrectas, cuando intenta iniciar sesión, entonces no obtiene sesión.

4.5.9 Diseño y tareas vinculadas a cada AC

Diseño en plan.md; tareas en tasks.md.

Tabla 46. *Diseño y tareas vinculadas a cada AC (5)*

AC	Tareas
AC-S05-01, 02, 03, 13	T-S05-01, T-S05-04
AC-S05-04, 11	T-S05-02, T-S05-04
AC-S05-05, 06	T-S05-03
AC-S05-07 a 10	T-S05-05
AC-S05-12	T-S05-06
Todos	T-S05-06

4.5.10 Pruebas y resultados esperados

El resultado esperado se define aquí, antes de ejecutar, y no se calcula con la función de producción. Salvo que se indique otra cosa, cada prueba parte de una base SQLite nueva y aislada que solo contiene las cinco cuentas del seed, con el reloj del servidor fijado en `2026-10-01T08:00:00Z`. Todas las pruebas de esta tabla existen en `tests/` y se ejecutaron dentro de la batería completa. Los resultados están en `docs/validacion/validacion.md` y la ejecución sobre el tag de entrega, en `docs/validacion/ejecucion-final.md`.

Tabla 47. *Pruebas y resultados esperados (5)*

ID	AC	Precondición y entrada	Esperado	Prueba automatizada
P-S05-01	AC-S05-01, 11	Tres incidencias: `INC-000001` de `solicitante1` `ASIGNADA` a `tecnico1`, `INC-000002` de `solicitante1`, `INC-000003` de `solicitante2`. Cada usuario abre el listado y varios detalles	Listado: `solicitante1` 2, `solicitante2` 1, `tecnico1` 1, `tecnico2` 0, `coordinador1` 3. Detalle propio 200; ajeno o inexistente 404	`test_p_s05_01_listado_segun_permisos`, `test_p_s05_01_detalle_segun_permisos`
P-S05-02	AC-S05-02, 03	Cuatro incidencias: 001 `CRITICA` `ASIGNADA`, 002 `CRITICA` `REGISTRADA`, 003 `ALTA` `ASIGNADA`, 004 `NORMAL` `REGISTRADA` de `solicitante2`. Filtros `estado=ASIGNADA`, `prioridad=CRITICA`, ambos, vacíos, `estado=CERRADA`, `estado=ABIERTA`, `prioridad=URGENTE`	003 y 001; 002 y 001; 001; las cuatro; 200 y lista vacía; 400; 400. Un solicitante o un técnico nunca ve incidencias ajenas al filtrar	`test_p_s05_02_filtros`, `test_p_s05_02_filtro_invalido`, `test_un_filtro_no_amplia_la_visibilidad`
P-S05-03	AC-S05-07, 08	Tablero sin incidencias; con una `CERRADA` de cuatro; después de reabrirla	`0,0 %` y ceros en todo; `25,0 %`; `0,0 %` con 1 cierre conservado	`test_p_s05_03_tablero_vacio`, `test_p_s05_03_porcentaje_de_cierre_y_reapertura`
P-S05-04	AC-S05-05	Con una incidencia `CERRADA`, `UPDATE` y `DELETE` directos sobre `eventos`, `soluciones` y `cierres`	La base rechaza las seis operaciones; el contenido de las tres tablas no cambia	`test_p_s05_04_el_historial_es_inmutable`
P-S05-05	AC-S05-12	Se crea una incidencia `CERRADA`, y una segunda instancia de la aplicación abre el mismo archivo de base de datos	Mismo listado, mismo detalle y los mismos eventos fila a fila; siguen existiendo 5 usuarios	`test_p_s05_05_los_datos_permanecen_al_reiniciar`
P-S05-06	AC-S05-10, 13	Tablero como `solicitante1` y `tecnico1`; listado, detalle, tablero y formulario sin sesión	403; 403; 302 al inicio de sesión en los cuatro	`test_p_s05_06_tablero_solo_para_el_coordinador`, `test_p_s05_06_sin_sesion_redirige_al_login`
P-S05-07	AC-S05-04	Una incidencia recorre registro, asignación, inicio, solución, rechazo, nueva solución, confirmación y reapertura; abren el detalle `solicitante1`, `coordinador1` y `tecnico1`	8 eventos con acción, estado anterior y nuevo, actor y fecha; los tres ven las dos soluciones, el motivo del rechazo y el de la reapertura, en ese orden	`test_historial_completo`

ID	AC	Precondición y entrada	Esperado	Prueba automatizada
P-S05-08	AC-S05-06	`PENDIENTE_VALIDACION` con 4 eventos; siete operaciones no permitidas: asignar y confirmar como coordinador, iniciar como `tecnico1`, solución como `tecnico2`, confirmar como `solicitante2`, rechazo con motivo corto y reapertura	Respuestas 409, 403, 409, 404, 404, 400 y 409; `PENDIENTE_VALIDACION` con `tecnico1`; eventos idénticos fila a fila; 1 solución, 0 cierres, 1 asignación	`test_las_operaciones_rechazadas_no_dejan_rastro`
P-S05-09	AC-S05-09	Cuatro incidencias: `CRITICA CERRADA`, `CRITICA ASIGNADA`, `ALTA REGISTRADA`, `NORMAL REGISTRADA`	Por estado 2, 1, 0, 0, 1; por prioridad 2, 1, 1; críticas no cerradas: solo `INC-000002`; `25,0 %`	`test_tablero_cantidades_y_criticas`
P-S05-10	AC-S05-07, 08	Porcentaje para 0 de 0, 1 de 4, 1 de 3, 2 de 3, 1 de 16 y 5 de 5	`0,0 %`; `25,0 %`; `33,3 %`; `66,7 %`; `6,3 %`; `100,0 %`	`test_formato_y_redondeo_del_porcentaje`
P-S05-11	AC-S05-05	Se listan las rutas de la aplicación	Ninguna acepta `PUT`, `PATCH` o `DELETE`; solo nueve aceptan `POST`: inicio y cierre de sesión, registro y las seis transiciones	`test_no_hay_rutas_para_editar_o_borrar`
P-S05-12	AC-S01-07	Descripción con `<b onmouseover=alert(1)>`; el coordinador abre el detalle	Se muestra escapada	`test_el_listado_no_ejecuta_html`

4.5.11 Decisión de revisión / versión aprobada

Tabla 48. *Decisión de revisión / versión aprobada (5)*

Fecha	Versión revisada	Revisor	Decisión	Observaciones
Pendiente	0.5	Jorge Luis González Arroyo (propuesto)	Pendiente	

4.5.12 Plan de diseño · S05 Consultar, historizar y tablero

Versión 0.5 · 2026-10-08 · ajustado a la implementación · depende de arquitectura y de S01 a S04.

Componentes que intervienen

Tabla 49. *Componentes que intervienen (5)*

Capa	Elemento	Responsabilidad
Rutas	`GET /incidencias/`	Leer filtros, llamar a la consulta, mostrar la lista.
Rutas	`GET /incidencias/<codigo>`	Detalle e historial.
Rutas	`GET /tablero`	Resumen del coordinador.
Dominio	`dominio.validar_filtros(estado, prioridad)`	Catálogo o vacío.
Dominio	`dominio.porcentaje_cierre(cerradas, total)`	Función pura con `Decimal` y redondeo mitad hacia arriba; devuelve texto como `25,0 %`.

Capa	Elemento	Responsabilidad
Consultas	`consultas.listar(db, usuario, estado, prioridad)`	Aplica la visibilidad por rol y después los filtros, con parámetros SQL.
Consultas	`consultas.historial(db, incidencia_id)`	Eventos con autor, solución, cierre y motivo.
Consultas	`consultas.tablero(db)`	Conteos, críticas no cerradas y porcentaje.
Servicio	`servicios._registrar_evento(...)`	Único punto que inserta eventos; lo usan todas las transiciones.
Persistencia	Disparadores `BEFORE UPDATE` y `BEFORE DELETE` en `eventos`, `soluciones` y `cierres`	Inmutabilidad garantizada por la base.

Decisiones

- El historial se ordena por el identificador del evento (`ORDER BY e.id`), que es el orden de inserción y resuelve los empates de fecha.
- La condición de visibilidad se construye en un único lugar y se reutiliza en listado, detalle y operaciones, para que un filtro no pueda ampliarla.
- Los filtros se pasan como parámetros de la consulta, nunca concatenados en el SQL.
- El tablero cuenta el estado actual de `incidencias`; no usa la tabla `cierres`, que conserva los cierres históricos.
- Los conteos parten del catálogo de estados y prioridades para mostrar también los ceros.
- La persistencia se prueba abriendo dos instancias sucesivas de la aplicación sobre el mismo archivo SQLite.

Orden de comprobaciones

Sesión → rol (solo tablero) → validación de filtros → consulta con visibilidad.

4.5.13 Tareas · S05 Consultar, historizar y tablero

Versión 0.5 · 2026-10-08. Cada tarea se registra como issue de GitHub con su RF, SPEC y AC.

El estado refleja lo que existe en el repositorio. «Implementada» significa que el código y sus pruebas existen y están integrados en `main`, no que la tarea esté terminada: ninguna se da por terminada hasta que otro integrante la revise. La integración la hizo el autor sin esa revisión. El revisor lo define el equipo.

Tabla 50. *Tareas · S05 Consultar, historizar y tablero*

Tarea	Descripción	AC	Archivos	Prueba	Responsable	Revisor	Estado
T-S05-01	Consulta de listado con visibilidad por rol y filtros validados.	AC-S05-01, 02, 03	`app/consultas.py`, `app/dominio.py`	P-S05-01, P-S05-02	Luis Carlo Daza Ospino, con IA	Por asignar	Implementada en `786cf42`; integrada en `main`; revisión de otro integrante pendiente
T-S05-02	Consulta de historial con soluciones, motivos y cierres.	AC-S05-04, 11	`app/consultas.py`	P-S05-01	Luis Carlo Daza Ospino, con IA	Por asignar	Implementada en `7bc08f2`; integrada en `main`; revisión de otro integrante pendiente
T-S05-03	Disparadores de inmutabilidad y punto único de registro de eventos.	AC-S05-05, 06	`app/schema.sql`, `app/servicios.py`	P-S05-04	Luis Carlo Daza Ospino, con IA	Por asignar	Implementada en `c5e7ef7`, `7bc08f2`; integrada en `main`; revisión de otro integrante pendiente
T-S05-04	Rutas y plantillas de listado y detalle.	AC-S05-01 a 04, 13	`app/rutas.py`, `app/templates/`	P-S05-01, P-S05-02, P-S05-06	Luis Carlo Daza Ospino, con IA	Por asignar	Implementada en `7bc08f2`, `786cf42`; integrada en `main`; revisión de otro integrante pendiente
T-S05-05	Tablero del coordinador y cálculo del porcentaje.	AC-S05-07 a 10	`app/consultas.py`, `app/dominio.py`, `app/templates/tablero.html`	P-S05-03, P-S05-06	Luis Carlo Daza Ospino, con IA	Por asignar	Implementada en `786cf42`; integrada en `main`; revisión de otro integrante pendiente
T-S05-06	Pruebas automatizadas de S05, incluida la persistencia tras reinicio.	Todos	`tests/test_s05_consulta_tablero.py`	P-S05-01 a 12	Luis Carlo Daza Ospino, con IA	Por asignar	Implementada en `786cf42`; integrada en `main`; revisión de otro integrante pendiente

5. Diseño

Tabla 51. *Diseño*

Campo	Valor
Versión	0.4 (ajustada a la implementación integrada en `main`; revisión pendiente)
Autor	Luis Carlo Daza Ospino, con asistencia de IA (Claude)
Revisor	Propuesto: Rafael Eduardo May Recuero. El equipo no ha confirmado la asignación; revisión pendiente
Fecha	2026-10-08

La versión 0.1 se escribió antes de programar. Esta versión se ajustó a lo implementado (aplicación en el commit `786cf42`, pruebas hasta el commit `6b99fec`) tras la segunda revisión asistida por IA. Los diagramas están en PlantUML junto a este archivo.

5.1 Componentes

Monolito modular en tres capas. Diagrama: componentes.puml.

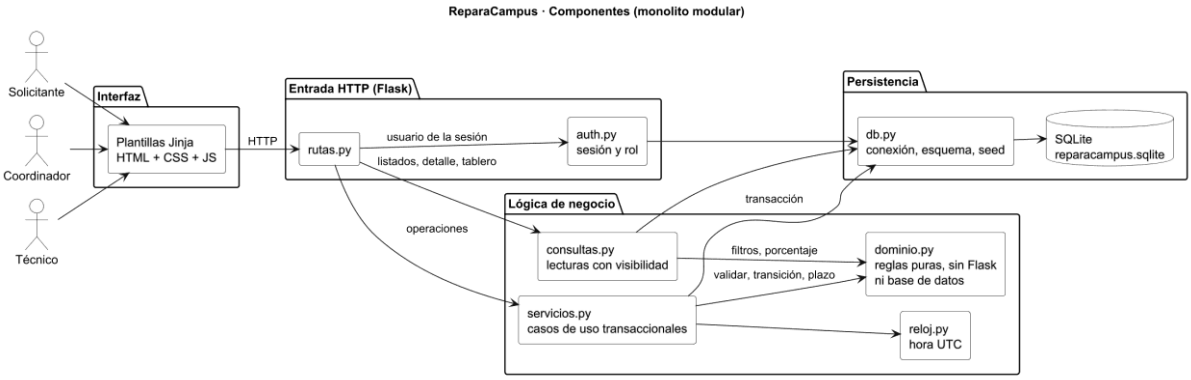


Figura 1. *Componentes del sistema. Fuente: docs/diseno/componentes.puml*

Tabla 52. *Componentes*

Capa	Módulo	Responsabilidad	No hace
Interfaz	`app/templates/`, `app/static/`	Formularios, listados, detalle, tablero. Muestra u oculta acciones según rol y estado.	No decide permisos ni reglas.
Entrada HTTP	`app/rutas.py`	Lee la petición, llama a servicios o consultas y muestra el resultado. Dos grupos de rutas: incidencias y tablero.	No contiene reglas de negocio ni SQL.
Entrada HTTP	`app/__init__.py`	Fábrica de la aplicación, configuración (base, reloj, método de hash) y traducción de los errores de dominio a códigos HTTP.	
Entrada HTTP	`app/auth.py`	Inicio y cierre de sesión, carga del usuario de la sesión, decoradores de sesión y rol.	
Lógica de negocio	`app/dominio.py`	Catálogos, validaciones, prioridad, tabla de transiciones, plazo de reapertura, porcentaje. Funciones puras.	No conoce Flask ni la base.
Lógica de negocio	`app/servicios.py`	Casos de uso que cambian datos. Cada uno es una transacción: estado + registros + evento.	

Capa	Módulo	Responsabilidad	No hace
Lógica de negocio	`app/consultas.py`	Lecturas con la visibilidad por rol.	No modifica datos.
Lógica de negocio	`app/reloj.py`	Única fuente de la hora UTC; reemplazable en pruebas.	
Persistencia	`app/db.py`, `app/schema.sql`	Conexión SQLite por petición, transacción todo o nada (`db.transaccion`), esquema y seed ficticio.	

Dónde se ejecutan las reglas: siempre en el servidor, en `dominio` y `servicios`. La interfaz solo evita mostrar acciones que el servidor rechazaría.

5.2 Modelo de datos

Diagrama: datos.puml.

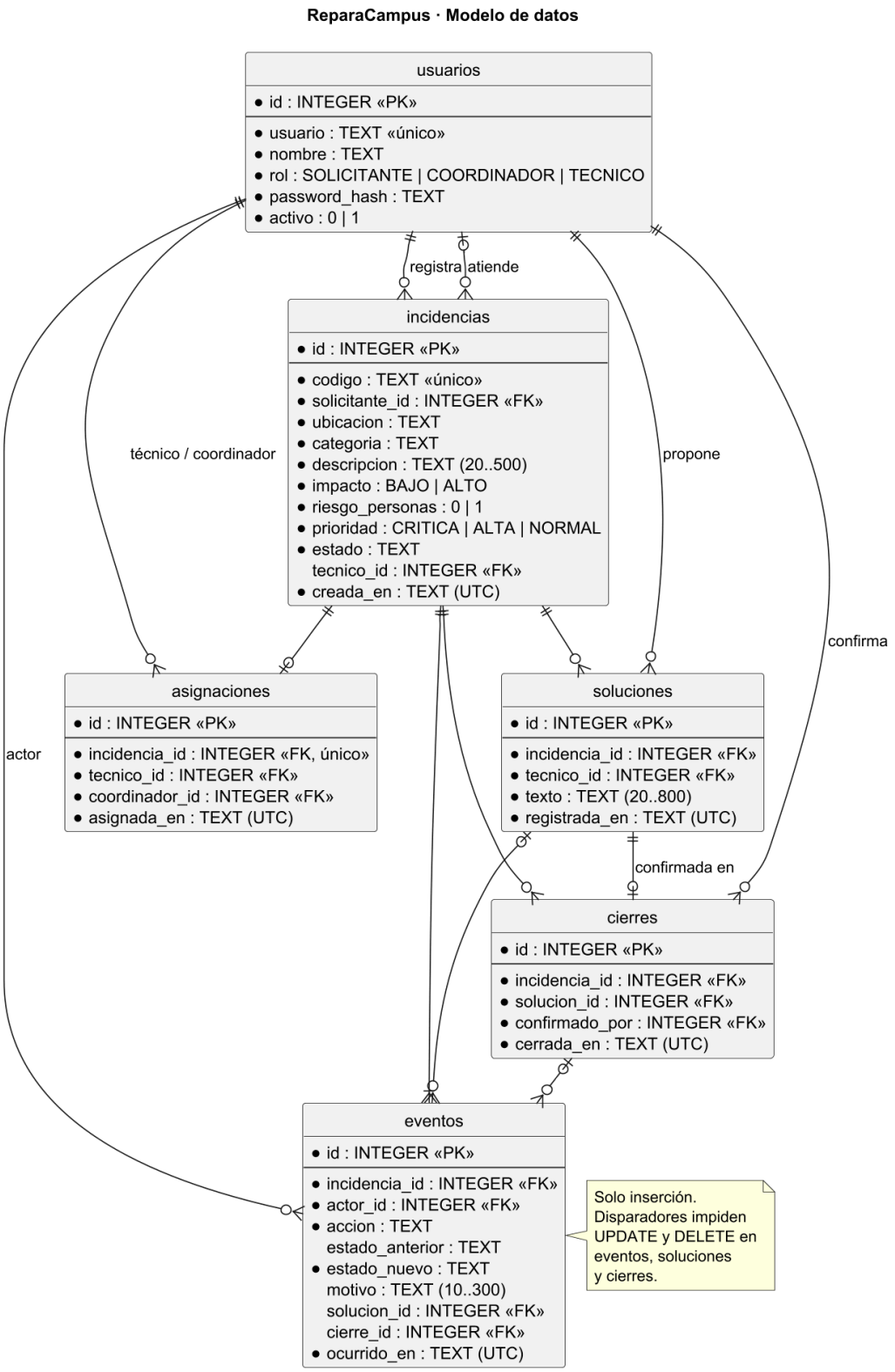


Figura 2. *Modelo de datos. Fuente: docs/diseno/datos.puml*

Tabla 53. *Modelo de datos*

Tabla	Contenido	Notas
`usuarios`	Cuenta, nombre, rol, hash de contraseña, activo.	Cinco cuentas ficticias en el seed.

Tabla	Contenido	Notas
`incidencias`	Código, solicitante, ubicación, categoría, descripción, impacto, riesgo, prioridad, estado actual, técnico.	Único registro que se actualiza; solo cambian `estado` y, una vez, `tecnico_id`.
`asignaciones`	Incidencia, técnico, coordinador, fecha.	`incidencia_id` único: no hay reasignación.
`soluciones`	Incidencia, técnico, texto, fecha.	Solo inserción.
`cierres`	Incidencia, solución confirmada, solicitante, fecha.	Solo inserción; uno por cada confirmación.
`eventos`	Incidencia, actor, acción, estado anterior, estado nuevo, motivo, solución, cierre, fecha.	Solo inserción; es el historial.

Las fechas se guardan como texto ISO 8601 en UTC con microsegundos (`2026-10-08T15:04:05.000000Z`). Los catálogos tienen restricciones `CHECK`. Las claves foráneas están activas.

`eventos`, `soluciones` y `cierres` tienen disparadores que abortan cualquier `UPDATE` o `DELETE`.

5.3 Estados y transiciones

Diagrama: estados.puml.

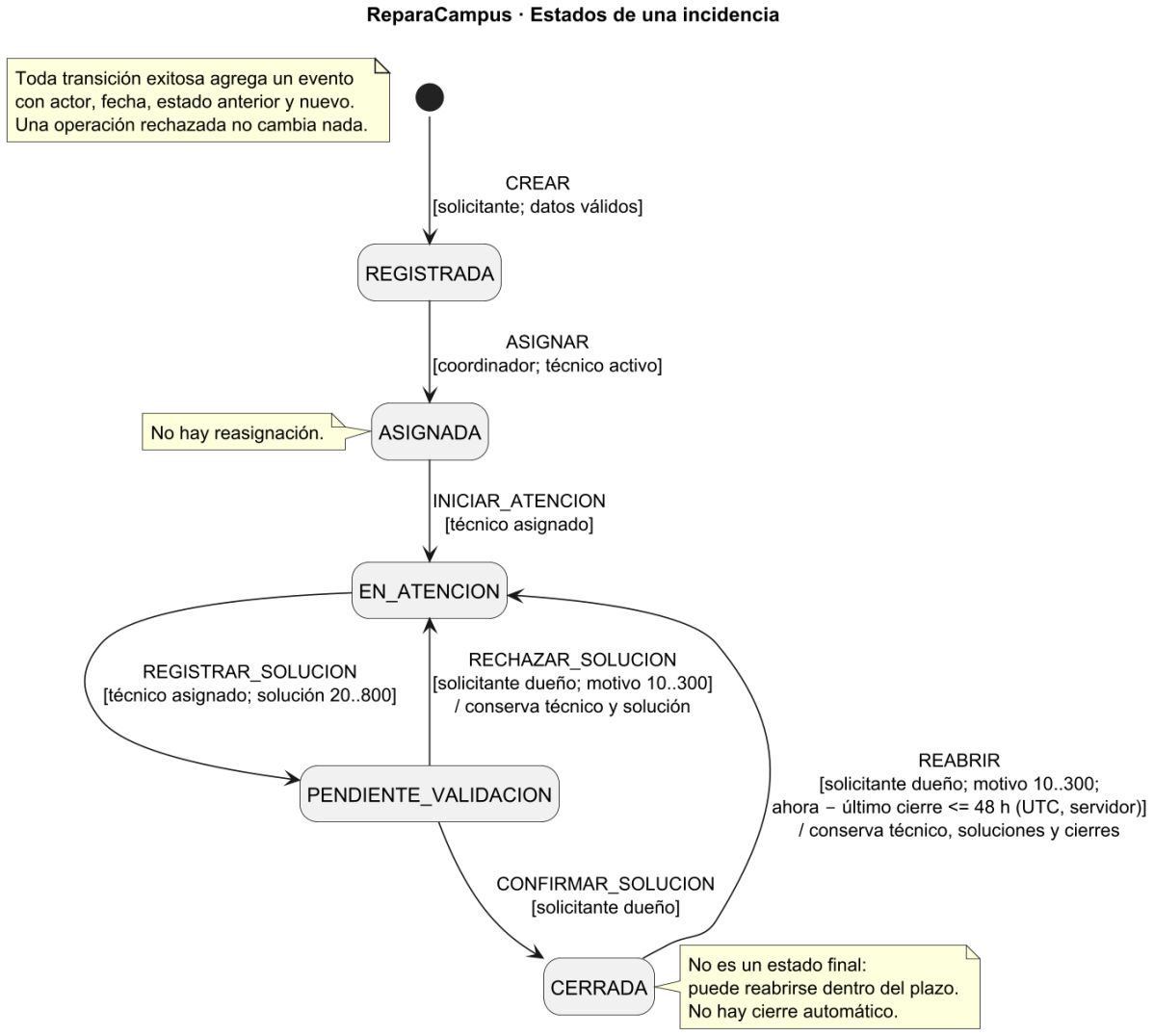


Figura 3. *Diagrama UML de estados de una incidencia. Fuente: docs/diseno/estados.puml*

Tabla 54. *Estados y transiciones*

Acción	Origen	Destino	Quién	SPEC
`CREAR`	—	`REGISTRADA`	Solicitante	S01
`ASIGNAR`	`REGISTRADA`	`ASIGNADA`	Coordinador	S02
`INICIAR_ATENCION`	`ASIGNADA`	`EN_ATENCION`	Técnico asignado	S03
`REGISTRAR_SOLUCION`	`EN_ATENCION`	`PENDIENTE_VALIDACION`	Técnico asignado	S03
`CONFIRMAR_SOLUCION`	`PENDIENTE_VALIDACION`	`CERRADA`	Solicitante dueño	S04
`RECHAZAR_SOLUCION`	`PENDIENTE_VALIDACION`	`EN_ATENCION`	Solicitante dueño	S04
`REABRIR`	`CERRADA`	`EN_ATENCION`	Solicitante dueño, hasta 48 h después del último cierre	S04

No existe ninguna otra transición. Esta tabla se implementa una sola vez en `dominio.TRANSICIONES` y todos los servicios la consultan.

5.4 Contratos compartidos

5.4.1 Rutas

Tabla 55. *Rutas*

Método y ruta	Rol	SPEC
`GET`, `POST /login`; `POST /logout`	Cualquiera	Autenticación
`GET /`	Con sesión	Redirige al listado o al tablero
`GET /incidencias/?estado=&prioridad=`	Con sesión	S05
`GET`, `POST /incidencias/nueva`	Solicitante	S01
`GET /incidencias/<codigo>`	Con visibilidad	S05
`POST /incidencias/<codigo>/asignar`	Coordinador	S02
`POST /incidencias/<codigo>/iniciar`	Técnico asignado	S03
`POST /incidencias/<codigo>/solucion`	Técnico asignado	S03
`POST /incidencias/<codigo>/confirmar`	Solicitante dueño	S04
`POST /incidencias/<codigo>/rechazar`	Solicitante dueño	S04
`POST /incidencias/<codigo>/reabrir`	Solicitante dueño	S04
`GET /tablero`	Coordinador	S05

Una operación exitosa responde 302 hacia el detalle. Un error de datos (400) o de estado (409) se muestra en el mismo detalle de la incidencia, con su código; un error de permiso (403) o de visibilidad (404) se muestra en una página de error. En el registro, los errores de datos se muestran en el formulario, campo por campo.

5.4.2 Errores

Tabla 56. *Errores*

Código	Significado	Excepción de dominio
302 a `/login`	No hay sesión	—
400	Dato ausente, fuera de catálogo o fuera de límites	`ErrorValidacion`
403	El rol no puede ejecutar la operación	`ErrorPermiso`

Código	Significado	Excepción de dominio
404	La incidencia no existe o el usuario no puede verla	`NoEncontrada`
409	El estado actual o el plazo no permiten la operación	`ErrorEstado`

5.4.3 Orden de comprobación de una operación

1. **Sesión.** ¿Hay un usuario activo en la sesión?
2. **Rol.** ¿Su rol puede ejecutar esta acción?
3. **Visibilidad.** ¿La incidencia existe y le pertenece (dueño o técnico asignado)?
4. **Estado.** ¿El estado actual es el origen de la transición? En la reapertura, ¿está dentro del plazo?
5. **Datos.** ¿El texto o el técnico enviados son válidos?
6. **Transacción.** Cambio de estado, registros asociados y evento; todo o nada.

Si falla cualquier paso del 1 al 5 no se abre ninguna escritura. Si falla el paso 6 se revierte completo.

5.4.4 Transacciones

Cada servicio abre una transacción con `BEGIN IMMEDIATE`, actualiza el estado con una condición sobre el estado de origen (`UPDATE … WHERE id = ? AND estado = ?`), comprueba que afectó una fila, inserta los registros asociados y el evento, y confirma. Ante cualquier excepción se ejecuta `ROLLBACK`. Así estado e historial quedan siempre juntos.

El código de una incidencia nueva se calcula dentro de esa transacción, cuando el bloqueo de escritura ya está tomado, de modo que dos registros simultáneos no obtienen el mismo consecutivo; la columna `codigo` es además única.

El historial se lee ordenado por el identificador del evento, que es el orden de inserción y sirve de desempate cuando dos eventos tienen la misma marca de tiempo. La «solución vigente» de una incidencia es su última solución registrada, y el «último cierre», su cierre más reciente.

5.4.5 Seguridad

- Contraseñas con `werkzeug.security.generate_password_hash` (método `scrypt`); nunca en texto plano.
- Sesión de Flask (cookie firmada, `HttpOnly`, `SameSite=Lax`). La sesión guarda solo el identificador del usuario; el rol se lee de la base en cada petición.
- No hay selector de rol: el rol es un atributo de la cuenta.
- Jinja con autoescape; no se usa `safe` con texto del usuario. Las consultas SQL usan parámetros.
- Las operaciones que cambian datos solo aceptan `POST`.
- No hay token CSRF en los formularios; se declara como limitación conocida.

5.5 Flujo de validación de una operación

Secuencia del rechazo de una solución, con permisos y evento: secuencia-rechazo.puml.

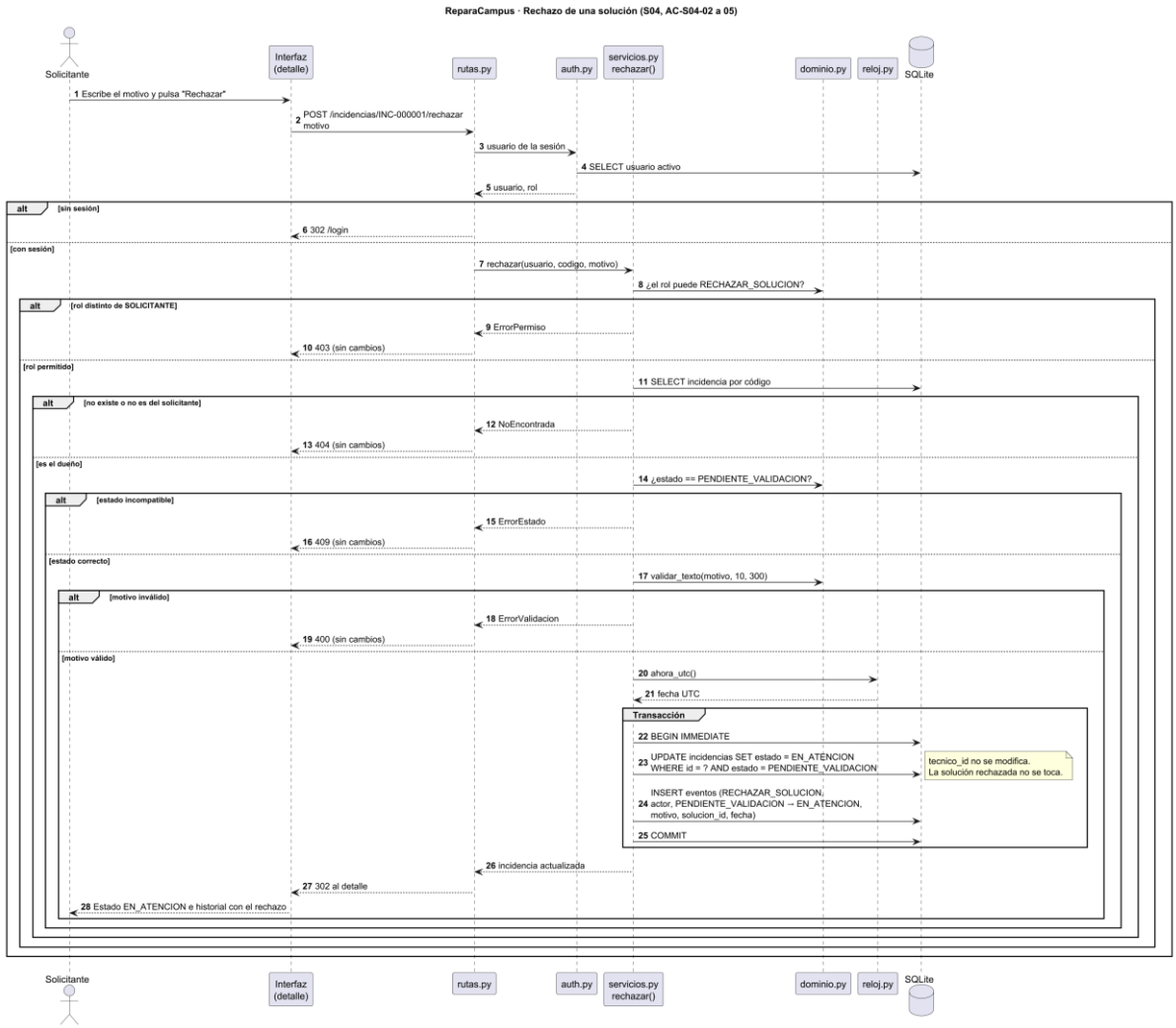


Figura 4. *Diagrama UML de secuencia del rechazo de una solución. Fuente: docs/diseno/secuencia-rechazo.puml*

5.6 Decisión de arquitectura

ADR-001: reglas de negocio en una capa de dominio y servicios, con SQL directo sobre SQLite.

5.7 Registro de revisión

Tabla 57. *Registro de revisión (2)*

Fecha	Versión	Revisor	Decisión	Observaciones
Pendiente	0.4	Rafael Eduardo May Recuero (propuesto)	Pendiente	

5.8 ADR-001 · Reglas de negocio en una capa de dominio y servicios, con SQL directo sobre SQLite

Tabla 58. *ADR-001 · Reglas de negocio en una capa de dominio y servicios, con SQL directo sobre SQLite*

Campo	Valor
Estado	Propuesta, pendiente de revisión del equipo
Fecha	2026-10-08

Campo	Valor
Autor	Luis Carlo Daza Ospino, con asistencia de IA (Claude)

5.8.1 Contexto

El caso exige que las reglas se ejecuten en el servidor, que estado e historial se guarden juntos sin efectos parciales, que una operación rechazada no deje rastro y que el plazo de 48 horas se pueda probar sin esperar tiempo real. El equipo trabaja con Flask y SQLite, tiene una semana y varias personas programan en paralelo.

5.8.2 Decisión

1. Separar el monolito en tres capas: rutas (HTTP), lógica de negocio (`dominio`, `servicios`, `consultas`) y persistencia.
2. Escribir las reglas —validaciones, prioridad, transiciones, plazo, porcentaje— como funciones puras en `dominio`, sin dependencia de Flask ni de la base.
3. Hacer de cada operación un servicio que ejecuta una única transacción con el módulo `sqlite3` de la biblioteca estándar y SQL explícito.
4. Obtener la hora de un reloj inyectable.

5.8.3 Alternativas consideradas

Tabla 59. *Alternativas consideradas*

Alternativa	A favor	En contra
Reglas dentro de las funciones de ruta	Menos archivos; rápido al inicio.	Las reglas quedan mezcladas con HTTP; cada ruta repetiría permisos y transiciones; difícil probar sin servidor.
ORM (Flask-SQLAlchemy)	Menos SQL manual; modelos declarativos.	Otra dependencia y otra capa que aprender en una semana; el control de la transacción y de los disparadores es menos visible.
Reglas en la base (disparadores para todo)	Imposible saltarlas desde el código.	Difíciles de leer, probar y relacionar con cada AC; los mensajes de error son pobres.
Capa de dominio + servicios + SQL directo (elegida)	Cada regla está en un solo lugar y se prueba sola; la transacción es explícita; sin dependencias adicionales.	Más SQL escrito a mano; hay que mantener la disciplina de no poner reglas en las rutas.

5.8.4 Consecuencias

- Las pruebas unitarias cubren `dominio` sin base de datos; las de integración usan el cliente de Flask contra un archivo SQLite real.
- La tabla de transiciones es el único lugar donde se define qué cambios de estado existen. Una transición inventada (por una persona o por la IA) no funciona si no está allí.
- La base conserva una segunda barrera: restricciones `CHECK`, unicidad de la asignación y disparadores de inmutabilidad.
- Cambiar de SQLite a otro motor exigiría revisar el SQL y los disparadores. Se acepta porque el prototipo no lo requiere.

- Un integrante puede trabajar en la interfaz, otro en un servicio y otro en las pruebas con pocos conflictos.

6. IA e implementación

6.1 Herramientas y versiones

Tabla 60. *Herramientas y versiones*

Actividad	Herramienta	Versión
Asistente de IA (especificación, código, pruebas, contraste e integración)	Claude Code (aplicación de escritorio), modelo Claude Opus 5.5	—
Asistente de IA (segunda revisión documental de las SPECS)	ChatGPT	Modelo no indicado
Servidor	Python + Flask	Python 3.14.3 · Flask 3.1.3 (Werkzeug 3.1.9, Jinja2 3.1.6)
Persistencia	SQLite, módulo `sqlite3` de la biblioteca estándar	3.50.4
Interfaz	HTML, CSS y JavaScript con plantillas Jinja	—
Pruebas	pytest y cliente de pruebas de Flask	9.1.1
Versiones y equipo	Git y GitHub	Git 2.53.0
Modelado	PlantUML	1.2026.8
SPECS	Markdown versionado en `specs/`	—
Documento final	Microsoft Word, exportado a PDF	—

6.2 Cómo se usó la IA

Se usaron dos asistentes: Claude Code para especificar, implementar, probar y contrastar, y ChatGPT para una segunda revisión documental de las SPECS. Se entregó al asistente el enunciado, y después cada SPEC con su plan, el modelo de estados y los contratos compartidos. La implementación se pidió en cambios acotados: una rama y un commit por SPEC, con las pruebas de esa SPEC ejecutadas antes de pasar a la siguiente. No se generó la aplicación con un único prompt.

6.3 Evidencia de las cinco funcionalidades

Tabla 61. *Evidencia de las cinco funcionalidades*

Funcionalidad	SPEC	Rama	Commit	Pruebas	Cómo verla en el prototipo
Registrar	S01	`feat/s01-registrar`	`7bc08f2`	26 + 5 de concurrencia	`solicitante1` → Nueva incidencia
Priorizar y asignar	S02	`feat/s02-priorizar-asignar`	`8479352`	21 + 1 de atomicidad	`coordinador1` → Incidencias → abrir una REGISTRADA → Asignar
Atender	S03	`feat/s03-atender`	`20ebd53`	19 + 2 de atomicidad	`tecnico1` → Incidencias → Iniciar atención → Registrar solución
Validar y reabrir	S04	`feat/s04-validar`	`70468c0`	43 + 3 de integración + 3 de atomicidad	`solicitante1` → abrir la incidencia → Confirmar, Rechazar o Reabrir
Consultar, historial y tablero	S05	`feat/s05-consultar-historizar`	`786cf42`	46 aprobadas	Cualquier usuario → Incidencias; `coordinador1` → Tablero

La autenticación con las cinco cuentas ficticias está en `feat/base` (commit `c5e7ef7`, 7 pruebas).

La columna de pruebas cuenta las que existen en la versión integrada. Las ramas `feat/*` se conservan con las pruebas de su momento; las agregadas después (commits `de49d68` y `6b99fec`) llegaron a `main` con la rama `docs/evidencias`.

6.4 Bitácora de IA

Seis intervenciones relevantes: especificación, revisión, código, pruebas, una segunda revisión con sus pruebas, y la integración final. No se copia la conversación completa.

> ***Pendiente de Luis:*** *los campos "Decisión humana" recogen lo que consta en la conversación. Donde dice por confirmar, la decisión todavía no ha sido expresada por una persona y no debe entregarse así. La lista completa de lo confirmado y lo pendiente está en decisiones.md. Ninguna decisión de esta bitácora es del equipo: todas son del autor o propuestas del asistente. Las revisiones de los demás integrantes siguen pendientes.*

6.4.1 Intervención 1 · Especificación

Tabla 62. *Intervención 1 · Especificación*

Campo	Contenido
Fecha / autor / herramienta / modelo	2026-10-08 · Luis Carlo Daza Ospino · Claude Code (aplicación de escritorio) · Claude Opus 5.5
Historia y SPEC / versión	H01 a H05 · S01 a S05 v0.1
Objetivo y contexto	Convertir RF01 a RF05 en cinco historias y cinco SPECS con la plantilla del examen. Se entregó el enunciado completo en PDF y los nombres del equipo.
Prompt o instrucción	«hagamos este proyecto», con la captura de la actividad; después, el PDF del enunciado y la lista de integrantes.
Respuesta relevante	Cinco SPECS con entre 8 y 13 criterios Dado/Cuando/Entonces, plan y tareas por SPEC, tres preguntas al cliente y once supuestos. El asistente se detuvo antes de programar y pidió la revisión de otro integrante, citando la regla central del examen.
Decisión humana y justificación	Luis no aprobó las SPECS él mismo: pidió un formato para que sus compañeros opinaran. Decidió además continuar con los supuestos, porque la docente no puede ser consultada antes del cierre.
Error o limitación	La tabla de pruebas de S03 afirmaba que la solución de ejemplo tenía 49 caracteres; tiene 50. Limitación: las preguntas al cliente quedaron sin respuesta.
Corrección y verificación	Se quitó la cifra antes de versionar. Los supuestos se declararon uno por uno en `specs/00-problema-y-alcance.md`.
Archivo / commit / prueba	`specs/` · commits `c96e01b`, `50fb915`, `d301e48`

6.4.2 Intervención 2 · Revisión

Tabla 63. *Intervención 2 · Revisión*

Campo	Contenido
Fecha / autor / herramienta / modelo	2026-10-08 · Luis Carlo Daza Ospino · borrador de revisión generado con un asistente de IA (**herramienta por indicar**); evaluado con Claude Code · Claude Opus 5.5
Historia y SPEC / versión	S01 a S05 v0.1 → v0.2

Campo	Contenido
Objetivo y contexto	Obtener observaciones sobre las SPECS usando el formato de revisión. El asistente que generó el borrador solo tuvo el formato y el README; no leyó las SPECS porque consultó la rama `main`.
Prompt o instrucción	Se entregó el PDF «ReparaCampus_revision_borrador_simulado» para incorporarlo y continuar.
Respuesta relevante	Trece sugerencias. Entre ellas: «aclarar que solo el coordinador cierra».
Decisión humana y justificación	*Por confirmar por Luis.* Propuesta del asistente: aceptar cuatro sugerencias, dar ocho por ya cubiertas o aclaradas y descartar una. El borrador no se registró como revisión de ningún integrante porque él mismo advierte que no lo es.
Error o limitación	**Regla inventada:** el coordinador no cierra; según RF04 cierra el solicitante dueño. Además, toda la lista de comprobación venía en «Pendiente».
Corrección y verificación	La sugerencia errónea se descartó y se contrastó con AC-S04-04. Las aceptadas produjeron la versión 0.2. La mutación M8 confirma que las pruebas impiden que otro rol cierre (31 fallos).
Archivo / commit / prueba	`docs/revision/revision-asistida-ia-2026-10-08.md` · commit `a06aaf2` · `test_p_s04_05_solo_el_duenio_valida`

6.4.3 Intervención 3 · Código

Tabla 64. *Intervención 3 · Código*

Campo	Contenido
Fecha / autor / herramienta / modelo	2026-10-08 · Luis Carlo Daza Ospino · Claude Code · Claude Opus 5.5
Historia y SPEC / versión	H01 a H05 · S01 a S05 v0.2
Objetivo y contexto	Implementar las cinco funcionalidades a partir de las SPECS, el diseño y los contratos compartidos (transiciones, errores, orden de comprobación).
Prompt o instrucción	«por el momento solo continuaremos haciendo el trabajo». El asistente trabajó en cambios acotados: una rama y un commit por SPEC, con las pruebas de esa SPEC ejecutadas antes de pasar a la siguiente.
Respuesta relevante	`feat/base` y `feat/s01` a `feat/s05`. Reglas en `dominio.py`, operaciones transaccionales en `servicios.py`, visibilidad en `consultas.py`.
Decisión humana y justificación	Luis abrió la aplicación y reportó: «hay algunas paginas que no salen bien», con capturas. Tres eran plantillas abiertas como archivo, sin servidor; la cuarta, servida por Flask, se veía bien. Aceptación del código: *por confirmar por Luis* y sujeta a la revisión de cada pull request.
Error o limitación	Un comando de edición dañó las tildes de `app/servicios.py` al implementar S02. Limitación: el código se escribió antes de la revisión de las SPECS por otro integrante.
Corrección y verificación	Se restauró el archivo desde Git y se rehízo el cambio; 45 pruebas aprobadas antes del commit. El archivo dañado nunca se versionó.
Archivo / commit / prueba	`app/` · commits `c5e7ef7`, `7bc08f2`, `8479352`, `20ebd53`, `70468c0`, `786cf42`

6.4.4 Intervención 4 · Pruebas

Tabla 65. *Intervención 4 · Pruebas*

Campo	Contenido
Fecha / autor / herramienta / modelo	2026-10-08 · Luis Carlo Daza Ospino · Claude Code · Claude Opus 5.5
Historia y SPEC / versión	S01 a S05 v0.2
Objetivo y contexto	Automatizar los escenarios del examen, incluidas tres pruebas de integración, y comprobar que la batería detecta errores reales.
Prompt o instrucción	«ahora que veine». El asistente propuso completar la evidencia de validación y comenzó por verificar las pruebas.
Respuesta relevante	153 pruebas con esperados literales tomados de las SPECS; reloj controlado para el plazo de 48 horas; ocho mutaciones del código.
Decisión humana y justificación	*Por confirmar por Luis.*
Error o limitación	Todas las pruebas pasaron en la primera ejecución, lo que por sí solo no demuestra que puedan fallar. Al llenar la matriz se vio que AC-S02-06 y AC-S03-04 no se probaban en todos los estados. Limitación: código y pruebas vienen del mismo asistente.
Corrección y verificación	(Estado a ese momento; la intervención 5 lo amplía a 176 pruebas y doce mutaciones.) Se agregaron cinco casos (de 148 a 153). Se introdujeron ocho errores, uno a la vez: todos hicieron fallar entre 2 y 31 pruebas. La instalación y las pruebas se repitieron en una copia limpia.
Archivo / commit / prueba	`tests/` · commit `de49d68` · `docs/validacion/validacion.md`, secciones 5 y 6

6.4.5 Intervención 5 · Segunda revisión y pruebas derivadas

Tabla 66. *Intervención 5 · Segunda revisión y pruebas derivadas*

Campo	Contenido
Fecha / autor / herramienta / modelo	2026-10-08 · Luis Carlo Daza Ospino · ChatGPT para la revisión documental (modelo no indicado); Claude Code · Claude Opus 5.5 para el contraste, las correcciones y las pruebas
Historia y SPEC / versión	H01 a H05 · S01 a S05 v0.2 → v0.3 → v0.4
Objetivo y contexto	Revisar las SPECS con un segundo asistente y comprobar cada observación contra el código y las pruebas antes de aceptar nada. Se entregaron al asistente las 24 observaciones y la instrucción de auditar primero, sin modificar archivos.
Prompt o instrucción	Instrucciones sucesivas del autor: «Empieza con la auditoría. No modifiques ni integres nada hasta presentarme el diagnóstico»; después, la autorización de la Etapa 1 (correcciones documentales), la de la Etapa 2 (pruebas) y la de la Etapa 3 (evidencias), cada una con sus límites.
Respuesta relevante	Diagnóstico: 11 observaciones ya cubiertas, 6 con prueba existente pero mal reflejada en la SPEC y 7 vacíos reales; ningún defecto del código. Cuatro criterios nuevos (AC-S01-10, AC-S02-10, AC-S03-09, AC-S04-11) escritos antes que sus pruebas. Después, 23 casos de prueba: atomicidad de seis transiciones, registros simultáneos, riesgo estricto y motivos solo con espacios.

Campo	Contenido
Decisión humana y justificación	**Confirmadas por Luis:** trabajar por etapas autorizadas una a una; no integrar pull requests, no crear el tag ni el PDF final; no agregar el token CSRF por ahora; no marcar revisiones humanas como hechas; no cambiar código de producción ni ajustar una SPEC a un resultado fallido. **Por confirmar por Luis:** si acepta las pruebas nuevas y si mantiene los supuestos SUP-12, SUP-13 y SUP-14.
Error o limitación	En el diagnóstico el asistente contó "27 observaciones, 17 cubiertas y 10 vacíos"; eran 24, con 11, 6 y 7. En la intervención 2 había escrito "nueve" sugerencias cubiertas donde eran ocho. Limitaciones: la prueba de concurrencia usa ocho hilos de un mismo proceso contra SQLite y no es una prueba de carga; la atomicidad se prueba con un fallo en el último paso de la transacción; con más carga, un escritor podría agotar la espera de 5 segundos del bloqueo de SQLite.
Corrección y verificación	Los recuentos se corrigieron en el registro de revisión y en esta bitácora. Verificación: `python -m pytest` dio 176 aprobadas y 0 fallidas sobre `6b99fec`; las pruebas nuevas se ejecutaron primero una por una. Cuatro mutaciones (M9 a M12) confirmaron que las pruebas nuevas fallan cuando se rompe lo que vigilan. No apareció ningún defecto y `app/` no se modificó.
Archivo / commit / prueba	`docs/revision/revision-asistida-ia-02-chatgpt.md` (`46e8995`) · SPECS v0.3 (`83861a7`) y v0.4 (`1335535`) · `tests/test_atomicidad.py`, `tests/test_concurrencia.py` (`6b99fec`) · `docs/validacion/validacion.md` (`925d22d`)

6.4.6 Intervención 6 · Integración y cierre

Tabla 67. *Intervención 6 · Integración y cierre*

Campo	Contenido
Fecha / autor / herramienta / modelo	2026-10-08 · Luis Carlo Daza Ospino · Claude Code · Claude Opus 5.5
Historia y SPEC / versión	H01 a H05 · S01 a S05 v0.5
Objetivo y contexto	Integrar las ocho ramas en una sola versión, dejar la documentación coherente, repetir la validación sobre la versión integrada y preparar la entrega.
Prompt o instrucción	«Necesito que trabajes sobre el repositorio real, corrijas los defectos encontrados, integres el proyecto, ejecutes las verificaciones finales y prepares los archivos de entrega», con la prohibición de inventar revisiones, aprobaciones o datos.
Respuesta relevante	Análisis previo: las ramas forman una cadena lineal desde `a06aaf2` y ningún archivo fue modificado por los dos lados. Ocho fusiones sin conflictos en la rama `integracion/release-examen`, en orden de dependencia. Después, correcciones documentales, validación, tag y documento de entrega.
Decisión humana y justificación	**Confirmada por Luis:** integrar y cerrar sin esperar las revisiones de sus compañeros, declarando esa ausencia. Esta decisión reemplaza su indicación anterior de no integrar. La aceptación del resultado por el equipo sigue pendiente.

Campo	Contenido
Error o limitación	La auditoría previa encontró errores del asistente en la evidencia: una decisión del autor registrada sin que él la hubiera expresado (implementar antes de la revisión), un recuento inexacto de ejecuciones de la prueba de concurrencia (55 en lugar de 65) y revisores escritos como «asignados» cuando solo estaban propuestos. Limitación: los pull requests no fueron revisados por otro integrante, y el asistente no puede editar la descripción del pull request #2 ni crear issues en GitHub.
Corrección y verificación	Los tres errores se corrigieron en `docs/decisiones.md`, `docs/validacion/validacion.md` y las SPECS v0.5. La validación final consta en `docs/validacion/ejecucion-final.md`, con el SHA probado y la salida de pytest.
Archivo / commit / prueba	Fusiones `87f4572` a `8130550` · `docs/validacion/ejecucion-final.md` · tag `release-examen`

6.5 Revisión asistida por IA de las SPECS v0.1

Tabla 68. *Revisión asistida por IA de las SPECS v0.1*

Campo	Valor
Fecha	2026-10-08
Documento recibido	`ReparaCampus_revision_borrador_simulado.pdf`, entregado por Luis Carlo Daza Ospino
Naturaleza	Propuesta **simulada** generada con un asistente de IA sobre el formato de revisión
Versión de las SPECS	0.1

6.5.1 Qué es y qué no es

El documento recibido se declara a sí mismo como “propuesta simulada” y advierte que **no constituye revisiones realizadas ni aprobaciones personales de los estudiantes**. Además, quien lo generó no leyó las SPECS: consultó la rama `main`, donde aún no estaban, y dejó toda la lista de comprobación en “Pendiente”.

Por eso:

- **No cuenta como la revisión previa que exige el examen.** La revisión de cada integrante sigue pendiente y así consta en cada SPEC.
- Los nombres que aparecen en el documento se toman solo como una **propuesta** de revisor por SPEC.
- Sus sugerencias se evaluaron una por una contra las SPECS reales, como cualquier otra salida de IA. El resultado está abajo.

6.5.2 Propuesta de revisores

Los nombres salen del borrador simulado. Ningún integrante ha confirmado esta asignación (ver `docs/decisiones.md`, B-2).

Tabla 69. *Propuesta de revisores*

SPEC	Revisor propuesto	Estado
S01	Rafael Eduardo May Recuero	Pendiente

SPEC	Revisor propuesto	Estado
S02	Jean Marco Oyola De Martino	Pendiente
S03	José Leonardo Hernández Pedrosa	Pendiente
S04	Cristian David Diaz España	Pendiente
S05	Jorge Luis González Arroyo	Pendiente
Alcance y diseño	Rafael Eduardo May Recuero	Pendiente

6.5.3 Sugerencias evaluadas

Evaluación preparada con el asistente (Claude) y pendiente de confirmación por el autor de las SPECS.

Tabla 70. *Sugerencias evaluadas*

N.º	SPEC	Sugerencia de la IA	Decisión	Justificación y comprobación
1	S01	Que cada error indique código HTTP, mensaje y ausencia de cambios.	Ya cubierta	Tabla “Errores y efectos” de S01 v0.1.
2	S01	Pruebas con descripción de 19, 20, 500 y 501 caracteres.	**Aceptada**	AC-S01-03 ya tenía los cuatro límites, pero P-S01-02 solo probaba 19 y 20. Se amplió P-S01-02.
3	S01	Definir si se normalizan espacios y mayúsculas.	Ya cubierta	SUP-03 (comparación exacta) y SUP-04 (recorte de espacios externos).
4	S02	Documentar la tabla de prioridad, el 409 de la asignación repetida, el 400 del técnico inválido y la prohibición de reasignar.	Ya cubierta	Tabla de prioridad y AC-S02-06, 07 de S02 v0.1.
5	S03	Decidir entre 404 y 403 para el técnico no asignado con el docente.	Ya cubierta	Es la pregunta Q1; queda como supuesto pendiente.
6	S03	Pruebas de 19, 20, 800 y 801 caracteres.	Ya cubierta	P-S03-05 de S03 v0.1.
7	S03	“Aclarar que solo el coordinador cierra”.	**Descartada: es un error**	Contradice RF04 y la regla “el coordinador no confirma ni reabre en nombre del solicitante”. Quien cierra es el solicitante dueño. AC-S04-04 exige 403 para el coordinador. Se conserva como ejemplo de regla inventada por la IA (riesgo de la Parte 4).
8	S04	Dejar Q3 como supuesto pendiente; especificar UTC, cierre de referencia y límite inclusivo; conservación de soluciones y cierres.	Ya cubierta	Reglas de reapertura y AC-S04-06, 07, 09 de S04 v0.1.
9	S04	Pruebas con motivo de 9, 10, 300 y 301 caracteres.	**Aceptada**	AC-S04-03 tenía los cuatro límites, pero P-S04-04 solo probaba 9 y 10. Se amplió P-S04-04.
10	S05	Distinguir filtro inválido de búsqueda sin resultados.	**Aceptada**	No estaba dicho. Se agregó a AC-S05-03: valor válido sin coincidencias responde 200 con lista vacía.
11	S05	Precisar qué roles pueden consultar el historial.	**Aceptada**	Estaba implícito en la visibilidad. Se agregó una frase explícita en “Reglas y proceso”.
12	S05	Definir numerador, denominador, redondeo y caso vacío del porcentaje.	Ya cubierta	Regla del tablero y AC-S05-07, 08 de S05 v0.1.

N.º	SPEC	Sugerencia de la IA	Decisión	Justificación y comprobación
13	General	Subir los archivos de `specs/` y `docs/diseno/` a la rama pública.	Aclarada	Están en la rama `specs/s01-s05`; llegarán a `main` cuando se integre su pull request.

6.5.4 Resultado

Las SPECS pasan a la versión 0.2 con los cambios 2, 9, 10 y 11. El campo “Decisión de revisión” de cada SPEC sigue en **Pendiente** hasta que el integrante propuesto la lea y registre su decisión.

6.6 Segunda revisión asistida por IA de las SPECS (v0.2)

Tabla 71. *Segunda revisión asistida por IA de las SPECS (v0.2)*

Campo	Valor
Fecha	2026-10-08
Quién la pidió	Luis Carlo Daza Ospino, autor de las SPECS
Herramienta de la revisión	ChatGPT, en una conversación de Luis (según él informó)
Qué se revisó	SPECS S01 a S05 v0.2, de forma documental
Contraste	Claude Code (Claude Opus 5.5) contrastó cada observación con las SPECS, el código y las pruebas, sin modificar nada, y ejecutó `python -m pytest`: 153 aprobadas
Versión resultante	SPECS, planes, tareas y diseño v0.3

6.6.1 Qué es y qué no es

Es una revisión hecha con asistentes de IA a petición del autor. **No es la revisión de otro integrante** que exige el examen: esa sigue pendiente en cada SPEC y en el pull request #2, que al momento de este registro no tenía ninguna revisión ni comentario. El 2026-10-08 el autor integró esa rama en `main` sin que llegara ninguna.

Las 24 observaciones llegaron como hipótesis de una lectura documental. Ninguna resultó ser un defecto del código; siete señalan vacíos reales en las SPECS o en las pruebas.

6.6.2 Decisión humana registrada

Luis autorizó trabajar por etapas: primero las correcciones documentales (este cambio), después las pruebas que faltan y por último las evidencias. Indicó no integrar pull requests, no crear el tag, no generar el PDF final, no marcar revisiones humanas como hechas y dejar el token CSRF como mejora opcional.

6.6.3 Observaciones y resultado del contraste

Tabla 72. *Observaciones y resultado del contraste*

N.º	SPEC	Observación	Resultado	Evidencia	Acción
1	S01	Precisar cómo se interpretan los booleanos del formulario	Cubierta, mejorable	El dominio solo acepta `true` y `false`; lo demás se rechaza	SPEC v0.3 lo dice de forma explícita; prueba P-S01-12 por automatizar
2	S01	Las pruebas con `INC-000001` deben usar una base aislada	Cubierta	Cada prueba crea su propio archivo SQLite temporal	Se escribió la precondición en la tabla de pruebas
3	S01	Generación de códigos segura ante solicitudes concurrentes	**Vacío real**	El diseño usa `BEGIN IMMEDIATE` y unicidad, pero no hay prueba	AC-S01-10, SUP-14 y P-S01-11 por automatizar
4	S01	Especificar mejor la prueba de campos generados por el servidor	Prueba existe; tabla imprecisa	`test_p_s01_06_el_servidor_ignora_campos_generados` envía cinco campos; la tabla nombraba dos	P-S01-06 corregida
5	S01	Coherencia de la prioridad con S02	Cubierta	`test_p_s01_01_registro_valido` y `test_p_s02_01_prioridad_calculada`	Ninguna
6	S02	Pruebas de asignación en estados incompatibles y reasignación	Cubierta	`test_p_s02_03_asignacion_repetida`, `test_no_se_asigna_fuera_de_registrada` (commit `de49d68`)	P-S02-06 agregada a la tabla
7	S02	Prueba explícita de que el cliente no manipula la prioridad	Prueba existe; sin identificador	`test_la_prioridad_no_se_cambia_al_asignar`	P-S02-08 agregada
8	S02	Comportamiento de 404 y 409	Cubierta	`test_incidencia_inexistente`, `test_p_s02_03_asignacion_repetida`	Se aclaró que el rol se comprueba antes que la existencia
9	S02	Unicidad de la asignación también en la base	Cubierta	`UNIQUE (incidencia_id)` y disparadores; `test_la_base_impide_una_segunda_asignacion`	P-S02-09 agregada
10	S03	Precondiciones de las pruebas que esperan cantidades de eventos	**Vacío real** (documental)	La tabla daba el conteo final sin el estado de partida	Tabla de pruebas con precondición
11	S03	Una solución rechazada no elimina las anteriores	Cubierta	`test_p_s04_02_rechazo_y_nueva_solucion`; disparadores sobre `soluciones`	P-S03-10 la referencia

N.º	SPEC	Observación	Resultado	Evidencia	Acción
12	S03	Coherencia con el rechazo y la reapertura de S04	Cubierta	Pruebas de integración	Ninguna
13	S03	El técnico no cierra ni se salta la validación	Cubierta	`test_p_s04_05_solo_el_duenio_valida`; mutación M8: 31 pruebas fallan	P-S03-11 la referencia
14	S04	Pruebas de motivo vacío en rechazo y reapertura	Cubierta en parte	Vacío sí se prueba en ambos; solo espacios, no	**Vacío real**: P-S04-15 por automatizar
15	S04	Definir la solución vigente	**Vacío real**	El código toma la última; la SPEC no lo decía	Definición en S04 y SUP-12
16	S04	Transiciones inválidas de confirmar, rechazar y reabrir	Cubierta	`test_validar_en_estado_incompatible`, `test_reabrir_en_estado_incompatible`	P-S04-09 y 10 agregadas
17	S04	Atomicidad de estado, cierres y eventos	**Vacío real**	Solo el registro tiene una prueba que fuerza un fallo	AC-S02-10, AC-S03-09 y AC-S04-11; pruebas por automatizar
18	S05	P-S05-04 debe cubrir `eventos`, `soluciones` y `cierres`	Prueba existe; tabla imprecisa	`test_p_s05_04_el_historial_es_inmutable` ya cubre las tres	P-S05-04 corregida
19	S05	Prueba explícita de AC-S05-06	Prueba existe; sin identificador	`test_las_operaciones_rechazadas_no_dejan_rastro`	P-S05-08 agregada
20	S05	Prueba verificable de AC-S05-09	Prueba existe; sin identificador	`test_tablero_cantidades_y_criticas`	P-S05-09 agregada
21	S05	Precisar el conteo de eventos de AC-S05-04	Cubierta	El criterio ya dice ocho eventos y los enumera	P-S05-07 agregada
22	S05	Criterio de desempate para eventos con la misma fecha	**Vacío real**	El código ordena por identificador; la SPEC no lo decía	Regla en S05 y SUP-13
23	S05	Caso de redondeo del porcentaje	Prueba existe; sin identificador	`test_formato_y_redondeo_del_porcentaje`	P-S05-10 agregada
24	S05	Visibilidad del historial para el técnico asignado	**Vacío real** (documental)	La prueba lo cubría; el criterio no lo nombraba	AC-S05-04 corregido

Resumen: 11 cubiertas, 6 con prueba existente pero mal reflejada en la SPEC y 7 vacíos reales. En el diagnóstico inicial el asistente escribió “27 observaciones, 17 cubiertas y 10 vacíos”; el recuento correcto es este.

6.6.4 Seguimiento

- Las seis pruebas que quedaron «Por automatizar» (P-S01-11, P-S01-12, P-S02-10, P-S03-12, P-S04-15 y P-S04-16) se implementaron en el commit `6b99fec` de la rama `docs/evidencias`. La batería completa dio 176 pruebas aprobadas y no reveló ningún defecto; el código de producción no se modificó. Las SPECS las registran con su nombre real desde la v0.4.

6.6.5 Pendiente

- Revisión de cada SPEC por el integrante asignado.

7. Validación

7.1 Ejecución final sobre el tag de entrega

Este registro se agregó a `main` después de crear el tag, porque un commit no puede contener su propio identificador. El commit etiquetado es el que se probó; los commits posteriores solo agregan este registro, la salida de pytest y el documento de entrega, sin tocar `app/` ni `tests/`.

7.1.1 Versión probada

Tabla 73. *Versión probada*

Dato	Valor
Repositorio	https://github.com/LUCHOMAN0529/reparacampus
Tag	`release-examen`
SHA exacto	`b59faa1128708c5cc405d0b315c0226c610c5489`
Rama	`main`, sin cambios sin confirmar al ejecutar
Fecha y hora	2026-10-08, de 02:57:23 a 02:58:06 (UTC−05:00)
Entorno	Microsoft Windows 11 Home Single Language 10.0.26300 · Python 3.14.3 · Flask 3.1.3 · Werkzeug 3.1.9 · Jinja2 3.1.6 · SQLite 3.50.4 · pytest 9.1.1
Quién ejecutó	El asistente de IA (Claude Code), por instrucción del autor

7.1.2 Resultados

Tabla 74. *Resultados*

N.º	Comando	Aprobadas	Fallidas	Salida completa
1	`python -m pytest`	176	0	ejecucion/pytest.txt
2	`python -m pytest -m integracion -v`	3 (173 no seleccionadas)	0	ejecucion/pytest-integracion.txt
3	`python -m pytest -v`	176	0	ejecucion/pytest-v.txt

El comando 2 se ejecutó con `-v` para que la salida nombre las tres pruebas. A los tres comandos se les añadió `-p no:cacheprovider`, que solo evita crear la carpeta de caché de pytest.

7.1.3 Pruebas de integración contra SQLite real

Tabla 75. *Pruebas de integración contra SQLite real*

Prueba	Recorrido	Resultado
`test_p_s04_01_flujo_de_cierre`	Registrar → asignar → iniciar → proponer → confirmar	Aprobada
`test_p_s04_02_rechazo_y_nueva_solucion`	… → proponer → rechazar → nueva solución → confirmar	Aprobada
`test_p_s04_03_reapertura_y_nuevo_cierre`	Cierre → reapertura a las 48 h exactas → nueva solución → nuevo cierre → reapertura rechazada a 48 h + 1 µs	Aprobada

7.1.4 Qué cubre esta ejecución

Tabla 76. *Qué cubre esta ejecución*

Comprobación pedida	Pruebas que la cubren (todas aprobadas)
Límite exacto de 48 horas y caso fuera de plazo	`test_reabrir_exactamente_a_las_48_horas`, `test_reabrir_un_instante_despues_de_48_horas`, `test_p_s04_03_reapertura_y_nuevo_cierre`
Permisos por rol y pertenencia	`test_p_s01_04_otros_roles_no_registran`, `test_p_s02_04_otros_roles_no_asignan`, `test_p_s03_03_tecnico_no_asignado`, `test_p_s04_05_solo_el_duenio_valida`, `test_p_s05_01_detalle_segun_permisos`
Persistencia tras reinicio	`test_p_s05_05_los_datos_permanecen_al_reiniciar`
Efectos de operaciones rechazadas	`test_las_operaciones_rechazadas_no_dejan_rastro`, `test_un_fallo_al_guardar_el_evento_revierte_toda_la_operacion`
Métricas del tablero	`test_p_s05_03_tablero_vacio`, `test_p_s05_03_porcentaje_de_cierre_y_reapertura`, `test_tablero_cantidades_y_criticas`, `test_formato_y_redondeo_del_porcentaje`
Texto con HTML	`test_p_s01_05_el_texto_no_se_ejecuta_como_html`, `test_la_solucion_no_se_ejecuta_como_html`, `test_el_listado_no_ejecuta_html`
Contraseñas con hash y acceso con cuentas	`tests/test_auth.py` (7 pruebas)

7.1.5 Otras comprobaciones sobre la versión integrada

Hechas el mismo día sobre el commit `256605a`, anterior al tag. Entre ese commit y el tag solo cambian documentos; `app/` y `tests/` son idénticos.

Tabla 77. *Otras comprobaciones sobre la versión integrada*

Comprobación	Resultado
Copia limpia (`git clone`, entorno virtual nuevo, `pip install -r requirements.txt`, `python -m pytest`)	176 aprobadas, 0 fallidas
Recorrido de extremo a extremo por HTTP contra el servidor real, con recorrido_http.py	32 de 32 comprobaciones correctas

7.1.6 Lo que esta ejecución no demuestra

- No hay revisión de otro integrante ni recorrido en navegador hecho por una persona.
- La prueba de concurrencia usa ocho hilos en un mismo proceso contra SQLite; no es una prueba de carga.
- Se ejecutó en un solo equipo y sistema operativo.

Tabla 78. *Lo que esta ejecución no demuestra*

Campo	Valor
Versión del código	Versión integrada en `main`. `app/` no cambia desde `786cf42` y `tests/` no cambia desde `6b99fec`
SPECS	S01 a S05, versión 0.5, integradas en `main`
Fecha de ejecución	2026-10-08
Comando	`python -m pytest`
Resultado	**176 pruebas aprobadas, 0 fallidas** sobre `6b99fec` (20,0 s) y de nuevo sobre la versión integrada `256605a`, en el repositorio de trabajo y en una copia limpia
Entorno	Windows 11, Python 3.14.3, Flask 3.1.3, SQLite 3.50.4, pytest 9.1.1
Datos	Cada prueba crea un archivo SQLite temporal con las cinco cuentas del seed y un reloj controlado que inicia en `2026-10-01T08:00:00Z`

La ejecución sobre el commit exacto del tag `release-examen`, con la salida de pytest, está en ejecucion-final.md.

7.2 Matriz de trazabilidad

Tabla 79. *Matriz de trazabilidad*

Requisito	Historia	SPEC	Criterios	Tareas	Código principal	Commit	Archivo de pruebas	Pruebas
RF01 Registrar	H01	S01 v0.2	AC-S01-01 a 10	T-S01-01 a 07	`dominio.validar_registro`, `servicios.registrar_incidencia`, `rutas.nueva`	`7bc08f2`	`tests/test_s01_registro.py`, `tests/test_concurrencia.py`	26 + 5
RF02 Priorizar y asignar	H02	S02 v0.2	AC-S02-01 a 10	T-S02-01 a 06	`dominio.calcular_prioridad`, `servicios.asignar`	`8479352`	`tests/test_s02_prioridad_asignacion.py`, `tests/test_atomicidad.py`	21 + 1
RF03 Atender	H03	S03 v0.2	AC-S03-01 a 09	T-S03-01 a 06	`servicios.iniciar_atencion`, `servicios.registrar_solucion`	`20ebd53`	`tests/test_s03_atencion.py`, `tests/test_atomicidad.py`	19 + 2
RF04 Validar	H04	S04 v0.2	AC-S04-01 a 11	T-S04-01 a 07	`servicios.confirmar`, `rechazar`, `reabrir`; `dominio.dentro_de_plazo_reapertura`	`70468c0`	`tests/test_s04_validacion.py`, `tests/test_integracion.py`, `tests/test_atomicidad.py`	43 + 3 + 3
RF05 Consultar e historizar	H05	S05 v0.2	AC-S05-01 a 13	T-S05-01 a 06	`consultas.listar`, `historial`, `tablero`; disparadores de `schema.sql`	`786cf42`	`tests/test_s05_consulta_tablero.py`	46
Autenticación (límites del prototipo)	—	Alcance	AC-S05-13	T-S01-01, 02	`auth.py`, `db.py`	`c5e7ef7`	`tests/test_auth.py`	7

Esquema, transiciones y errores compartidos: commit `c5e7ef7`. Estados faltantes de AC-S02-06 y AC-S03-04: commit `de49d68`. Atomicidad, registros simultáneos, riesgo estricto y motivos solo con espacios: commit `6b99fec`.

7.3 Diez escenarios automatizados de referencia

Los esperados se escribieron en las SPECS antes de ejecutar y aparecen como literales en las pruebas; ninguno se calcula con la función de producción. Todos se ejecutaron con `python -m pytest` el 2026-10-08 sobre `6b99fec`.

Tabla 80. *Diez escenarios automatizados de referencia*

ID	SPEC / AC	Tipo	Entrada	Esperado	Observado
P-S01-01	S01 / AC-S01-01	Funcional	`solicitante1` registra `LAB-01`, `ELECTRICIDAD`, `BAJO`, riesgo `false`	302; `INC-000001`, `REGISTRADA`, `NORMAL`; 1 evento `CREAR`	Igual al esperado
P-S01-02	S01 / AC-S01-03	Límites	Descripción de 19, 20, 500 y 501 caracteres	400, 302, 302, 400; 2 incidencias	Igual al esperado
P-S02-01	S02 / AC-S02-01 a 03	Regla	(riesgo, impacto): (sí, BAJO), (sí, ALTO), (no, ALTO), (no, BAJO)	`CRITICA`, `CRITICA`, `ALTA`, `NORMAL`	Igual al esperado
P-S02-03	S02 / AC-S02-06	Estado	Segunda asignación a `tecnico2`	409; técnico `tecnico1`; 1 asignación; 2 eventos	Igual al esperado
P-S03-03	S03 / AC-S03-03	Permisos	`tecnico2` inicia o soluciona la incidencia de `tecnico1`	404; sin cambios	Igual al esperado
P-S03-04	S03 / AC-S03-04	Estado	Solución con la incidencia en `ASIGNADA`	409; 0 soluciones; 2 eventos	Igual al esperado
P-S04-01	S04 / AC-S04-01	**Integración**	Registrar → asignar → iniciar → proponer → confirmar	`CERRADA`; 1 solución; 1 cierre; 5 eventos en orden	Igual al esperado
P-S04-02	S04 / AC-S04-02, AC-S03-06	**Integración**	… → proponer → rechazar “El daño sigue presente” → nueva solución → confirmar	Tras el rechazo: `EN_ATENCION`, `tecnico1`, 1 solución. Final: `CERRADA`, 2 soluciones, 1 cierre, 7 eventos	Igual al esperado
P-S04-03	S04 / AC-S04-06, 07, 09	**Integración**	Cierre en `2026-10-01T10:00Z` → reapertura a las 48 h exactas → nueva solución → nuevo cierre → reapertura a 48 h + 1 µs del segundo cierre	302; `CERRADA` con 2 soluciones y 2 cierres, 8 eventos; después 409 sin cambios	Igual al esperado
P-S05-03	S05 / AC-S05-07, 08	Tablero	Sin incidencias; una cerrada de cuatro; tras reabrirla	`0,0 %`; `25,0 %`; `0,0 %` con 1 cierre conservado	Igual al esperado

Las tres de integración se ejecutan solas con `python -m pytest -m integracion`: recorren el flujo por HTTP, cada paso con la sesión del actor que corresponde, contra un archivo SQLite real, y comprueban los efectos con una conexión independiente.

7.4 Matriz SPEC frente a código

Estado de cada criterio de aceptación de las SPECS vigentes sobre `6b99fec`. “Observado” resume el resultado de la prueba y los efectos en la base.

7.4.1 S01 · Registrar (RF01, H01) — app/dominio.py, app/servicios.py, app/rutas.py

Tabla 81. *S01 · Registrar (RF01, H01) — app/dominio.py, app/servicios.py, app/rutas.py*

Criterio	Comportamiento esperado	Prueba	Observado y efectos en la base	Estado
AC-S01-01	Registro válido crea incidencia `REGISTRADA` con código, autor y fecha, y un evento `CREAR`	`test_p_s01_01_registro_valido`	302; 1 fila en `incidencias`, 1 en `eventos`; fecha del reloj del servidor	Cumple
AC-S01-02	Otros roles 403; sin sesión, redirección; nada se crea	`test_p_s01_04_otros_roles_no_registran`, `test_sin_sesion_no_registra`	403 y 302; 0 filas	Cumple
AC-S01-03	Descripción de 20 a 500 tras retirar espacios	`test_p_s01_02_limites_de_la_descripcion`, `test_los_espacios_externos_no_cuentan`	19 y 501: 400; 20 y 500: 302; se guarda recortada	Cumple
AC-S01-04	Impacto inválido se rechaza	`test_p_s01_03_valores_invalidos` (`MEDIO`, `alto`, vacío)	400; 0 filas	Cumple
AC-S01-05	Riesgo inválido o ausente se rechaza; solo valen `true` y `false`	`test_p_s01_03_valores_invalidos` (`quiza`, ausente), `test_p_s01_12_el_riesgo_solo_acepta_true_o_false` (`True`, `1`, `on`, `TRUE`, `si`, `true`)	400; 0 filas	Cumple
AC-S01-06	Ubicación o categoría fuera de catálogo se rechaza	`test_p_s01_03_valores_invalidos`	400; 0 filas	Cumple
AC-S01-07	El texto no se ejecuta como HTML	`test_p_s01_05_el_texto_no_se_ejecuta_como_html`, `test_el_listado_no_ejecuta_html`	Guardado literal; la página lo muestra escapado	Cumple
AC-S01-08	El servidor ignora código, estado, prioridad, autor y fecha enviados	`test_p_s01_06_el_servidor_ignora_campos_generados`	`REGISTRADA`, `NORMAL`, autor de la sesión	Cumple
AC-S01-09	Si falla el evento no queda la incidencia	`test_sin_efectos_parciales_si_falla_el_evento`	0 incidencias, 0 eventos	Cumple
AC-S01-10	Registros simultáneos: todos se crean con códigos distintos y consecutivos	`test_p_s01_11_registros_simultaneos` (5 repeticiones de 8 hilos)	8 respuestas 302; `INC-000001` a `INC-000008` sin repetir; cada incidencia con su autor, su descripción y un evento `CREAR`	Cumple

7.4.2 S02 · Priorizar y asignar (RF02, H02) — app/dominio.py, app/servicios.py

Tabla 82. *S02 · Priorizar y asignar (RF02, H02) — app/dominio.py, app/servicios.py*

Criterio	Comportamiento esperado	Prueba	Observado y efectos en la base	Estado
AC-S02-01 a 03	Tabla de prioridad	`test_p_s02_01_prioridad_calculada`	`CRITICA`, `CRITICA`, `ALTA`, `NORMAL`	Cumple
AC-S02-04	El coordinador asigna una `REGISTRADA` a un técnico activo	`test_p_s02_02_asignacion_correcta`	`ASIGNADA`; 1 fila en `asignaciones`; evento `ASIGNAR`	Cumple
AC-S02-05	Solicitante y técnico no asignan	`test_p_s02_04_otros_roles_no_asignan`	403; sigue `REGISTRADA` sin técnico	Cumple
AC-S02-06	Asignación repetida o en otro estado: 409 sin cambios	`test_p_s02_03_asignacion_repetida`, `test_no_se_asigna_fuera_de_registrada`, `test_la_base_impide_una_segunda_asignacion`	409 en los cuatro estados; conserva `tecnico1`; la base rechaza un segundo registro	Cumple (tras el hallazgo H-03)
AC-S02-07	Técnico ausente, inexistente, inactivo o no técnico: 400	`test_p_s02_05_tecnico_invalido`	400; 0 asignaciones	Cumple
AC-S02-08	Incidencia inexistente: 404	`test_incidencia_inexistente`	404	Cumple
AC-S02-09	La prioridad no se manipula	`test_la_prioridad_no_se_cambia_al_asignar`, `test_p_s01_06_el_servidor_ignora_campos_generados`	Prioridad calculada	Cumple
AC-S02-10	Si falla el evento al asignar, no queda nada	`test_un_fallo_al_guardar_el_evento_revierte_toda_la_operacion[P-S02-10-asignar]`	Las cinco tablas idénticas fila a fila; después la operación se completa	Cumple

7.4.3 S03 · Atender (RF03, H03) — app/servicios.py

Tabla 83. *S03 · Atender (RF03, H03) — app/servicios.py*

Criterio	Comportamiento esperado	Prueba	Observado y efectos en la base	Estado
AC-S03-01	El técnico asignado inicia la atención	`test_p_s03_01_iniciar_atencion`	`EN_ATENCION`; evento con actor y fecha	Cumple
AC-S03-02	Registra solución y pasa a validación	`test_p_s03_02_registrar_solucion`	`PENDIENTE_VALIDACION`; 1 solución enlazada al evento	Cumple
AC-S03-03	Técnico no asignado 404; otros roles 403	`test_p_s03_03_tecnico_no_asignado`, `test_otros_roles_no_atienden`	Sin cambios de estado ni eventos	Cumple

Criterio	Comportamiento esperado	Prueba	Observado y efectos en la base	Estado
AC-S03-04	Estados incompatibles: 409	`test_p_s03_04_solucion_antes_de_iniciar`, `test_iniciar_en_estado_incompatible`, `test_segunda_solucion_sin_rechazo_se_rechaza`	409; sin soluciones nuevas	Cumple (tras el hallazgo H-03)
AC-S03-05	Solución de 20 a 800	`test_p_s03_05_limites_de_la_solucion`	19, 801 y vacía: 400; 20 y 800: 302	Cumple
AC-S03-06	Una nueva solución conserva las anteriores	`test_p_s04_02_rechazo_y_nueva_solucion`, `test_p_s04_03_reapertura_y_nuevo_cierre`	2 filas en `soluciones`, la primera intacta	Cumple
AC-S03-07	El técnico no cierra	`test_p_s04_05_solo_el_duenio_valida` (`tecnico1`, `tecnico2`)	403; 0 cierres	Cumple; probado en `PENDIENTE_VALIDACION`, que es el único estado desde el que existe el cierre
AC-S03-08	La solución no se ejecuta como HTML	`test_la_solucion_no_se_ejecuta_como_html`	Escapada en el detalle	Cumple
AC-S03-09	Si falla el evento al iniciar o al registrar la solución, no queda nada	`test_un_fallo_al_guardar_el_evento_revierte_toda_la_operacion[P-S03-12-iniciar]`, `[P-S03-12-solucion]`	Las cinco tablas idénticas fila a fila; 0 soluciones nuevas	Cumple

7.4.4 S04 · Validar (RF04, H04) — app/servicios.py, app/dominio.py, app/reloj.py

Tabla 84. *S04 · Validar (RF04, H04) — app/servicios.py, app/dominio.py, app/reloj.py*

Criterio	Comportamiento esperado	Prueba	Observado y efectos en la base	Estado
AC-S04-01	El dueño confirma y queda `CERRADA`	`test_confirmar`, `test_p_s04_01_flujo_de_cierre`	1 fila en `cierres` con solución, dueño y fecha	Cumple
AC-S04-02	Rechazo conserva técnico y solución y registra quién, cuándo y por qué	`test_rechazar_conserva_tecnico_y_solucion`, `test_p_s04_02_rechazo_y_nueva_solucion`	`EN_ATENCION`, `tecnico1`, solución intacta, evento con motivo	Cumple
AC-S04-03	Motivo de rechazo de 10 a 300; solo espacios se rechaza	`test_p_s04_04_limites_del_motivo_de_rechazo`, `test_p_s04_15_rechazo_con_motivo_solo_de_espacios`	9, 301, vacío y solo espacios (15, 300 y tabulaciones): 400 sin eventos nuevos; 10 y 300: 302	Cumple
AC-S04-04	Otro solicitante 404; coordinador y técnico 403	`test_p_s04_05_solo_el_duenio_valida`, `test_solo_el_duenio_reabre`	Sin cambios; 0 cierres nuevos	Cumple
AC-S04-05	Estados incompatibles: 409	`test_validar_en_estado_incompatible`, `test_reabrir_en_estado_incompatible`	409 en los ocho y cuatro casos	Cumple

Criterio	Comportamiento esperado	Prueba	Observado y efectos en la base	Estado
AC-S04-06	Reapertura a las 48 h exactas	`test_reabrir_exactamente_a_las_48_horas`	302; `EN_ATENCION`; cierre y solución conservados	Cumple
AC-S04-07	Reapertura a 48 h + 1 µs se rechaza	`test_reabrir_un_instante_despues_de_48_horas`	409; mismo historial de 5 eventos	Cumple
AC-S04-08	Motivo de reapertura inválido; solo espacios se rechaza	`test_motivo_de_reapertura_invalido`, `test_p_s04_15_reapertura_con_motivo_solo_de_espacios`	400; sigue `CERRADA`; los mismos 5 eventos	Cumple
AC-S04-09	Cierres sucesivos; el plazo cuenta desde el último	`test_p_s04_03_reapertura_y_nuevo_cierre`	2 cierres; 409 a 48 h + 1 µs del segundo; 302 a las 48 h	Cumple
AC-S04-10	La hora la decide el servidor	`test_la_hora_la_decide_el_servidor`	409 aunque el formulario envíe una fecha dentro del plazo	Cumple
AC-S04-11	Si falla el evento al confirmar, rechazar o reabrir, no queda nada	`test_un_fallo_al_guardar_el_evento_revierte_toda_la_operacion[P-S04-16-confirmar]`, `[P-S04-16-rechazar]`, `[P-S04-16-reabrir]`	Las cinco tablas idénticas fila a fila; 0 cierres nuevos	Cumple

7.4.5 S05 · Consultar, historizar y tablero (RF05, H05) — app/consultas.py, app/schema.sql

Tabla 85. *S05 · Consultar, historizar y tablero (RF05, H05) — app/consultas.py, app/schema.sql*

Criterio	Comportamiento esperado	Prueba	Observado y efectos en la base	Estado
AC-S05-01	Listado según rol	`test_p_s05_01_listado_segun_permisos`	2, 1, 1, 0 y 3 incidencias	Cumple
AC-S05-02	Filtros por estado y prioridad, sin ampliar la visibilidad	`test_p_s05_02_filtros`, `test_un_filtro_no_amplia_la_visibilidad`	Solo coincidencias visibles	Cumple
AC-S05-03	Filtro inválido 400; válido sin resultados 200	`test_p_s05_02_filtro_invalido`, `test_p_s05_02_filtros`	400; lista vacía	Cumple
AC-S05-04	Historial completo y ordenado	`test_historial_completo`	8 eventos con actor, fecha, estados y acción; soluciones y motivos visibles en orden	Cumple
AC-S05-05	Historial inmutable	`test_p_s05_04_el_historial_es_inmutable`, `test_no_hay_rutas_para_editar_o_borrar`	`UPDATE` y `DELETE` abortan en las tres tablas; mismo contenido	Cumple
AC-S05-06	Una operación rechazada no deja rastro	`test_las_operaciones_rechazadas_no_dejan_rastro`	7 rechazos (403, 404, 409, 400); eventos idénticos fila a fila	Cumple

Criterio	Comportamiento esperado	Prueba	Observado y efectos en la base	Estado
AC-S05-07	Tablero vacío: ceros y `0,0 %`	`test_p_s05_03_tablero_vacio`	Igual	Cumple
AC-S05-08	1 de 4 = `25,0 %`; reabrir baja y conserva el cierre	`test_p_s05_03_porcentaje_de_cierre_y_reapertura`, `test_formato_y_redondeo_del_porcentaje`	`25,0 %` → `0,0 %`; 1 cierre	Cumple
AC-S05-09	Cantidades y críticas no cerradas	`test_tablero_cantidades_y_criticas`	Estados 2, 1, 0, 0, 1; prioridades 2, 1, 1; solo `INC-000002`	Cumple
AC-S05-10	Tablero solo para el coordinador	`test_p_s05_06_tablero_solo_para_el_coordinador`	403	Cumple
AC-S05-11	Detalle ajeno: 404	`test_p_s05_01_detalle_segun_permisos`	404 en los cuatro casos ajenos	Cumple
AC-S05-12	Los datos permanecen al reiniciar	`test_p_s05_05_los_datos_permanecen_al_reiniciar`	Segunda instancia sobre el mismo archivo muestra los mismos datos	Cumple; el reinicio se simula creando otra instancia de la aplicación
AC-S05-13	Sin sesión, redirección; credenciales incorrectas, sin acceso	`test_p_s05_06_sin_sesion_redirige_al_login`, `tests/test_auth.py`	302 y 401	Cumple

Resumen: 53 criterios (los 49 de la v0.2 más AC-S01-10, AC-S02-10, AC-S03-09 y AC-S04-11), 53 en “Cumple”. Ninguno en “Parcial” o “No cumple” sobre `6b99fec`.

7.5 Comprobación manual

Tabla 86. *Comprobación manual*

Fecha	Qué se hizo	Resultado
2026-10-08	Con el servidor en `localhost:5057`, `solicitante1` inició sesión y registró una incidencia con impacto `ALTO`, riesgo Sí y la descripción `Prueba de humo: <b>chispas</b>` …	Se creó `INC-000001` con prioridad `CRITICA`; la descripción se mostró como texto, sin negrita; el contador de caracteres funcionó; sin errores en la consola
2026-10-08	Copia limpia: `git clone`, entorno virtual, `pip install -r requirements.txt`, `python -m pytest`, `flask --app app init-db`	Las pruebas pasaron y la base se creó, siguiendo solo el README
2026-10-08	Recorrido automatizado de extremo a extremo por HTTP contra el **servidor real** de Flask (no el cliente de pruebas), en una copia limpia de la versión integrada `256605a`: recorrido_http.py. Los cinco usuarios inician sesión y recorren registro, prioridad, asignación, atención, rechazo, nueva solución, cierre, reapertura, filtros, tablero y permisos; al final el servidor se detiene y se vuelve a arrancar	**32 de 32 comprobaciones correctas**, incluidas: `50,0 %` con una cerrada de dos y `0,0 %` al reabrir; 403 y 404 según rol y pertenencia; texto con HTML escapado; historial de ocho eventos; los datos permanecen tras reiniciar el servidor

Pendiente: el recorrido completo **en navegador hecho por una persona**. El recorrido por HTTP anterior lo ejecutó un script, no un integrante; comprueba el servidor real, pero no la apariencia de las páginas ni el uso con ratón y teclado.

7.6 Hallazgos y regresión

Tabla 87. *Hallazgos y regresión*

ID	Diferencia detectada	Cómo se detectó	Corrección	Regresión
H-01	La SPEC S03 v0.1 decía que la solución de ejemplo tenía 49 caracteres; tiene 50	Recuento al revisar la tabla de pruebas, antes de versionar	Se quitó la cifra de P-S03-02	Commit `50fb915`; no afectaba código
H-02	P-S01-02 y P-S04-04 no probaban los límites superiores (500/501 y 300/301) que sus criterios sí exigían	Revisión asistida por IA de las SPECS v0.1	SPECS v0.2 y casos agregados	Commit `a06aaf2`; pruebas en `7bc08f2` y `70468c0`
H-03	AC-S02-06 exige 409 al asignar en cualquier estado distinto de `REGISTRADA`, pero solo se probaba en `ASIGNADA`; AC-S03-04 no se probaba en `CERRADA`	Al llenar esta matriz criterio por criterio	Cinco casos nuevos	Commit `de49d68`; 153 pruebas aprobadas. El código ya se comportaba bien: faltaba la prueba
H-05	La atomicidad solo se probaba en el registro; no había prueba de registros simultáneos, de riesgo `True`/`1`/`on` ni de motivos formados solo por espacios	Segunda revisión asistida por IA, contrastada con el código	Cuatro criterios nuevos en las SPECS v0.3 y 23 casos de prueba	Commit `6b99fec`; 176 pruebas aprobadas. **No se encontró ningún defecto**: el código ya cumplía y no se modificó `app/`
H-04	Un comando de edición dañó las tildes de `app/servicios.py` (codificación) durante la implementación de S02	Lectura del archivo tras el cambio	Se restauró desde Git y se rehízo la edición	45 pruebas aprobadas antes del commit `8479352`; el archivo dañado nunca se versionó

7.7 ¿Las pruebas detectan errores?

Las pruebas y el código se generaron con el mismo asistente, así que podrían compartir un mismo error. Para comprobarlo se introdujo **un error a la vez** en una copia del código y se ejecutó la batería completa. Una mutación que no hace fallar ninguna prueba indicaría una prueba complaciente.

Ejecución: 2026-10-08, sobre `de49d68`, 153 pruebas por mutación.

Tabla 88. *¿Las pruebas detectan errores?*

ID	Error introducido	Pruebas que fallan	Dónde
M1	Plazo de reapertura exclusivo (< en lugar de <=)	2	Integración, S04
M2	Prioridad invertida cuando no hay riesgo	7	S01, S02, S05
M3	El solicitante ve todas las incidencias	9	S04, S05
M4	No se comprueba el rol de la operación	14	S02, S03, S04, S05
M5	Los límites de texto cuentan los espacios externos	3	S01, S03, S04
M6	El porcentaje se trunca en lugar de redondear	2	S05
M7	El rechazo no guarda el motivo	3	Integración, S04, S05
M8	El técnico puede cerrar una incidencia	31	Integración, S02, S03, S04, S05

Las pruebas agregadas en `6b99fec` se comprobaron igual, ejecutando solo los archivos que vigilan cada propiedad:

Tabla 89. *¿Las pruebas detectan errores? (2)*

ID	Error introducido	Pruebas ejecutadas	Fallan
M9	La transacción confirma en lugar de revertir cuando hay un error	`test_atomicidad.py`, `test_s01_registro.py`	7 de 32 (las seis transiciones y el registro)
M10	El consecutivo se calcula fuera de la transacción, con una pausa de 10 ms	`test_concurrencia.py`	5 de 5
M11	Los límites de texto no retiran los espacios externos	Casos `p_s04_15`	4 de 6
M12	El riesgo acepta mayúsculas y espacios	Casos `p_s01_12`	3 de 6

En M11 y M12 los casos que no fallan son los que el error introducido tampoco altera: una cadena de 9 espacios y tabulaciones sigue por debajo del mínimo, y `1`, on y `si` siguen sin ser `true`.

Resultado: las doce mutaciones fueron detectadas. Es una muestra de doce errores, no una prueba de mutación exhaustiva.

7.8 Límites de las pruebas nuevas

- **Concurrencia.** Son ocho hilos dentro de un mismo proceso, cada uno con su sesión y su conexión, contra un archivo SQLite; no es una prueba de carga ni usa varios procesos servidores. Sobre `6b99fec` el archivo se ejecutó en 13 corridas (una sola, diez repetidas, una con los archivos en otro orden y la batería completa), es decir, 65 veces el escenario de ocho hilos, sin ningún fallo. Una versión anterior de este documento decía 55; el recuento correcto es este. La mutación M10 necesitó una pausa artificial de 10 ms para abrir la ventana de carrera.
- **Atomicidad.** El fallo se inyecta en el guardado del evento, que es el último paso de cada transacción; no se simulan cortes de energía ni fallos del disco.
- **Espera por bloqueo.** Un escritor espera el bloqueo de la base hasta 5 segundos, que es el valor por defecto de `sqlite3`; con más carga de la probada, un registro podría fallar con error en lugar de esperar. No dejaría datos parciales, pero ese caso no está probado.

8. Riesgos

Versión 0.2 · 2026-10-08 · evidencias sobre el commit `6b99fec` (176 pruebas aprobadas).

Cuatro riesgos: dos relacionados con la IA y dos con el producto. Cada control es algo que existe en el repositorio y se puede ejecutar; una advertencia en un prompt no cuenta como control.

Los responsables son una **propuesta del asistente** basada en la propuesta de revisores. Ningún integrante la ha aceptado todavía: es una decisión del equipo (ver decisiones, categoría B).

8.1 R1 (IA) · La IA inventa una regla de negocio

Tabla 90. *R1 (IA) · La IA inventa una regla de negocio*

Aspecto	Detalle
Riesgo	El asistente agrega una transición o un permiso que el caso no tiene; por ejemplo, que el técnico o el coordinador puedan cerrar un reporte.
Causa	El modelo completa con lo que es habitual en otros sistemas de tickets, no con lo que dice el enunciado.
Ocurrió	Sí. El 2026-10-08 una revisión generada con IA propuso “aclarar que solo el coordinador cierra”, lo contrario de RF04. Ver revisión asistida, sugerencia 7.
Impacto	Incidencias cerradas sin que el solicitante compruebe la solución: justo el problema que el cliente quiere eliminar.
Control 1	Las transiciones y el rol que ejecuta cada una están en un solo lugar, `dominio.TRANSICIONES`; una operación que no esté allí no existe.
Control 2	Criterios que prohíben el cierre por otros roles: AC-S03-07 y AC-S04-04, con pruebas.
Control 3	Toda sugerencia de IA se contrasta con el enunciado, el código y las pruebas antes de aceptarla, y el resultado queda escrito. Así se trataron las 13 sugerencias de la primera revisión y las 24 observaciones de la segunda.
Responsable propuesto	Cristian David Diaz España (revisor propuesto de S04). Sin confirmar.
Verificación	`test_p_s04_05_solo_el_duenio_valida` y `test_no_hay_rutas_para_editar_o_borrar` aprobadas. Mutación M8 (permitir que el técnico cierre): **31 pruebas fallan**.

8.2 R2 (IA) · Las pruebas aceptan el mismo error que el código

Tabla 91. *R2 (IA) · Las pruebas aceptan el mismo error que el código*

Aspecto	Detalle
Riesgo	El asistente escribe el código y sus pruebas con la misma interpretación equivocada, y todo pasa en verde.
Causa	Mismo autor para ambos; esperados calculados con la función que se está probando; pruebas que solo miran el código de respuesta.
Impacto	Falsa confianza: un defecto llega a la entrega con un reporte de pruebas “aprobado”.
Control 1	Los resultados esperados se escriben en las SPECS antes que la prueba y en las pruebas son valores literales, no llamadas a producción. En la segunda revisión los criterios AC-S01-10, AC-S02-10, AC-S03-09 y AC-S04-11 se escribieron (SPECS v0.3) antes de que existieran sus pruebas.
Control 2	Las pruebas comprueban los efectos en la base (estado, técnico, soluciones, cierres, eventos), no solo la respuesta HTTP. Las de atomicidad comparan las cinco tablas fila a fila.

Aspecto	Detalle
Control 3	Prueba de mutación: se introduce un error a la vez y se verifica que alguna prueba falle.
Control 4	La matriz SPEC/código se llena criterio por criterio, lo que revela criterios sin prueba.
Control 5	Una segunda revisión con otro asistente (ChatGPT) sobre las SPECS, contrastada después con el código.
Responsable propuesto	Jorge Luis González Arroyo (revisor propuesto de S05). Sin confirmar.
Verificación	Validación, sección 6: doce mutaciones (M1 a M12), las doce detectadas. La matriz reveló el hallazgo H-03 y la segunda revisión el H-05; ambos se cerraron con pruebas nuevas (`de49d68`, `6b99fec`) sin que apareciera ningún defecto del código.
Límite	Código, pruebas y contraste siguen viniendo de asistentes de IA. La revisión de otro integrante, que es el control que el examen exige, no se ha hecho.

8.3 R3 (producto) · Un usuario accede a reportes ajenos

Tabla 92. *R3 (producto) · Un usuario accede a reportes ajenos*

Aspecto	Detalle
Riesgo	Un solicitante lee, confirma o reabre la incidencia de otro, o un técnico atiende una que no tiene asignada, cambiando el código en la dirección o enviando la petición a mano.
Causa	Confiar en que la interfaz oculta los botones; comprobar el rol pero no la pertenencia.
Impacto	Exposición de reportes de terceros y cierres hechos por quien no corresponde.
Control 1	La identidad sale de la sesión, nunca del formulario. La sesión guarda solo el identificador y el rol se lee de la base en cada petición.
Control 2	La condición de visibilidad está en un único punto, `consultas._visibilidad`, y la usan el listado, el detalle y todas las operaciones.
Control 3	Lo que no es visible responde 404, sin revelar que existe (supuesto Q1).
Responsable propuesto	José Leonardo Hernández Pedrosa (revisor propuesto de S03). Sin confirmar.
Verificación	`test_p_s05_01_detalle_segun_permisos` (8 casos), `test_un_filtro_no_amplia_la_visibilidad`, `test_p_s03_03_tecnico_no_asignado`, `test_p_s04_05_solo_el_duenio_valida`. Mutación M3 (el solicitante ve todo): **9 pruebas fallan**.
Límite	Los formularios no llevan token CSRF; se declara como limitación conocida y mejora opcional.

8.4 R4 (producto) · Historial engañoso tras una operación fallida

Tabla 93. *R4 (producto) · Historial engañoso tras una operación fallida*

Aspecto	Detalle
Riesgo	Una operación rechazada o interrumpida deja un evento sin cambio de estado, un cambio de estado sin evento, o alguien altera el historial después.
Causa	Guardar estado e historial en pasos separados; validar después de escribir; permitir `UPDATE` o `DELETE` sobre el historial.
Impacto	El historial deja de ser prueba de lo ocurrido, que es la razón de ser del sistema.
Control 1	Orden fijo: rol, visibilidad, estado y datos se comprueban **antes** de abrir cualquier escritura.

Aspecto	Detalle
Control 2	Cada operación es una transacción (`BEGIN IMMEDIATE` … `COMMIT`, o `ROLLBACK` ante cualquier error) que guarda estado, registros y evento juntos.
Control 3	Disparadores en la base que abortan `UPDATE` y `DELETE` sobre `eventos`, `soluciones` y `cierres`. No existe ninguna ruta para editar o borrar.
Responsable propuesto	Rafael Eduardo May Recuero (revisor propuesto de S01 y del diseño). Sin confirmar.
Verificación	`test_las_operaciones_rechazadas_no_dejan_rastro` (siete rechazos; eventos idénticos fila a fila), `test_p_s05_04_el_historial_es_inmutable`. Atomicidad de las siete operaciones que escriben: `test_sin_efectos_parciales_si_falla_el_evento` (registro) y `test_un_fallo_al_guardar_el_evento_revierte_toda_la_operacion` (asignar, iniciar, solución, confirmar, rechazar y reabrir; 6 casos aprobados en `6b99fec`). Mutación M9 (la transacción confirma en lugar de revertir): **7 pruebas fallan**. Mutación M7 (rechazo sin motivo): **3 pruebas fallan**.
Límite	El fallo se inyecta en el guardado del evento, que es el último paso de la transacción. No se simulan cortes de energía ni fallos del disco.

8.5 Riesgos residuales declarados

No forman parte de los cuatro riesgos analizados. Se registran para que la entrega no afirme más de lo que se probó.

Tabla 94. *Riesgos residuales declarados*

ID	Riesgo residual	Qué se sabe	Qué no se probó
RR-1	**Espera de bloqueo de SQLite con más carga.** Un escritor espera el bloqueo de la base hasta 5 segundos, el valor por defecto del módulo `sqlite3`. Si muchas operaciones de escritura coinciden, alguna podría agotar esa espera y responder con error.	La transacción no llega a abrirse o se revierte, así que no quedarían datos parciales. Con ocho registros simultáneos no ocurrió en ninguna de las ejecuciones.	No hay prueba con más de ocho escritores, con varios procesos servidores ni de carga sostenida. El caso de agotamiento de la espera no está probado.
RR-2	**Alcance de la prueba de concurrencia.** `test_p_s01_11_registros_simultaneos` usa ocho hilos de un mismo proceso contra un archivo SQLite.	Aprobó en todas sus ejecuciones (65 veces el escenario sobre `6b99fec`, más las de la validación final). La mutación M10 la hace fallar, pero necesitó una pausa artificial de 10 ms para abrir la ventana de carrera.	No es una prueba de carga ni una garantía general de concurrencia. Solo cubre el registro; las transiciones simultáneas sobre una misma incidencia se protegen con la condición sobre el estado de origen, sin prueba con hilos.
RR-3	**Sin token CSRF.**	La cookie de sesión usa `SameSite=Lax` y las operaciones que cambian datos exigen `POST`.	No hay defensa específica ni prueba contra una petición falsificada desde otro sitio. Queda como mejora opcional, no implementada por decisión del autor.

8.6 Situaciones abiertas del trabajo del equipo

Deben declararse en la entrega si siguen así:

- La revisión de las SPECS por otro integrante no se había hecho cuando se escribió el código; el historial de Git lo muestra. El 2026-10-08 el autor integró todas las ramas en `main` sin que el pull request #2 hubiera recibido ninguna revisión ni comentario.

- Al 2026-10-08 todos los commits son de un solo integrante.
- Las tres preguntas al cliente no tienen respuesta.
- Los responsables de estos riesgos son una propuesta sin confirmar.

9. Entrega reproducible

9.1 Repositorio y versión evaluada

Tabla 95. *Repositorio y versión evaluada*

Dato	Valor
Repositorio	https://github.com/LUCHOMAN0529/reparacampus
Acceso	Público
Tag	release-examen
SHA exacto	b59faa1128708c5cc405d0b315c0226c610c5489
Resultado de las pruebas sobre ese SHA	176 pruebas aprobadas y 0 fallidas (python -m pytest, 2026-10-08, commit b59faa1)

9.2 Instalación desde una copia limpia

En Windows (PowerShell):

```
git clone https://github.com/LUCHOMAN0529/reparacampus.git
cd reparacampus
git checkout release-examen
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

En Linux o macOS la activación es `source .venv/bin/activate`.

9.3 Arranque y seed

```
flask --app app run
```

Se abre en http://127.0.0.1:5000. En el primer arranque la aplicación crea `instance/reparacampus.sqlite` con el esquema y las cinco cuentas ficticias. Los datos permanecen al reiniciar. `flask --app app init-db` crea la base sin arrancar el servidor y no borra datos existentes.

9.4 Cuentas ficticias

Tabla 96. *Cuentas ficticias*

Usuario	Contraseña	Rol
`solicitante1`	`Repara-sol1`	Solicitante
`solicitante2`	`Repara-sol2`	Solicitante
`coordinador1`	`Repara-coo1`	Coordinador
`tecnico1`	`Repara-tec1`	Técnico
`tecnico2`	`Repara-tec2`	Técnico

9.5 Comandos de pruebas

```
python -m pytest
python -m pytest -m integracion
python -m pytest -v
```

El primero ejecuta la batería completa; el segundo, solo las tres pruebas de integración; el tercero muestra el resultado prueba por prueba.

9.6 Limitaciones conocidas

- Las tres preguntas al cliente (Q1 a Q3) no tienen respuesta; el prototipo aplica los supuestos declarados en la sección 3.
- Los formularios no llevan token CSRF. La cookie de sesión usa `SameSite=Lax` y las operaciones que cambian datos exigen `POST`.
- El servidor de desarrollo de Flask no es apto para producción; el caso no pide hosting.
- No hay gestión de usuarios: las cuentas se crean solo con el seed.
- Sin paginación ni búsqueda por texto en el listado.
- Las fechas se muestran en UTC.

El README del repositorio conserva el procedimiento completo.

Después del tag, `main` puede tener documentos actualizados (comentarios de los integrantes y aclaraciones de las SPECS), pero el código y las pruebas son idénticos a los del tag.

10. Contribuciones y cierre

10.1 Contribuciones por integrante

Estado al 2026-10-08, tomado del historial de Git y de GitHub. Lo que no se ha hecho figura como pendiente.

Tabla 97. *Contribuciones por integrante*

Integrante	Tareas	Artefactos y commits	Revisión realizada	Resultado
Luis Carlo Daza Ospino	SPECS, diseño, implementación, pruebas, evidencias e integración, con asistencia de IA	`specs/`, `docs/`, `app/`, `tests/`; todos los commits del repositorio son de su autoría	No aplica: es el autor	176 pruebas aprobadas y 0 fallidas (python -m pytest, 2026-10-08, commit b59faa1)
Rafael Eduardo May Recuero	Propuesta: revisión de S01, del alcance y del diseño	Ninguno registrado	Pendiente	Pendiente
Jean Marco Oyola De Martino	Propuesta: revisión de S02	Ninguno registrado	Pendiente	Pendiente
José Leonardo Hernández Pedrosa	Revisión de S03	Respuesta escrita a la encuesta de revisión (2026-10-13), apoyada en ChatGPT según declara	Sí: comentario sobre S03, sin decisión de aprobación	Dos aclaraciones incorporadas en S03 v0.6; una observación ya cubierta; una mejora registrada
Cristian David Diaz España	Propuesta: revisión de S04	Ninguno registrado	Pendiente	Pendiente
Jorge Luis González Arroyo	Propuesta: revisión de S05	Ninguno registrado	Pendiente	Pendiente

La asignación de revisores es una propuesta que el equipo no ha confirmado. Los comentarios recibidos se transcriben tal cual en `docs/revision/comentarios-integrantes.md`. El requisito de que cada integrante aporte una contribución verificable y revise al menos un cambio de otra persona **no está cumplido** a esta fecha.

10.2 Integración

El 2026-10-08 el autor integró en `main` las ocho ramas de trabajo, en orden de dependencia y sin conflictos: `specs/s01-s05` (pull request #2), `feat/base`, `feat/s01-registrar`, `feat/s02-priorizar-asignar`, `feat/s03-atender`, `feat/s04-validar`, `feat/s05-consultar-historizar` y `docs/evidencias`. Las ramas se conservan. Ningún pull request fue revisado por otro integrante antes de integrarse.

10.3 Conclusión

Cumplimiento. El prototipo permite autenticarse con las cinco cuentas ficticias y ejecutar registro, prioridad, asignación, atención, validación, reapertura, filtros, tablero e historial según permisos. Los 53 criterios de aceptación de las cinco SPECS están en “Cumple”. Resultado de la batería sobre el tag de entrega: 176 pruebas aprobadas y 0 fallidas (python -m pytest, 2026-10-08, commit b59faa1). Doce errores introducidos a propósito fueron detectados por las pruebas.

Pendientes y desviaciones que se declaran.

- El código se escribió y se integró sin que otro integrante revisara las SPECS ni los cambios. El historial de Git muestra ese orden.
- Todos los commits son de un solo integrante.
- Las tres preguntas al cliente quedaron sin respuesta; se trabajó con supuestos declarados.
- El recorrido manual en navegador con los cinco usuarios no lo ha hecho ninguna persona. Existe un recorrido automatizado por HTTP contra el servidor real, descrito en la sección de validación.
- La concurrencia se probó con ocho registros simultáneos en un mismo proceso contra SQLite; no hay prueba de carga. Con más carga, un registro podría fallar por la espera del bloqueo, sin dejar datos parciales.
- Los formularios no llevan token CSRF.
- Los responsables de los riesgos y los revisores son propuestas sin confirmar por el equipo.
- El tablero de GitHub Projects y los issues 2 a 8 no se crearon; solo existe el issue #1.

Aceptación del equipo. Pendiente. Ningún integrante distinto del autor ha registrado su aceptación. El detalle de lo confirmado y lo pendiente está en `docs/decisiones.md`.

10.4 Comentarios de los integrantes sobre las SPECS

Cada integrante leyó la parte del documento de entrega que le correspondía y respondió la encuesta de revisión. Las respuestas se copian **tal como las escribió cada persona**, sin corregirlas. Después de cada una se indica qué se hizo con ella.

Los comentarios llegaron después de que el código estuviera escrito e integrado, así que no cuentan como revisión previa a la implementación.

Tabla 98. *Comentarios de los integrantes sobre las SPECS*

Integrante	Parte	Recibido	Estado
Rafael Eduardo May Recuero	S01 Registrar	—	Sin respuesta
Jean Marco Oyola De Martino	S02 Priorizar y asignar	—	Sin respuesta
José Leonardo Hernández Pedrosa	S03 Atender	2026-10-13, entregado al autor	Recibido
Cristian David Diaz España	S04 Validar	—	Sin respuesta
Jorge Luis González Arroyo	S05 Consultar y tablero	—	Sin respuesta

10.4.1 José Leonardo Hernández Pedrosa · S03 Atender una incidencia

Recibido el 2026-10-13. El integrante declara que usó ChatGPT como apoyo.

1. Con tus palabras: ¿qué hace esta parte del sistema?

> *Esta parte permite gestionar una incidencia asignándole un técnico y actualizando su estado. También permite registrar la solución del problema para llevar un control de la atención hasta que se resuelva.*

2. ¿Hay algo que no se entienda o que te parezca mal definido? ¿Qué?

> *Me parece que se podría explicar mejor qué sucede cuando una incidencia no puede resolverse y qué condiciones se deben cumplir para cerrarla. Así sería más claro cómo debe actuar el sistema en cada situación.*

3. ¿Cambiarías o agregarías algo? ¿Qué y por qué?

> *Agregaría mensajes de confirmación cuando se actualice una incidencia y validaciones para evitar guardar información incompleta. Esto ayudaría a prevenir errores y facilitaría el uso del sistema.*

4. ¿Usaste IA para revisarla? Si sí: ¿cuál, y con qué parte de su respuesta estás de acuerdo y con cuál no?

> *Sí, utilicé ChatGPT como apoyo para revisar esta parte. Estoy de acuerdo con la idea de mejorar las validaciones y los mensajes de confirmación, porque ayudan a evitar errores. Sin embargo, considero que las sugerencias deben compararse con los requisitos del proyecto antes de aplicarlas.*

Qué se hizo con cada observación

Tabla 99. *Qué se hizo con cada observación*

Observación	Contraste con el enunciado y el código	Acción
Explicar qué pasa si una incidencia no puede resolverse	El enunciado no define un estado para ese caso y prohíbe agregar transiciones. En el código, la incidencia simplemente sigue `EN_ATENCION` hasta que el técnico registra una solución.	**Aceptada como aclaración.** S03 v0.6 lo dice de forma explícita en «Alcance y exclusiones». No cambia ningún requisito ni el código.
Explicar qué condiciones se deben cumplir para cerrarla	Está definido en S04: solo el solicitante dueño cierra, confirmando una solución en `PENDIENTE_VALIDACION`. S03 no lo remitía a S04.	**Aceptada como aclaración.** S03 v0.6 remite a S04.
Validaciones para no guardar información incompleta	Ya existen: la solución exige de 20 a 800 caracteres después de retirar espacios (AC-S03-05) y una operación inválida no guarda nada (AC-S03-09). Pruebas aprobadas: `test_p_s03_05_limites_de_la_solucion`, `test_un_fallo_al_guardar_el_evento_revierte_toda_la_operacion`.	Ya cubierta. Sin cambios.
Mensajes de confirmación al actualizar una incidencia	Hoy, tras una operación exitosa, la página vuelve al detalle con el estado nuevo y el evento en el historial, pero sin un mensaje explícito. El enunciado no lo exige.	**Registrada como mejora opcional, no implementada.** Implementarla cambiaría código ya etiquetado y probado.

Su observación de que «las sugerencias deben compararse con los requisitos del proyecto antes de aplicarlas» coincide con el criterio que se siguió para las revisiones asistidas por IA.

El integrante no expresó una decisión de aprobar o rechazar la SPEC; por eso la tabla «Decisión de revisión» de S03 registra el comentario, no una aprobación.

10.5 Decisiones: confirmadas y pendientes

Versión 0.2 · 2026-10-08.

Este registro separa lo que ya decidió una persona de lo que solo ha propuesto el asistente de IA. Una propuesta del asistente, o algo conversado solo con el autor, **no** es una decisión del equipo ni de la docente.

10.5.1 Decisiones tomadas por el autor (Luis Carlo Daza Ospino)

Constan en la conversación de trabajo con el asistente el 2026-10-08. Las instrucciones de las etapas llegaron como texto pegado en esa conversación.

Tabla 100. *Decisiones tomadas por el autor (Luis Carlo Daza Ospino)*

N.º	Decisión	Evidencia	Efecto en el repositorio
D-01	Continuar el trabajo con supuestos, porque la docente no puede ser consultada antes del cierre.	Nota escrita al final del documento de revisión que entregó: «continuaremos haciendo el trabajo creando nosotros mismos el trabajo». Es una nota dentro de un documento, no un mensaje directo.	Preguntas Q1 a Q3 «sin respuesta» en `specs/00-problema-y-alcance.md`.
D-02	Pedir a los compañeros su opinión con un formato.	«dame un documento con el espacio para que ellos escriban lo que les parece».	`docs/revision/Formato_revision_SPECS.docx`.
D-04	Hacer una segunda revisión de las SPECS con ChatGPT y contrastarla con el código.	«La revisión documental […] se realizó en una conversación con ChatGPT»; «observaciones documentales que debes contrastar».	`docs/revision/revision-asistida-ia-02-chatgpt.md`.
D-05	Trabajar por etapas autorizadas una a una: correcciones documentales, pruebas y evidencias.	«Autorizo avanzar por etapas».	Commits `46e8995`, `83861a7`, `3d283d4`, `6b99fec`, `925d22d`, `1335535`, `688972d`, `984669f`.
D-07	No implementar el token CSRF; queda como mejora opcional y limitación declarada.	«No agregues CSRF todavía; déjalo como mejora opcional»; «Declara honestamente las limitaciones […] incluida la ausencia de protección CSRF si no se implementa».	Limitación declarada en README, arquitectura y riesgos.
D-08	No marcar ninguna revisión humana como hecha ni atribuir aprobaciones a compañeros.	«No marques revisiones humanas como completadas»; «No inventes revisiones, aprobaciones, contribuciones».	«Decisión de revisión: Pendiente» en todas las SPECS.
D-09	No cambiar código de producción para hacer pasar una prueba, ni cambiar una SPEC para que coincida con un resultado fallido.	Reglas de la Etapa 2 y de la instrucción de cierre.	`app/` no cambia desde `786cf42`.
D-10	Usar PlantUML, descargado de su repositorio oficial, para exportar los diagramas.	Respondió «si» a la descarga.	`docs/diseno/img/`.

N.º	Decisión	Evidencia	Efecto en el repositorio
D-11	Integrar todas las ramas en `main`, repetir la validación, generar el documento de entrega y crear el tag `release-examen`, sin esperar las revisiones de los compañeros y declarando que faltan. Reemplaza la indicación anterior de no integrar (D-06 en la versión 0.1 de este registro).	Instrucción de cierre: «integres el proyecto, ejecutes las verificaciones finales y prepares los archivos de entrega».	Fusiones `87f4572` a `8130550`; tag `release-examen`.

Corrección respecto de la versión 0.1. Allí figuraba como D-03 «seguir con la implementación sin esperar la revisión de las SPECS». El autor nunca lo dijo de forma expresa: el asistente lo dedujo. Pasa a la lista A como A-8.

10.5.2 A. Decisiones que el autor puede confirmar individualmente

Son propuestas del asistente o hechos que solo necesitan la palabra del autor. **Siguen sin confirmar.**

Tabla 101. *A. Decisiones que el autor puede confirmar individualmente*

N.º	Qué hay que decidir	Propuesta del asistente	Dónde queda registrado
A-1	Qué hacer con las 13 sugerencias de la primera revisión simulada.	Aceptar 4, dar 8 por cubiertas o aclaradas y descartar 1 por errónea («solo el coordinador cierra»).	`docs/bitacora-ia.md`, intervención 2
A-2	Si acepta el código generado.	Aceptarlo.	Intervención 3
A-3	Si acepta las pruebas y la comprobación por mutación.	Aceptarlas.	Intervenciones 4 y 5
A-4	Con qué herramienta se generó el primer borrador de revisión (el PDF «simulado»).	Ninguna: es un dato que solo conoce el autor.	Intervención 2
A-5	Si figura como responsable de las 32 tareas, con la nota «con IA».	Sí, porque todos los commits son suyos.	`specs/*/tasks.md`
A-6	Si mantiene los supuestos SUP-12 (solución vigente), SUP-13 (desempate del historial) y SUP-14 (registros simultáneos, con su alcance).	Mantenerlos: describen lo que el código hace y las pruebas confirman en el alcance probado.	`specs/00-problema-y-alcance.md`
A-7	Si los códigos de los integrantes van en la portada. El enunciado los pide; el autor dijo que no hacían falta.	Incluirlos, por ser un requisito de la portada.	`docs/entrega/datos.json`
A-8	Reconocer que el código se escribió antes de que otro integrante revisara las SPECS.	Declararlo como desviación de la regla central del examen; ya está declarado así en la entrega.	`docs/entrega/10-contribuciones-y-cierre.md`

10.5.3 B. Decisiones que debe acordar el equipo

Ninguna está tomada. El asistente no puede decidirlas, ni el autor en nombre de los demás.

Tabla 102. *B. Decisiones que debe acordar el equipo*

N.º	Qué hay que decidir	Estado actual	Quién
B-1	Revisión de cada SPEC con fecha, versión y decisión.	S03: comentario de José Leonardo recibido el 2026-10-13, sin decisión de aprobación. Pendiente en S01, S02, S04, S05, el alcance y el diseño. El pull request #2 se integró sin revisiones.	El revisor propuesto de cada una: Rafael (S01 y diseño), Jean Marco (S02), José Leonardo (S03), Cristian (S04), Jorge Luis (S05)
B-2	Si la asignación de revisores es esa.	Es una propuesta que proviene de un borrador generado con IA; ningún integrante la ha aceptado.	Equipo
B-3	Responsables funcionales de los cuatro riesgos.	Propuesta del asistente, sin confirmar.	Equipo
B-4	Revisor de cada tarea.	«Por asignar» en las 32 tareas.	Equipo
B-5	Aporte verificable de cada integrante y revisión de al menos un cambio de otra persona.	Solo hay commits de un integrante.	Cada integrante
B-6	Nombre del equipo y grupo para la portada.	Sin dato.	Equipo
B-7	Aceptación final del trabajo y pendientes que se declaran.	Sin dato.	Equipo
B-8	Si se agrega el token CSRF.	No implementado; mejora opcional.	Equipo, a propuesta del autor

Cómo registrar una revisión

1. Abrir el formato `docs/revision/Formato_revision_SPECS.docx` y la SPEC correspondiente en `specs/`.
2. Llenar la sección de esa SPEC con nombre, fecha, versión revisada (0.5) y decisión.
3. Dejar constancia propia en GitHub: un comentario en el issue #1 o un commit que llene la tabla «Decisión de revisión» de la SPEC. Así queda también el aporte verificable de quien revisa.

10.5.4 C. Decisiones que dependen de la docente o de una validación externa

Tabla 103. *C. Decisiones que dependen de la docente o de una validación externa*

N.º	Qué hay que resolver	Supuesto con el que se trabaja
C-1	Q1: ante una incidencia ajena, ¿403 o 404?	404, sin revelar que existe.
C-2	Q2: ¿el porcentaje de cierre usa el estado actual? ¿cómo se redondea?	Estado actual; redondeo aritmético a un decimal.
C-3	Q3: precisión del plazo de 48 horas.	Exactamente 48 horas se acepta; un microsegundo después se rechaza.
C-4	Si los supuestos SUP-01 a SUP-14 son aceptables para el cliente.	Se declaran uno por uno.
C-5	Si es aceptable que la revisión de las SPECS no haya ocurrido antes del código ni de la integración.	Se declara como desviación de la regla central.
C-6	Si las revisiones asistidas por IA cuentan como evidencia complementaria.	Se presentan como intervenciones de IA, no como revisión de un integrante.
C-7	Nombre de la docente y grupo para la portada.	Sin dato.

11. Referencias

Fuentes consultadas para el flujo de desarrollo guiado por especificaciones. Solo se listan las que realmente se abrieron.

Tabla 104. *Referencias*

Fuente	Enlace	Consultada	Para qué se usó
Enunciado del examen: *Especifica. Construye. Verifica.* Universidad Simón Bolívar, Ingeniería de Software I	https://examen-isi-specs-abril.ing-llanos-bravo.chatgpt.site	2026-10-08	Requisitos RF01 a RF05, reglas, plantillas y rúbrica.
GitHub Spec Kit, *Spec-Driven Development Quickstart*	https://github.github.com/spec-kit/quickstart.html	2026-10-08	Flujo especificar → aclarar → planear → tareas → implementar → converger, y los archivos `spec.md`, `plan.md` y `tasks.md`.
Kiro, *Specs*	https://kiro.dev/docs/specs/	2026-10-08	Flujo en tres fases: requisitos, diseño y tareas.

11.1 Correspondencia con este repositorio

El equipo usó Markdown versionado, sin instalar Spec Kit ni Kiro. La estructura equivale así:

Tabla 105. *Correspondencia con este repositorio*

Paso de Spec Kit	Fase de Kiro	En ReparaCampus
specify	Requisitos (`requirements.md`)	`specs/S0x-*/spec.md`: historia, entradas, reglas y criterios de aceptación
clarify	Requisitos	Preguntas al cliente y supuestos en `specs/00-problema-y-alcance.md`
plan	Diseño (`design.md`)	`specs/S0x-*/plan.md` y `docs/diseno/`
checklist	—	Formato de revisión en `docs/revision/`
tasks	Tareas (`tasks.md`)	`specs/S0x-*/tasks.md`
analyze	—	Revisión de consistencia entre SPEC, plan y tareas
implement	Ejecución de tareas	Una rama y un commit por SPEC
converge	—	Matriz SPEC frente a código en `docs/validacion/validacion.md`

Las consultas a estas dos guías se hicieron después de escribir las SPECS; sirvieron para comprobar la correspondencia, no para generarlas.